Onlinevarsity
your e-way to learning

Android Application Testing

Are you registered with **Onlinevarsity.com**?	Yes ●	No ●
Did you download this book from **Onlinevarsity.com**?	Yes ●	No ●

Scores

For each **YES** you score **50**
For each **NO** you score **0**
If you score less than 100 this book is illegal.
Register on **www.onlinevarsity.com**

Android Application Testing

© 2016 Aptech Limited

All rights reserved.

No part of this book may be reproduced or copied in any form or by any means – graphic, electronic or mechanical, including photocopying, recording, taping, or storing in information retrieval system or sent or transferred without the prior written permission of copyright owner Aptech Limited.

All trademarks acknowledged.

APTECH LIMITED
Contact E-mail: ov-support@onlinevarsity.com

Edition 1 - 2016





Preface

The book, Android Application Testing, aims to teach the basics of testing Android applications developed for Android enabled devices. The book introduces the developer to the most commonly available techniques, frameworks, and tools to improve the development of Android applications.

Android programming has undergone several reformations since its inception. This book intends to familiarize the reader with the techniques available for testing Android applications. The book begins with an introduction to the basic concepts of testing Android applications using Eclipse IDE. The book proceeds with the explanation of the various Test case classes that can be utilized for testing Android applications and also provides a clear, step-by-step instruction to show how to write tests for your application. This will enable the developer to create professional Android applications. The book concludes with an explanation of testing the different components in an Android application.

This book is the result of a concentrated effort of the Design Team, which is continuously striving to bring you the best and the latest in Information Technology. The process of design has been a part of the ISO 9001 certification for Aptech-IT Division, Education Support Services. As part of Aptech's quality drive, this team does intensive research and curriculum enrichment to keep it in line with industry trends.

We will be glad to receive your suggestions. Please send us your feedback, addressed to the Design Centre at Aptech's corporate office, Mumbai.

Design Team

Onlinevarsity



Table of Contents

Sessions

1. Introduction to Software Testing
2. Android Testing Framework
3. Components Used for Creating Tests
4. Android Testing Environment
5. Testing Android Projects
6. Testing for Different Situations



MANY COURSES ONE PLATFORM

01 Introduction to Software Testing



Welcome to the Session, **Introduction to Software Testing**.

This session explains the concept of software testing. It describes the purpose and need for testing. The session also introduces the advantages of testing. It lists the various types of testing with the advantages and disadvantages of each.

In this Session, you will learn to:

- Explain the concept of testing
- Explain the need and purpose for testing
- Describe the advantages of testing
- List the different types of testing

Session

01

Introduction to Software Testing

1.1 Introduction

Software systems are prone to errors which may crop up in any process. Some of the errors can be ignored while others can break the entire system or software. Such errors should be handled in time before the software is deployed. No matter how much time is invested in design and development of software program, mistakes are inevitable and bugs will appear. Software application development and mistakes are closely related with each other.

This session explains software testing which is an activity aimed for evaluating the quality of the program and improvising it by identifying the defects and problems. Finally, the session explains the different types of software testing.

1.2 Overview of Testing

Testing is one of the most important activities in verification and validation process. It is a process to identify the correctness, completeness, and quality of the developed software. Its goal is to create a defect free product for customers by verifying and validating whether the software is working according to the expected specifications. A simple definition of testing is that it is a process of executing a program with an intention of finding error/s. There are certain true facts about testing. They are as follows:

- The number of possible inputs is very large
- The number of possible outputs is very large
- The number of execution paths is very large

With such a large number of inputs, outputs, and execution paths, it can be stated that a good test is one that has a high probability of finding errors in the shortest time and with minimum effort. A successful test is one that can uncover the errors quickly and effortlessly.

1.2.1 What is Testing?

Software testing in simple words is a quality check process. It is a process of checking the correctness and completeness of a product before being released to the end users. The objectives of software testing are as follows:

- Meets the expected design and development specifications
- Functions as expected
- Uses same characteristics for implementation
- Conforms to the needs of end users

Software testing can be done at different points in the development process. However, the different software models determine when the testing method can be employed. While in traditional models, testing occurs after the requirements are defined and the coding process is complete, in newer models a test-driven method is executed. Here, the developer is responsible for a major portion of the testing before it passes to a team of testers. Agile software development is one such model.

There are some limitations to software testing. They are as follows:

- Testing cannot deliver an error-free product as its purpose is to detect errors.
- It cannot identify all errors within software. Its function is to compare the state and behavior of the product against certain components. The components can be standards, specifications, applicable laws, products that can be compared, previous versions of the same product, inferences about intended and expected product goal, user requirements, and so on.
- It can only identify defects under controlled situations. It cannot ascertain that a product can function properly under all conditions.

Software testing consists of examination and execution of code under various conditions as well as checking of the code against specifications. In the current software development scenario, a testing team may be separate from the development team. The skills required for a tester are varied but a major trait that they need to have is to ask questions regarding the functioning of the software and the possibility of defects due to its functioning.

1.2.2 Need for Testing

In February 2003, the US Treasury Department sent out 50,000 Social Security checks. The problem that they encountered was that they did not verify that the checks were mailed without a beneficiary's name. The error was attributed to a software maintenance defect. This is just one of the many examples that could have been avoided with software testing.

Therefore, testing helps to solve three main purposes namely verification, validation, and defect finding. Each of this process is essential to any software development process to enable them to produce a quality product.

- **Verification process**: In this process the software is tested against the given technical specifications. A 'specification' can be described as a measurable output value based on a specific input value under specification conditions. Software technical reviews represent one such method for verifying various products. A specification review will attempt to verify the specification requirement against the requirement description and the customer requirements specification. An example of a simple specification is an SQL query to retrieve monthly data for a single account against a multi-month account-summary table. The objective is to return the result within 3 seconds of submitting the request.
- **Validation process**: This process is performed at the end of the development stage to ascertain that the software meets the business requirements. A business requirement may be to retrieve the income tax data from a monthly salary account summary table. The objective is to return the result containing columns of data within 3 seconds of submitting the request.
- **Defect finding**: The purpose of this process is to check the actual results against the expected one. A defect determines the variance in the results. The defect can be backtracked to a cause in the specification, design, or development phases.

1.2.3 History of Testing

Software testing initially started in the fifties. Testing was primarily debugging, where the objective was to identify and eliminate bugs in the software. Slowly, it moved to destruction testing, where the purpose was to break down the code to identify the gaps present in them. In 1979, Glenford J. Myers introduced a distinction between debugging and testing. Later, the advent of prevention oriented methodologies, where the intention was to satisfy specifications and detect and prevent errors, originated.

1.2.4 Advantages of Testing

The advantages of software testing are as follows:

- **Time-saving**: In general, manual testing takes up more time both during software and application development. Such time can be considerably lessened with automated tools provided the scripts are standard and non-complex.
- **Reliability**: Automated tests remove the possibility of error, especially when the same sequence of steps needs to be performed again and again.
- **Cost-saving**: Automated tests save money by identifying the defects at different stages of the testing process.
- **Development downtime**: A quick testing process avoids or reduces development time by reducing a computer's unavailability time to minimum.
- **Customer service**: Automated testing ensures that the customers' requirements are met by developing better applications.
- **Versions of the same software**: Testing identifies the areas for modifications and enhancements in later versions and catalogs reusable modules and components.
- **Training**: Another advantage of testing is identifying the training areas for programmers and developers.

1.3 Types of Testing

Software testing is of different types depending upon where they are introduced in the software development process. The different types of software testing are described here.

1.3.1 Unit Testing

Unit testing of a software application is done by the developer during the coding process. The objective of unit testing is to isolate each part of the program and verify that each individual part is correct. It ensures that the codes used by the software application work independently. Unit testing may also be known as component testing. In other words, unit testing verifies that the lowest independent entity in a software application is working correctly.

The advantages of unit testing are as follows:

- **Facilitates change**: With unit testing it is possible to refractor code at a later date to ensure that the module continues to work correctly. Test cases are written for all functions and methods, so that whenever fault occurs, it can be quickly identified and corrected.
- **Simplifies integration**: Testing parts of the program and then, the sum of the parts makes integration testing easier. It may lessen uncertainty in the units themselves and use a bottom-up testing style approach.
- **Documentation**: Each unit test consists of a documentation of its functionality and characteristics that define the appropriate/inappropriate use of the unit.
- **Design**: A unit-driven test approach can be viewed as a formal design of each unit that specifies the class, method, and observable behavior of that unit.

The drawbacks of unit testing are as follows:

- Unit testing cannot identify system-level errors or integration errors.
- It cannot show an error-free product but can only record the presence of defects.
- It is necessary to maintain records of the tests performed on each of the unit but also of all the modifications performed on the source code. Use of a version control system becomes essential to detect units that have failed the current version test though they have been cleared in the previous versions.
- It is also important to follow a sustainable process to address test case failures immediately. In the absence of such a process, the application may become out of sync with the unit test suite and record false positives, thereby reducing the effectiveness of the test.

1.3.2 Bottom-up Testing Strategy

The bottom-up testing strategy tests different subsystems in the hierarchy. In this testing strategy, the lowest layer of subsystems is tested individually. Following this, the next level of subsystems are tested that call for the previously tested subsystems. This is continued until all subsystems are included.

The advantages of using this strategy are as follows:

- **Stubs**: There is no need for stubs. A stub is a component that the TestUnit depends on for implementation, though partially and for fake values.
- **Other systems**: This strategy is useful for integration testing of others systems such as object-oriented systems, real-time systems, and systems with strict performance requirements.

The disadvantages of using this strategy are as follows:

- Drivers are required for this type of testing. A driver is a component that calls the TestUnits and controls the test cases.
- The User Interface (UI) is the most important subsystem that is tested last.

1.3.3 Top-down Testing Strategy

Contrary to the bottom-up testing strategy, the top-down testing strategy tests the top most layers or the controlling subsystem first. The remaining subsystems that are called by the tested subsystems are combined together and the resulting collection of subsystem is tested. This is continued until all subsystems are incorporated in the test. Stubs are required for this type of testing when the sub modules have not been developed. Stub is a temporary program.

The advantages of using this strategy are as follows:

- **Drivers**: Drivers are not required for this test.
- **Test cases**: These can be defined in terms of functional requirements.

The disadvantages of using this strategy are as follows:

- Stubs must define all possible conditions of testing so writing them may be difficult.
- In case the lowest level of the system has many methods, a large number of stubs maybe needed.
- It is not possible to test some interfaces separately.

1.3.4 Integration Testing

Integration testing, as the name suggests integrates the individual units (components) and tests it as a group. Its aim is to expose defects in the interaction between the components. Integration testing is sometimes, termed as I&T (Integration and Testing), String testing, or Thread testing. Integration testing is done after unit testing and is used for detecting the problems in the interface of the two units.

The advantages of using this strategy are as follows:

- **Time saving**: System testing becomes time consuming if integration testing is not performed.
- **Off-the-shelf components**: Many off-the-shelf components cannot be unit tested.
- **Deduction of errors**: Many system failures are avoided by reduction of errors during the interaction of the subsystems.

The disadvantages of using this strategy are as follows:

- Failures that are not deducted by integration testing will be found after the system is deployed and will prove to be expensive.
- Conditions that are not defined in the integration tests will not be tested.
- Conditions that do not confirm to the execution of design items are not tested.

1.3.5 Functional/Acceptance Testing

Functional testing refers to testing the software application in accordance with the specified requirements. It focuses on the whole output and ignores the internal components. In other words, it ensures that the functionality as specified in the system requirement works.

Acceptance testing is a type of testing that is usually done at the client's end. The purpose is to check whether the output meets the specified requirements and works as expected. Acceptance testing are of different kinds but the typical types are as follows:

- **User acceptance testing**: This type of testing is based on the location of the test-factory or site. Factory acceptance testing is conducted by factory users before moving the product or application to its destination site. Site acceptance testing is done by users at the site.
- **Operational Acceptance Test (OAT)**: This test is also referred to as the operational readiness testing. The test is done to check that the processes and procedures are in place for a system to be used and maintained. The checks include back-up facilities, disaster recovery procedures and end user training, maintenance, and security procedures.
- **Contract and regulation acceptance testing**: A system is considered to be accepted after it is tested against the acceptance specifications documented in a contract. In regulation acceptance testing the system is tested against government, legal, and safety standards.
- **Alpha and beta testing**: Alpha testing is testing of the software in a controlled setting at the developer's environment with the developer fixing errors immediately. Beta testing is testing done at the client's end in the absence of the developer. The software is tested in a realistic target environment and changes, if any are made based on the feedback before being released to the end user. This type of testing is also called 'field testing'.

The advantages of using this strategy are as follows:

- **Cost-saving**: Running tests manually is expensive compared to creating an automated test.
- **Customer support**: When a project passes acceptance test, future releases, and long-term contract support becomes worthwhile.

The disadvantages of using this strategy are as follows:

- There is a possibility of repeated execution changes from project to project and long-term support and maintenance issues.
- After a product is launched it needs to be maintained. Sometimes, the investment to set and maintain a test for the product life becomes expensive. It may, however be less costly to manually test the product when the need arises.

1.3.6 Performance Testing

This test assesses the speed and effectiveness of the software application as given in the performance requirements. It ensures that the results are generated within the specified time and determines the stability of the application under varying user loads.

Performance testing is done by generating some tasks or activity on the system and then, testing by using some performance tools.

There are many types of performance testing. They are as follows:

- **Stress testing**: evaluates the stress limits of the system.
- **Volume testing**: determines the working of the system when large data are fed.
- **Configuration testing**: tests various software and hardware configurations.
- **Compatibility testing**: estimates the compatibility with the existing systems.
- **Timing testing**: determines the response and the performance time to complete a function.
- **Security testing**: tests violation of security standards.
- **Environmental testing**: tests tolerance to heat, humidity, and motion.
- **Quality testing**: determines the reliability, maintainability, and availability.
- **Recovery testing**: estimates the time taken to respond to error detection and data loss.
- **Human factors testing**: tests done with end users.
- **Load testing**: determines the application's response to various load levels within its acceptable limits. The main parameter here is the response time.
- **Soak/endurance testing**: measures the application's ability to handle continuous load for an extended period of time. The main parameter here is memory.

The advantages of using this strategy are as follows:

- **System performance**: Eliminates system failure, late deployment, and rework due to performance issues. Also, removes avoidable system tuning efforts.
- **Overhead costs**: Avoids additional overhead operational costs for handling system issues due to performance failures.

The disadvantages of using this strategy are as follows:

- Testing is expensive when it is done at each phase of the SDLC.
- The whole system can be seen as useless or the defect removal cost higher if performance activities are done after complete development.
- It may also be difficult to rectify the performance defects after the development process.

1.3.7 System Testing

Testing of the complete and fully integrated software application with the full computer system is called system testing. It consists of a series of different tests to evaluate its compliance with specified requirements.

The major types of system testing are as follows:

- **Alpha testing**: Performed by the test team in a controlled setting in the developing environment.
- **Beta testing**: Conducted by a select group of users offering valuable feedback to the developer on the working of the system.
- **Acceptance testing**: Performed by the customer to check the acceptance of the system for delivery.

The advantages of using this strategy are as follows:

- **End-to-end testing**: Provides a complete testing where every single input and its outcome is verified.
- **User experience**: Tests the experience of the user in working with the application.

The disadvantages of using this strategy are as follows:

- This test requires detailed test cases and test suites to test each part of the application.
- The type of system test to take is dependent on various factors such as availability of time, resource, and budget, the tester's experience, and the company the tester works for. The method used by large companies is different than those used by small and medium sized companies.

Check Your Progress

1. The primary objective of prevention oriented testing methodologies is to:

(A)	satisfy specifications, detect, and prevent errors	(B)	identify and eliminate bugs
(C)	break code and identify gaps	(D)	identify gaps and satisfy specifications

2. The three main purposes of software testing are _____, _____, and _____.

(A)	verification, validation, and fault finding	(B)	verification, fault finding, and examination
(C)	verification, examination, and execution	(D)	validation, examination, and fault finding

3. Which of the following tests isolates and verifies the correctness of each individual part of the program?

(A)	Component testing	(B)	System testing
(C)	Functional testing	(D)	Integration testing

4. The goal of string testing is to:

(A)	integrate components and test it as a group	(B)	test each unit individually
(C)	check against specifications	(D)	focus on the whole application

5. Which one of this test is done by the client?

(A)	integration testing	(B)	performance testing
(C)	system testing	(D)	acceptance testing

6. Which one of the following test uses a stub for testing subsystems?

(A)	Top-down testing	(B)	Bottom-up testing
(C)	Alpha testing	(D)	Beta testing

Answers

1.	A
2.	A
3.	A
4.	A
5.	D
6.	A

Summary

- Software testing checks the correctness and completeness of the project before its release to end users.
- Unit testing tests each part of the program and verifies that each individual part is correct.
- The bottom-up testing strategy tests different subsystems in hierarchy.
- The top-down testing strategy tests the top most layers or the controlling subsystem first and then, moves down to testing the remaining subsystems.
- Integration testing integrates the individual units (components) and tests it as a group.
- Functional testing checks the software application in accordance with the specified requirements.
- Acceptance testing checks whether the output meets the specified requirements and works as expected.
- Performance testing assesses the speed and effectiveness of the software application as given in the performance requirements.
- System testing checks the complete and fully integrated software application with the full computer system.

THE WORLD HAS GONE DIGITAL. WHAT ABOUT YOU?



Onlinevarsity

YOUR DIGITAL WORLD OF LEARNING

02 Android Testing Framework



Welcome to the Session, **Android Testing Framework**.

This session describes the test structure and the Android Testing API. The concepts of JUnit and instrumentation are also discussed. Further, the session explains testing Android applications in Eclipse and Android Studio.

In this Session, you will learn to:

- Describe the Android test structure
- Explain the Android testing API
- Identify Android specific-testing classes
- Explain testing of Android applications in Eclipse and Android Studio

Session

02

Android Testing Framework

2.1 Introduction

The Android testing framework is an essential element of the development environment. It constitutes the basic architecture and tools that help in testing every aspect of an application at various levels.

Android test suites are based on JUnit, an open source unit testing framework for Java programming language. The Android testing framework has a set of control methods that call for lifecycle methods (onCreate, onResume, onPause, and so on) of components to show how the application will behave under certain conditions. The Android SDK (Software Development Kit) also provides other test tools for stress testing of the application.

2.2 Test Structure in the Android Testing Framework

Android has a built-in testing framework that helps in testing every unit of an application. It is an advanced testing framework that extends the industry standard JUnit with specific features suitable for different types of testing. The test structure is a fundamental feature of the Android testing framework. Android build and test tools assuming that the test projects consist of an organized and standard structure of tests, test case classes, test packages, and test projects. Figure 2.1 displays the testing framework.

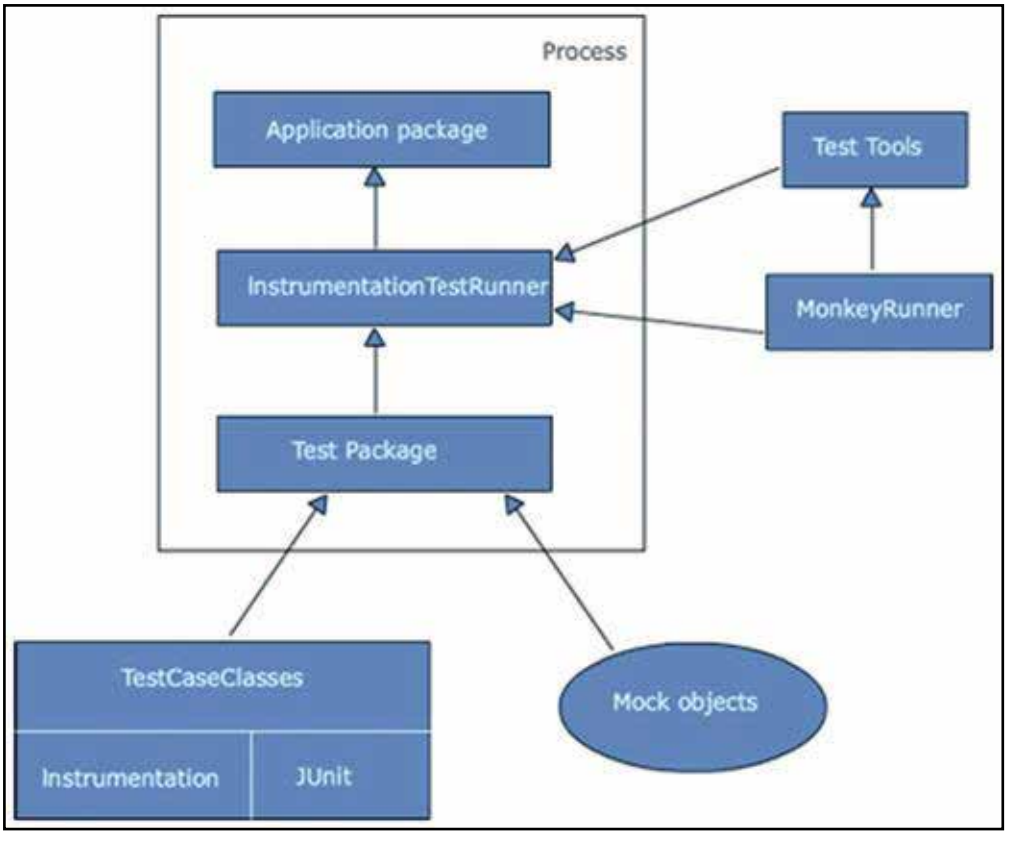


Figure 2.1: Android Testing Framework

Some of the key features of the testing framework are as follows:

- Android test suites are generally based on JUnit API. JUnit is an open source unit testing framework for Java programming language. A JUnit test is a set of Java classes used to test or evaluate individual methods in a Java class. It is used to build test cases when using agile development process. For a beginner who is not familiar with Android testing, simple test case classes such as AndroidTestCase can be used as the initial test case classes followed by more complex test classes.
- The function of Android JUnit extensions is to provide component-specific test case classes that help in creating mock objects and methods to control the lifecycle of a component.
- Also, there is no need to learn new set of tools or techniques to design or build tests. This is because test suites are similar to main application packages as they are contained in test packages.
- The SDK tools are a set of building and debugging tools. They are used with other IDEs for conducting tests in Eclipse with ADT plugin and for testing in command-line form. Android Studio bundle is specifically created for Android development and comes with built in tools for development and testing. The function of these tools is to obtain information from the project of the application that is under test. Then, they use this information to generate the build files, manifest file, and directory structure for the test package.
- Another feature of SDK tools is to send pseudo-random events to a device with the use of monkeyrunner and UI/Application Exercise Monkey. While monkeyrunner is an API testing device used with Python programs, UI/Application Exercise Monkey is a command tool for stress-testing UIs.

Following example demonstrates the use of JUnit in Eclipse. In this example, a new project is created in Eclipse that will multiply two numbers and will be tested with the help of JUnit. The steps are as follows:

1. Click File → New → Project to display the New Project dialog box.

2. Select Android → Android Application Project as shown in figure 2.2.

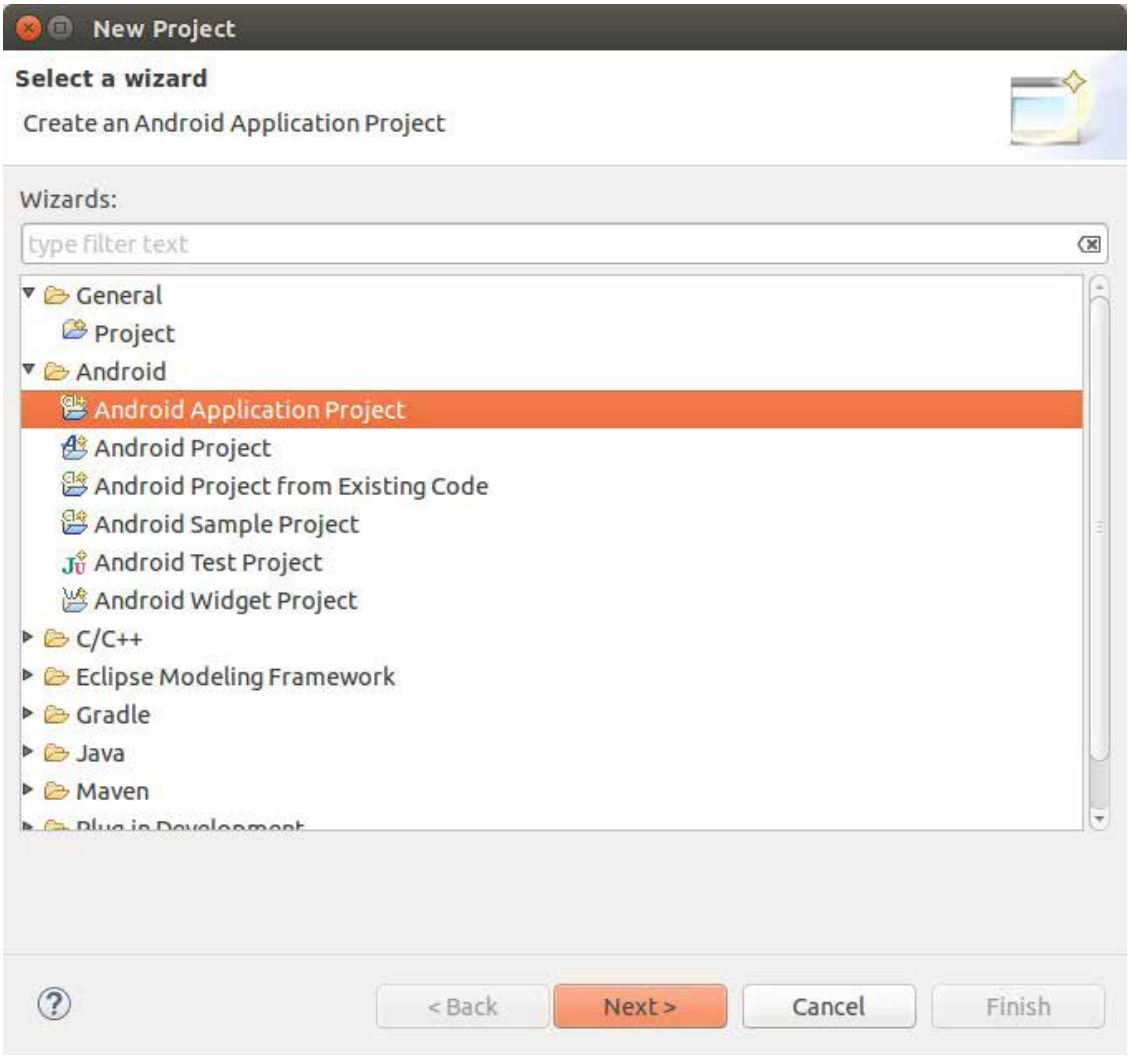


Figure 2.2: New Project Dialog Box

3. Click Next.

4. Type Calculator App in the Application Name box. By default, the Project Name box displays Calculator App and the Package Name box displays com.example.calculatorapp as shown in figure 2.3.

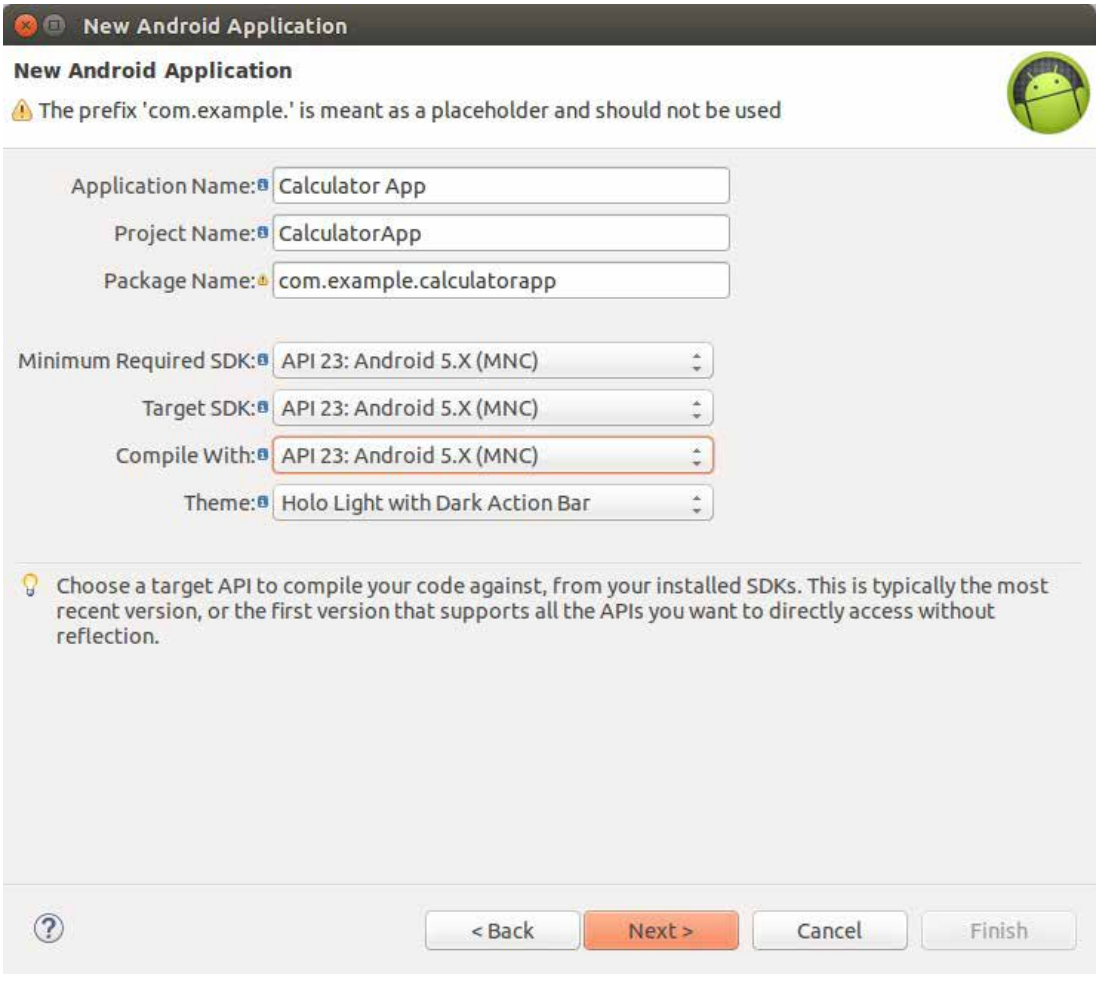


Figure 2.3: New Android Application Dialog Box, Minimum apk Version 23

5. Click Next.
6. Click Next again in the Configure Project pane of New Android Application dialog box.
7. Click Next in the Configure Launcher Icon screen of New Android Application dialog box.
8. Select Blank Activity in the Create Activity screen and click Next.
9. Give a desired activity name or leave the default name in the Blank Activity screen and click Finish. The new project, CalculatorApp is created.
10. Navigate to the src → com.example.calculatorapp.
11. Right-click the folder to display the context menu.

12. Select New → Class as shown in figure 2.4.

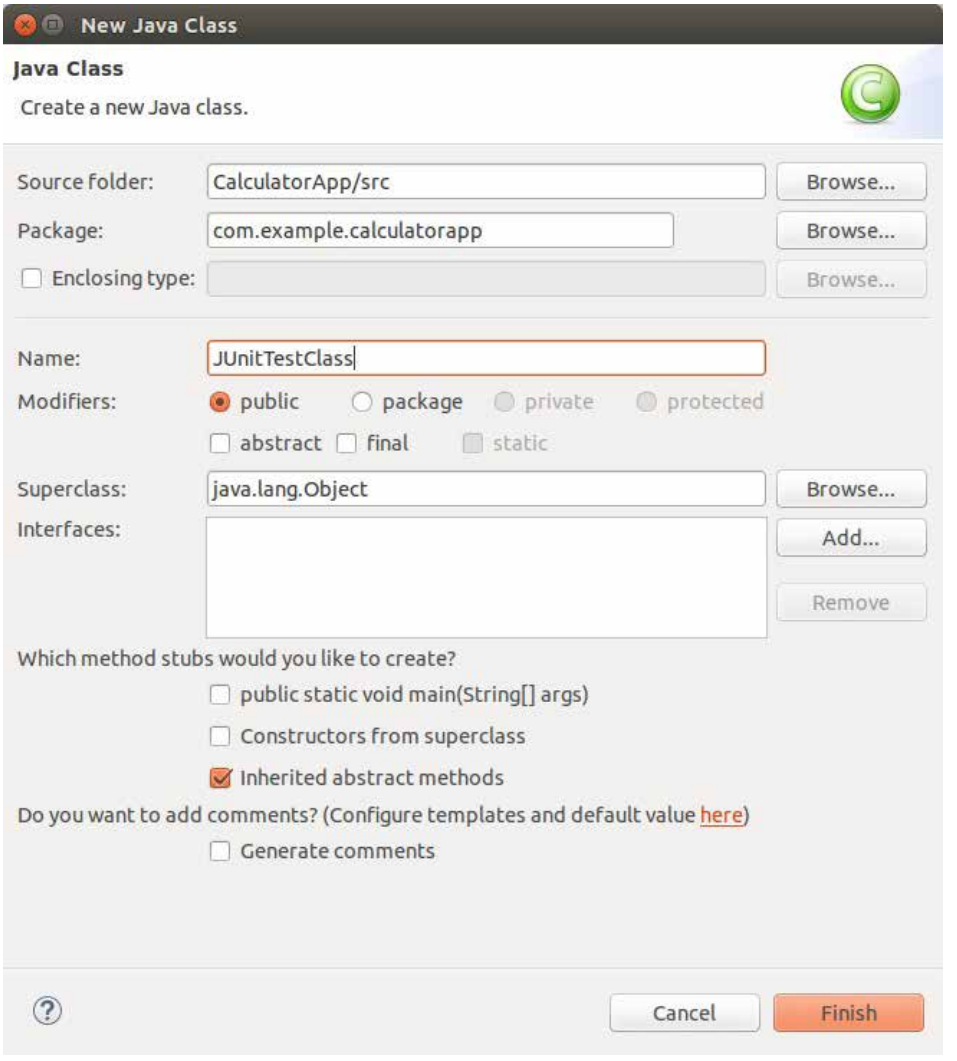


Figure 2.4: Creation of New Class

13. Type JUnitTestClass in the Name box.

14. Click Finish.

15. Add the code demonstrated in Code Snippet 1 in the new class.

Code Snippet 1:

```
package com.example.calculatorapp;
public class JUnitTestClass {
public int multiply(int a, int b) {
return a * b;
}
}
```

Next, a project will be created where JUnit framework will be used to test the method of CalculatorApp application. The steps to be performed are as follows:

1. Click File → New → Other to display the New dialog box.
2. Select Android → Android Test Project as shown in figure 2.5.

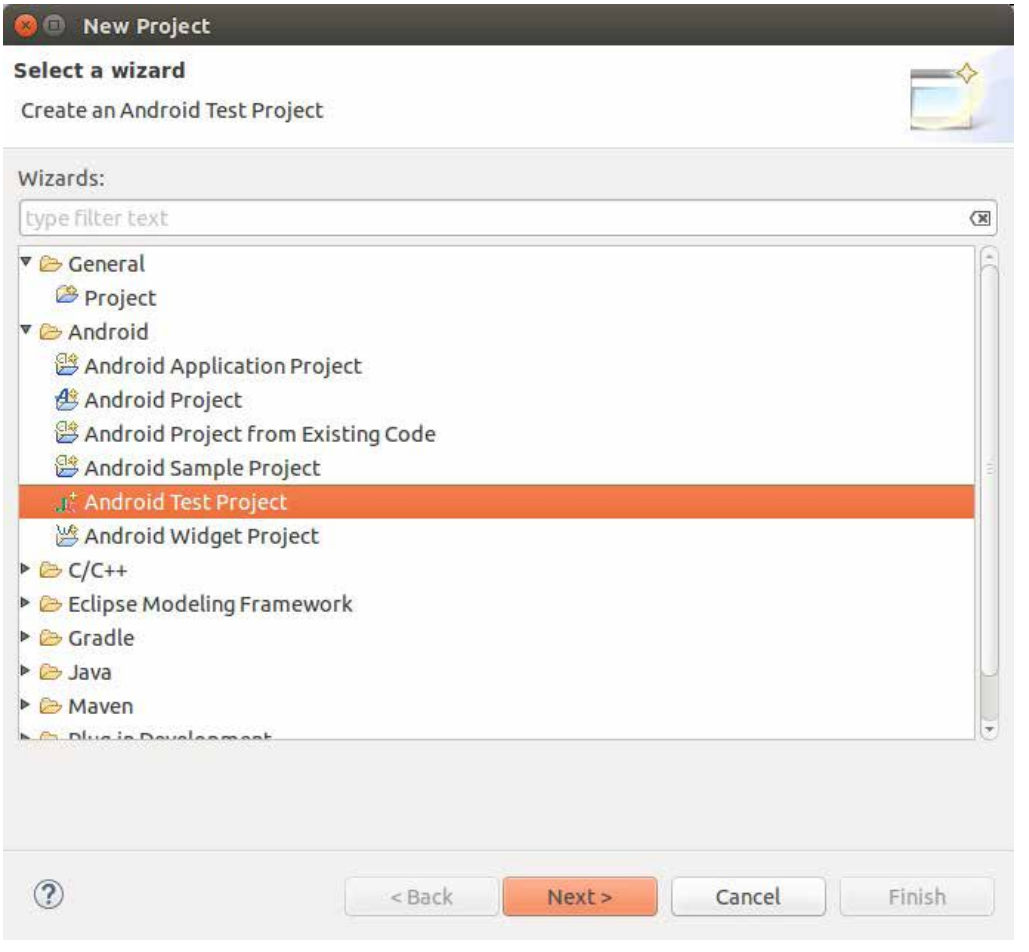


Figure 2.5: New Dialog Box

3. Click Next to display the New Android Test Project dialog box.

4. Type AndroidJUnitTest in the Project Name box as shown in figure 2.6.

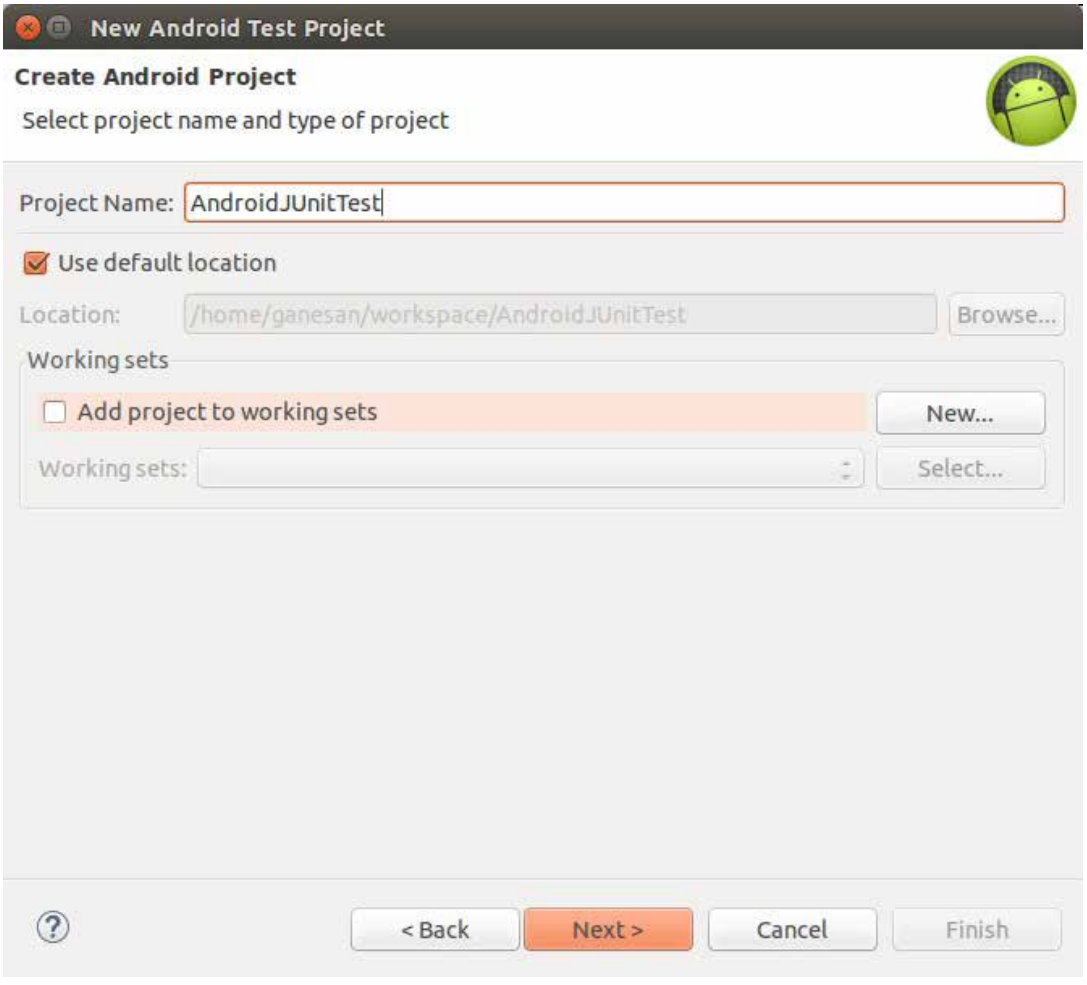


Figure 2.6: New Android Test Project Dialog Box

5. Click Next to display the Select Test Target screen of the New Android Test Project dialog box.

6. Choose the existing Android Project named CalculatorApp as shown in figure 2.7.

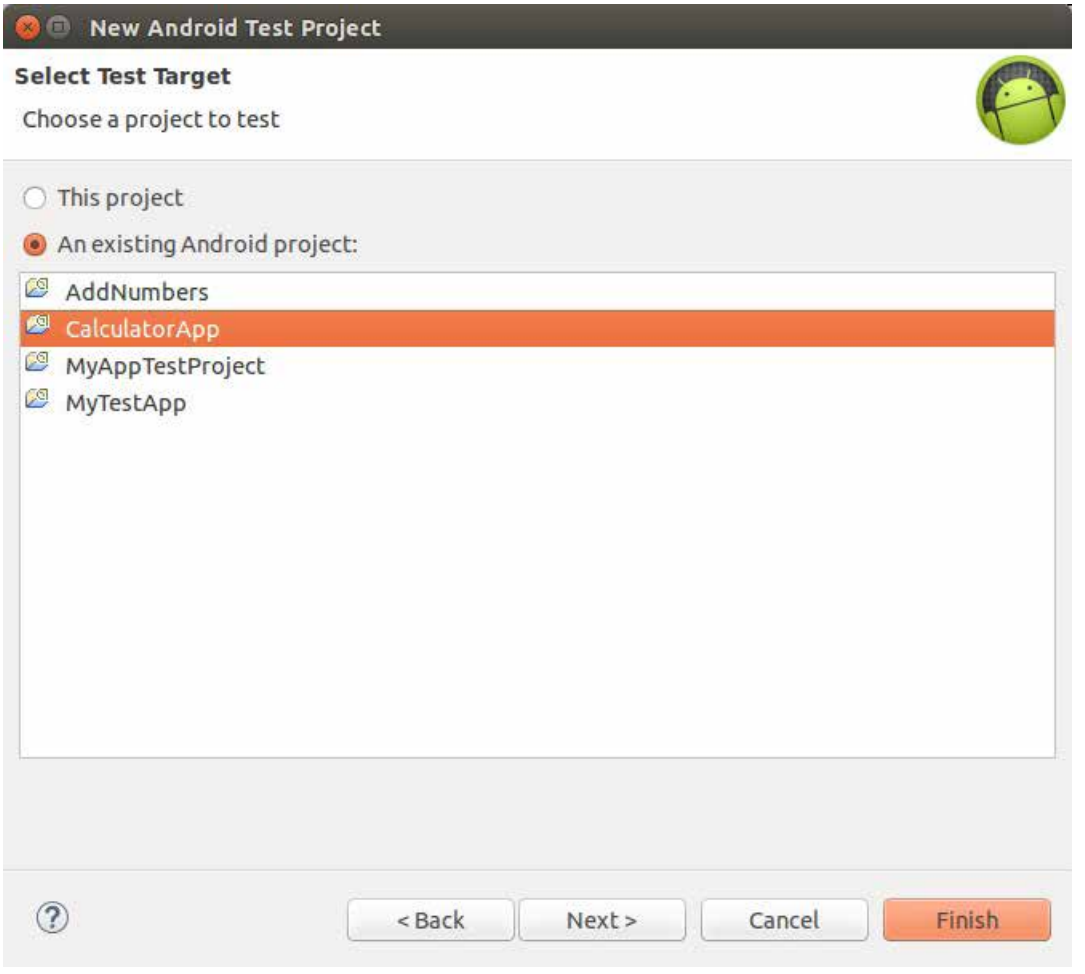


Figure 2.7: Select Test Target Screen

7. Click Next.

8. Select the Build Target as Android 6.0 and click Finish as shown in figure 2.8.

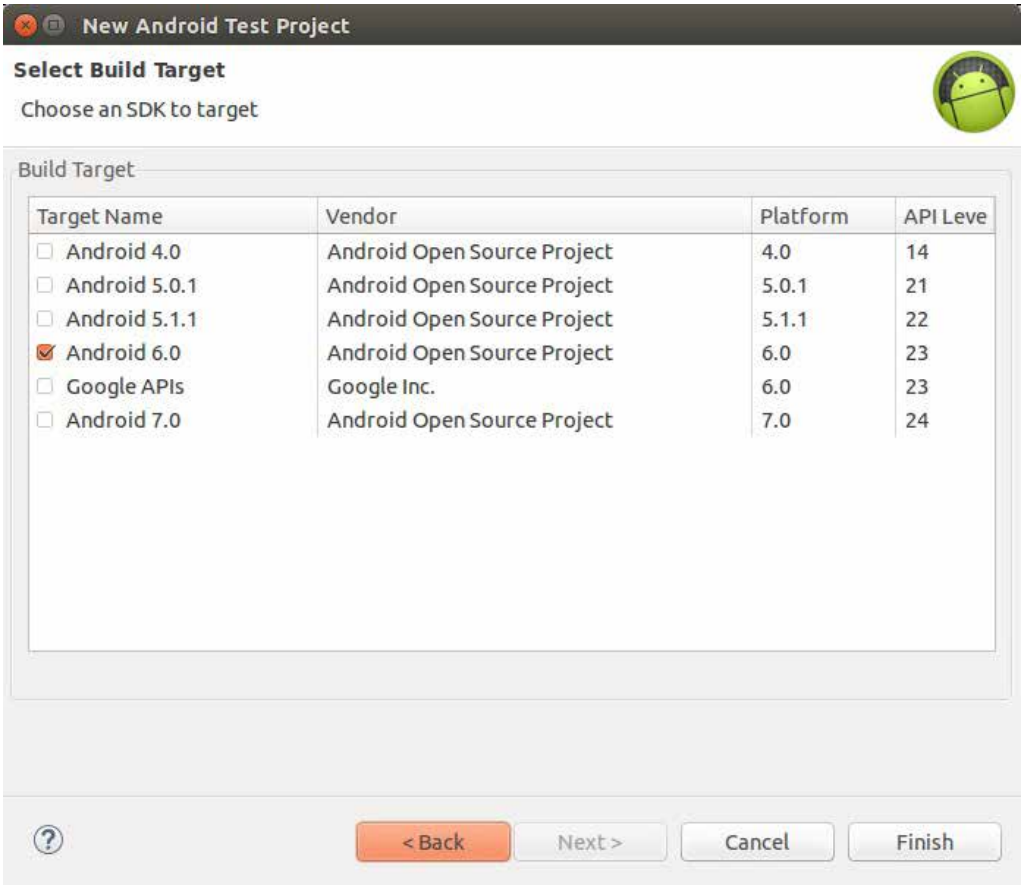


Figure 2.8: Select Build Target

9. Right-click the AndroidJUnitTest project to display the context menu.

10. Select New → Other → Java → JUnit → JUnit Test Case to display the Select a wizard pane as shown in figure 2.9.

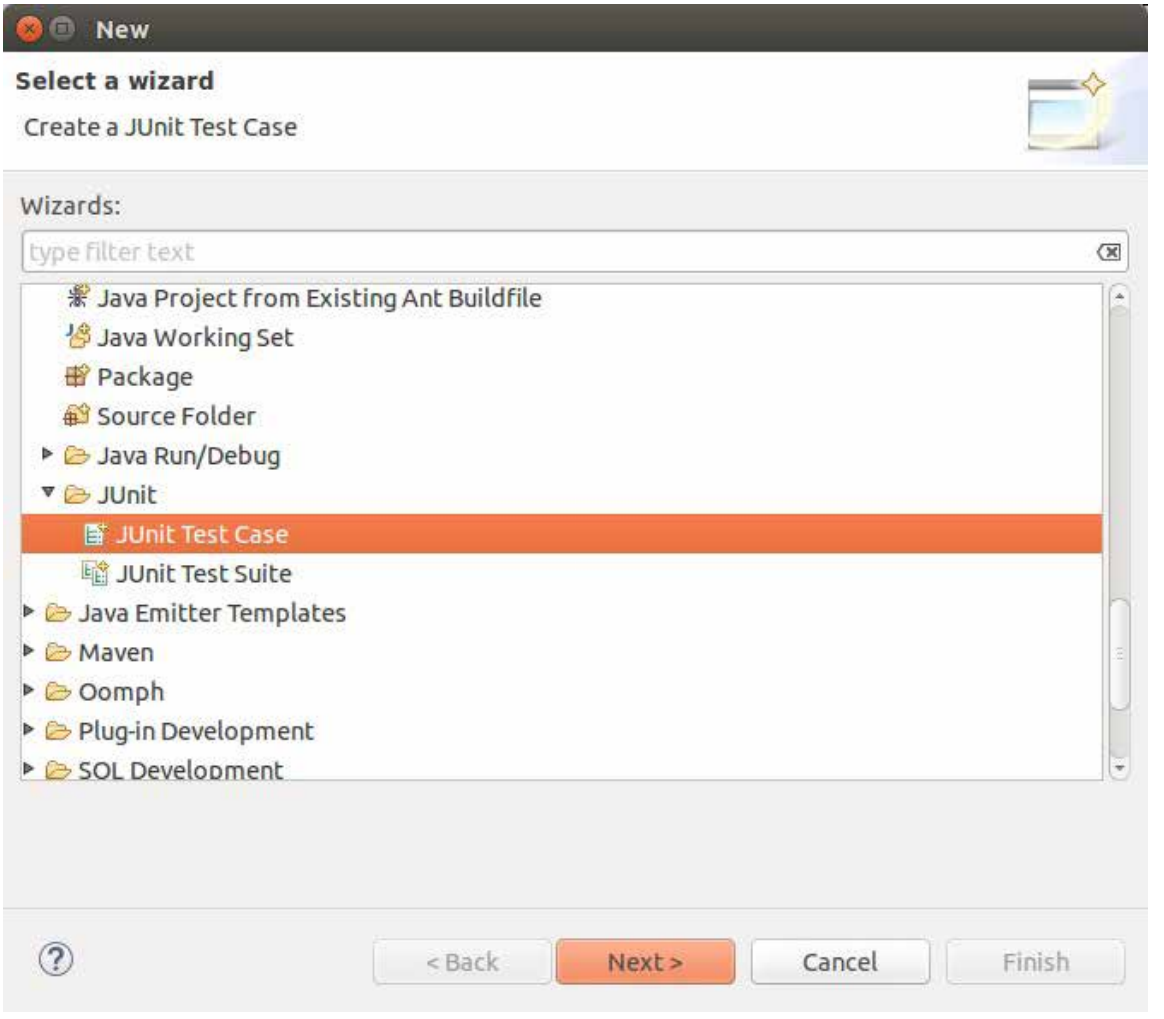


Figure 2.9: Select a wizard Screen

11. Click Next to display the New JUnit Test Case dialog box as shown in figure 2.10.

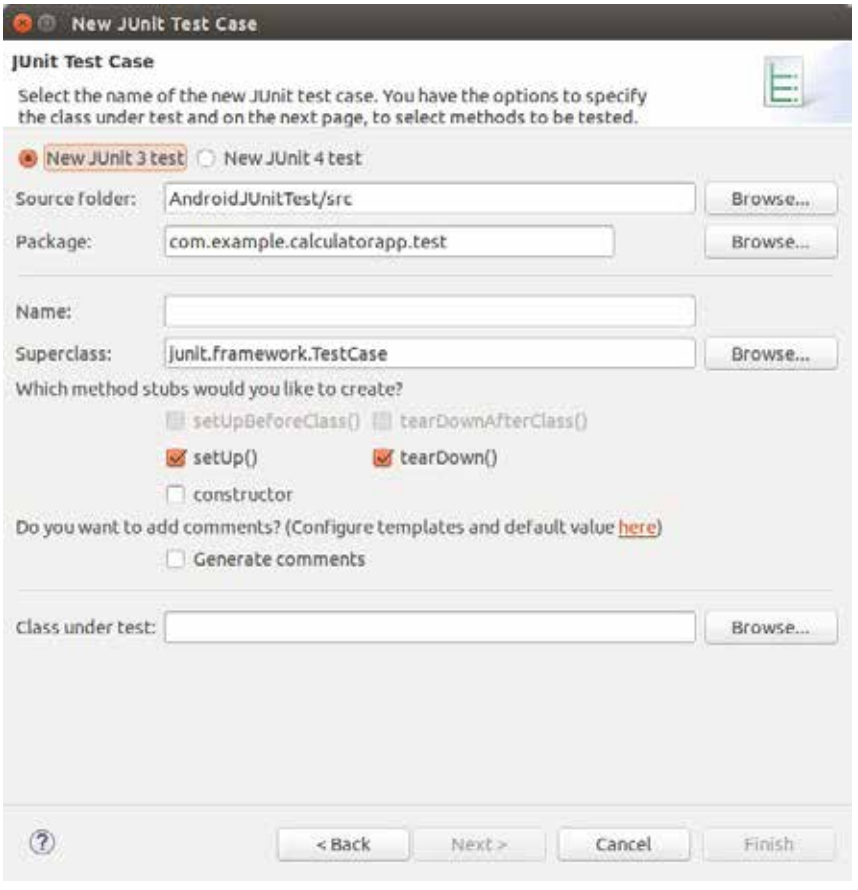


Figure 2.10: New JUnit Test Case Dialog Box

12. Type JUnitTest_Multiply in the Name box and click Finish. The automatically generated code for the test case will be as demonstrated in Code Snippet 2.

Code Snippet 2:

```
import junit.framework.TestCase;

public class JUnitTest_Multiply extends TestCase {

protected void setUp() throws Exception {

      super.setUp();

}

protected void tearDown() throws Exception {

      super.tearDown();

}

}
```

The setUp() method executes once before all the test methods are invoked whereas the tearDown() method executes after all the test methods are invoked.

13. Modify the code to add a testing method and perform some initialization in the setUp() method as demonstrated in Code Snippet 3.

Code Snippet 3:

```
import com.example.calculatorapp.JUnitTestClass;
import junit.framework.TestCase;
public class JUnitTest_Multiply extends TestCase {
      JUnitTestClass junitTestClass;
int mArg1;
int mArg2;
protected void setUp() throws Exception {
      junitTestClass = new JUnitTestClass();
      mArg1 = 6;
      mArg2 = 3;
      super.setUp();
}
protected void tearDown() throws Exception {
      super.tearDown();
}
public void testAdd() {
      assertEquals(12, junitTestClass.multiply(mArg1, mArg2));
}
}
```

Note - To add any method to test cases, prefix the method name with 'test'.

Here, the testAdd() method checks the output of the multiply method using an assertion. If the output is 12, the assertion will be true. In case the output is not 12, the assertion becomes false and an error will be generated by the test case.

14. Right-click the AndroidJUnitTest project and select Run As → Android JUnit Test. Figure 2.11 displays the output after execution of the application.

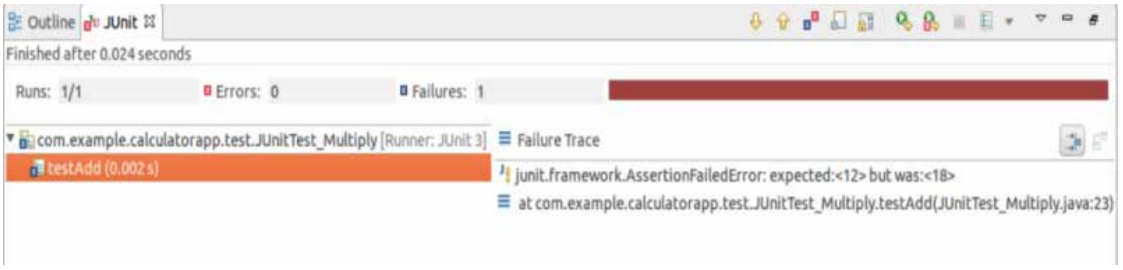


Figure 2.11: Test Output - Error

The output shows the AssertionFailedError exception. This is because the expected output was 12, whereas, the generated output is 18.

15. Now, change the value of mArg2 variable to 2 and run the test again. The output will be as shown in figure 2.12.



Figure 2.12: Test Output - Success

The figure shows a successful run of the test case after changing the value of mArg2 variable since, the assertion now becomes true.

2.2.1 Android Tools

Tests are put together into projects. A test project is a directory or an Eclipse project that is used to create the source code and manifest files for a test package. Android SDK tools can be used to create test projects. The tools that are used for Eclipse with ADT and command-line create directories and update test projects. The directories are used for source code and resources and the manifest files for the test package. Build files are created by command-line tools.

The other functions and uses of Android tools include:

- Assigns InstrumentationTestRunner automatically as the test case runner to run JUnit tests for the test package under test as demonstrated in Code Snippet 4.

Code Snippet 4:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/
android"
 package="com.example.calculatorapp.test"
 android:versionCode="1"
 android:versionName="1.0" >
 <uses-sdk android:minSdkVersion="21" />
 <instrumentation
  android:name="android.test.InstrumentationTestRunner"
  android:targetPackage="com.example.calculatorapp" />
 <application
  android:icon="@drawable/ic_launcher"
  android:label="@string/app_name" >
  <uses-library android:name="android.test.runner" />
 </application>
</manifest>
```

- Provides a relevant name for the test package. For example, if com.example.calculator is the package name for the application under test, then it generates and provides the test package name as com.example.calculatorapp.test. This type of naming system helps in identifying the relationship between the application package and the test package. It also prevents conflicts within the system.
- Creates proper build files, manifests files, and directory structure. This is an automated function for the test project. Thus, the Android tools help in building the test package and establishing a connection between the test package and the application under test. The build files do not require any modification.
- Creates a test project anywhere in the file system. This is a quicker method however; it is advisable to add the test project at the root. This makes it easier to trace the tests associated with an application since, the root directory tests/ is at the same level as the src/ directory of the main application's project.

For example, let the root directory of an application project be MyFirstProject, and then the directory structure will be as shown in figure 2.13.

```
MyFirstProject/
        AndroidManifest.xml
    res/
        ... (resources for main application)
    src/
        ... (source code for main application) ...
    tests/
        AndroidManifest.xml
    res/
        ... (resources for tests)
    src/
        ... (source code for tests)
```

Figure 2.13: Directory Structure of the Test Project

2.3 Testing Frameworks

There are various frameworks available for testing android applications including official as well as third-party. Some of these testing frameworks include:

- AndroidJUnitRunner
- Expresso
- UI automator
- calabash
- Appium

2.3.1 AndroidJUnitRunner

AndroidJUnitRunner is an official Android testing tool that can be used to test android applications. It requires Android 2.2 (API level 8) or higher.

The AndroidJUnitRunner class is a JUnit test runner. It allows the developer to run JUnit 3 or JUnit 4-style test classes on Android devices. AndroidJUnitRunner supersedes the InstrumentationTestRunner class. The InstrumentationTestRunner class only supports JUnit 3 tests. The developer can use the AndroidJUnitRunner on devices using the Espresso and UI Automator testing frameworks. The developer can use the JUnit TestCase class to do unit testing on a class that doesn't call Android APIs.

AndroidJUnitRunner automates testing by loading the test package and the app under test to a device, running the tests, and reporting of the test results.

It provides access to Instrumentation Information related to the test run using the InstrumentationRegistry class. This class includes useful information that helps when writing tests that depend on the Instrumentation or Context objects or when writing tests using the UI Automator framework.

AndroidJUnitRunner also provides support for JUnit 3 and JUnit 4, up to JUnit 4.10 tests. The developer must prefix the test class @RunWith(AndroidJUnit4.class) annotation when creating an instrumented JUnit 4 test class.

Note - It is advisable not to mix JUnit 3 and JUnit 4 test code in the same package because this can lead to unexpected results.

Annotations can be used to configure JUnit 4.x tests in AndroidJUnitRunner so as to reduce the use of boilerplate and conditional code in the tests. In addition to the standard annotations in JUnit 4, AndroidJUnitRunner supports some Android-specific annotations, such as @RequiresDevice, @SmallTest, @MediumTest, and @LargeTest. The @RequiresDevice annotation specifies that test should run only on physical devices and not on emulators. The @SdkSupress annotation prevents the test from running on a lower Android API level then the given level. The @SmallTest, @MediumTest, and @ LargeTest annotation specifies the duration for which a test should run. It also specifies the frequency of running the test.

AndroidJUnitRunners allows developers to split a single test suite into multiple shards. An index number is allocated to each shard to identify the shards. To run the tests under the same, similar shards can be grouped together. When running tests, the developer can use the -e numShards option to specify the number of separate shards to create and the -e shardIndex option to specify which shard to run.

2.3.2 Espresso

Espresso is a UI test framework which is a part of the Android Testing Support Library for Android. Espresso helps developers to write reliable automated UI tests for a single Android app with great ease. Espresso tests are instrumentation based tests and are run on an actual device or emulator. So, these tests behave as if an actual user is using the app.

Espresso provides a simple and extensible API that automatically synchronizes test actions with the user interface of the test application, and provides rich failure information. The Espresso framework ascertains that user activity is started before the tests run and lets the test wait until all observed background activities have finished.

Espresso has three basic components:

- **ViewAssertions**: This component allows the developer to assert state of a view.
- **ViewMatchers**: This component lets the developer to find a view in the current view hierarchy. If Espresso is unable to find a view via the ViewMatcher component, it includes the whole view hierarchy into the error message. This is useful to analyze the problem.

- **ViewActions**: This component permits the developer to perform actions on the views.

The standard case constructs for an Espresso test is to find a view using (ViewMatchers), take some action on that view using (ViewActions), and then validate some view properties using (ViewAssertions).

2.3.3 UI Automator

UI Automator testing framework is an instrumentation-based API. The Android Testing Support Library provides this framework. UI Automator tests can run on devices having Android 4.3 (API level 18) or higher versions.

This framework can be used to perform automated black-box testing on Android apps by interacting with the visible elements on a device, regardless of which Activity is in focus. UI Automator allows developers to imitate user actions on the widgets that constitute an app's user interface.

Tests in the UI Automator framework are written using convenient descriptors that can look up a UI component by text displayed in that component or its content description.

`Inspecting the UI on a device`

Before designing the test, developers must inspect the UI components that are visible on the device to ensure that the UI Automator tests can access these components. To make the components accessible to UI Automator, they must have visible text labels or android:contentDescription values or both configured.

Use the uiautomatorviewer tool to inspect the layout hierarchy and view the properties of UI components that are visible on the foreground of the device.

To launch the uiautomatorviewer tool:

1. Launch the target app on a physical device.
2. Connect the device to the development machine.
3. Open a terminal window and navigate to the <android-sdk>/tools/ directory.
4. Run the tool with the following command:

   $ uiautomatorviewer

   To view the UI properties of the application:

5. In the uiautomatorviewer interface, click Device Screenshot.
6. The uiautomatorviewertool identifies the UI components. To view them, move the mouse pointer over the interface in the left-hand panel.

The lower right-hand panel lists the properties and the upper right-hand panel displays the layout hierarchy as shown in figure 2.14.

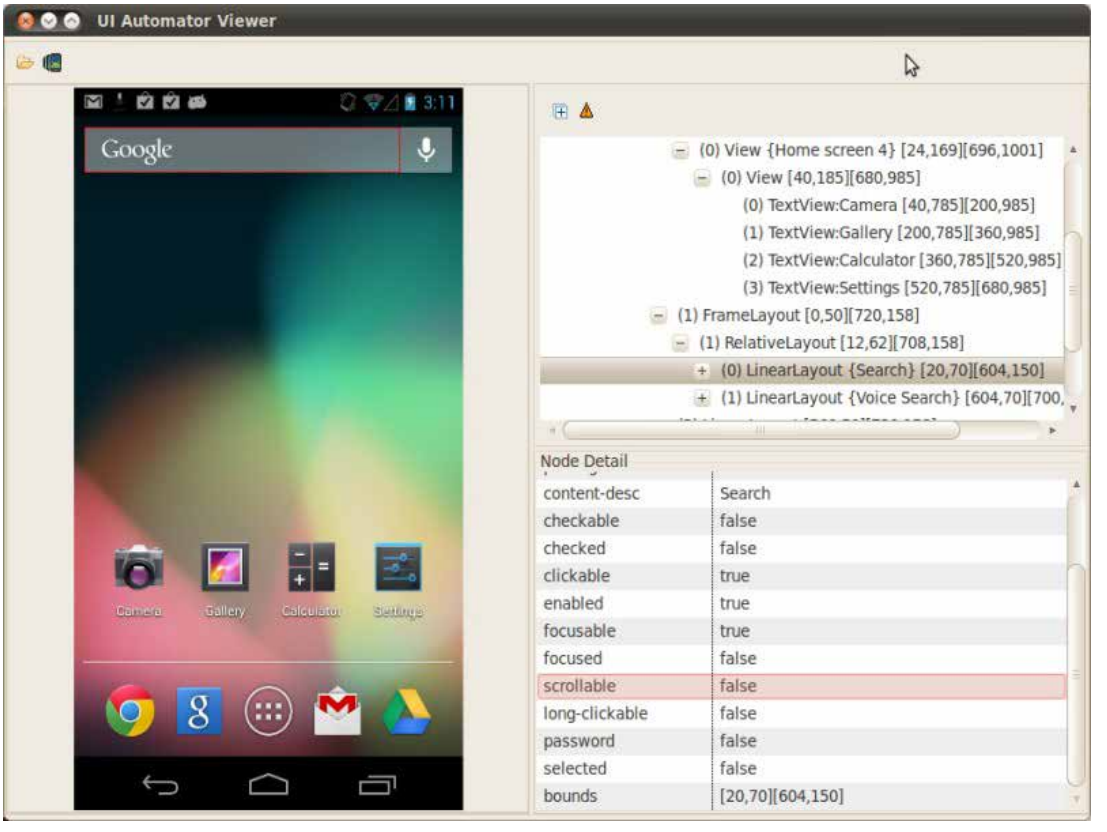


Figure 2.14: UI Automator Viewer

7. Optionally, click the Toggle NAF Nodes button to see UI components that are not accessible to UI Automator. Only limited information will be available for these components.

2.3.4 Calabash

Calabash is a test automation framework that allows mobile developers to create and execute automated acceptance tests for Android and iOS apps. Therefore, to test an application developed to run on both Android and iOS platforms, Calabash tests are a suitable choice. Calabash renders Automated UI Acceptance Tests written in Cucumber to be run on iOS and Android applications. The actual Calabash tests are written in Gherkin, backed with Ruby code, and run in the context of the Cucumber Framework. While Gherkin defines the tests and the step definitions provide the cementing code to interpret the tests into something actionable, Cucumber is the framework that actually runs the tests.

Calabash is viewed as a Behavior-Driven Development (BDD) test automation framework. So, it is similar to a philosophy of test-driven development or outside-in development. Hence, tests created in Calabash specify application behaviors instead of describing the

shape of APIs. Calabash carries out automated UI acceptance test automation. It enables UI interactions that are automatic within an application such as text input, button presses, response validation, and so on. Moreover, Calabash tests can be configured to be executed on hundreds of different Android and iOS real mobile devices. It proves to be a great test framework to collect real-time feedback and validation across many OS versions, hardware specifications, different form factors, and OEM customizations.

Calabash for Android

Using Calabash for Android requires that the latest version of SDK be installed. Create an environmental variable ANDROID_HOME. The variable will point to the location where the extracted or installed SDK folder exists. It will also ensure that calabash-android command is configured.

```
$ sudo gem install calabash-android
```

2.3.5 Appium

Appium is a freely distributed open-source mobile application UI testing framework. It lets the developer to arrive at cross-platform solution for native and hybrid mobile automation testing. It allows communications between the app being tested and other apps. Appium permits native, hybrid, and Web application testing. It also supports automation test on physical devices, or simulator, or on both.

Appium uses standard automation APIs on all platforms. Therefore, the test app does not need to be re-compiled or modified in any way when testing it with Appium. Appium carries out 'black box' testing so it can test not only the app developed by the developer but any *.apk installed on the phone or emulator. Selenium WebDriver and language-specific libraries are required to write Appium tests in any WebDriver compatible language such as PHP, Java, Ruby, Python, Objective-C, C#, and JavaScript with node.js.

2.4 Android Testing API

The Android testing API is modeled on the JUnit API. It has an instrumentation framework and Android-specific testing classes.

Unit test is a software test that helps to isolate the component under being tested and tests/evaluates it repeatedly. JUnit is the default standard for unit tests on Android. It automates unit testing.

A JUnit can be called as a method that allows testing of each part of an application. Test methods are organized into classes called test cases or test suites. Each test method is a stand-alone test conducted on an individual module present in the application under test. In addition, each class is a collection of related test methods. The class may be responsible for providing helper methods too.

Similar to JUnit, Android also uses SDK build tools to create one or more test source files into class files in an Android test package. While in JUnit, a test runner is used to execute test classes, Android uses test tools to execute an Android-specific test runner. The test tools first load the package and the application under test and then operate the Android-specific test runner.

AndroidTestCase extends the JUnit TestCase. JUnit TestCase class is called when unit testing is done on a class that does not use Android APIs.

Android-dependent objects can be tested using the AndroidTestCase class. Besides this, AndroidTestCase class also provides Android-specific setup, teardown, and helper methods.

Figure 2.15 displays the hierarchy of the AndroidTestCase class.

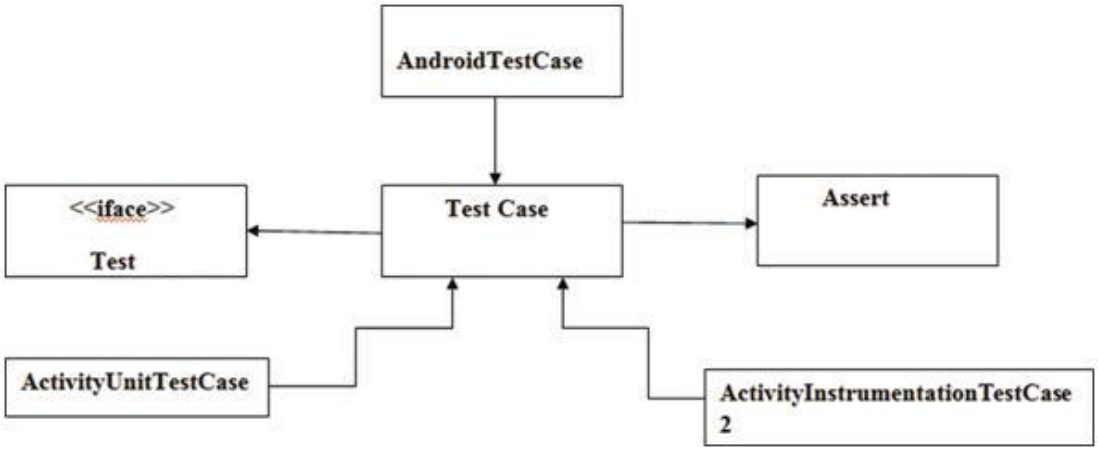


Figure 2.15: AndroidTestCase Class

2.4.1 AndroidTestCase Base Class

As the name suggests, this class is the base class for testing Android test cases. It is present in the android.test package. It extends both the TestCase and Assert classes. This class is used when access to an Activity Context in the file system is required. Activity Context can include resources, databases, or files in the file system. This class stores Context object as a field referred to as mContext. The mContext and getContext() methods can be used inside tests, if needed. Also, Context.startActivity() method can be used to start more than one Activity. The other test cases that extend this base class are as follows:

- **ApplicationTestCase<T extends Application>** - A class for testing an entire application.
- **ProviderTestCase2<T extends ContentProvider>** - A class for isolated testing of a single Content Provider.
- **ServiceTestCase<T extends Service>** - A class for isolated testing of a single Service. Apart from these, this class provides methods for testing permissions and a method that prevents memory leaks. It removes certain class references to protect from memory leaks.

Figure 2.16 shows the UML class diagram of AndroidTestCase and its related classes.

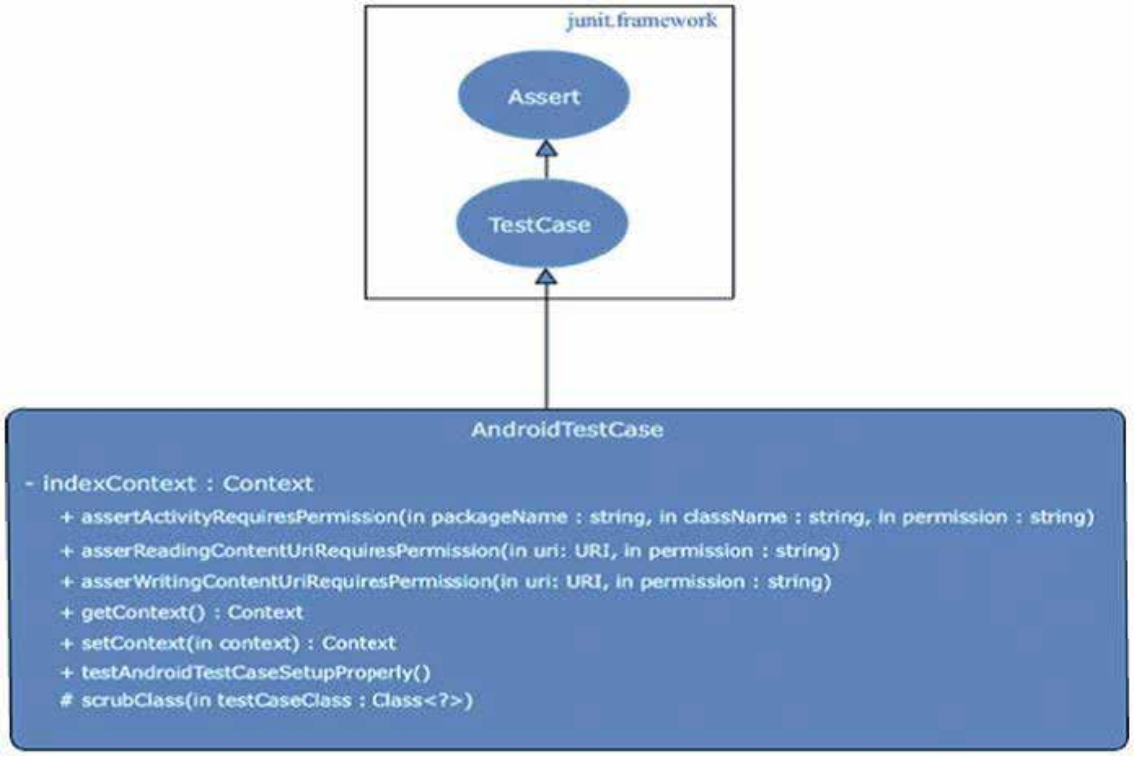


Figure 2.16: UML Class Diagram of AndroidTestCase

- **ApplicationTestCase**

This class is used for testing the application classes, specifically to test the setup and teardown of all application objects. The function of these objects is to take care of the global state of information that is applicable to all the components in the application package. The test case also verifies whether the <application> element in the manifest file is set up correctly. The <application> tag in the AndroidManifest.xml file will cause the class to be instantiated when the application/package is created. It needs to be remembered that this test case does not allow control testing of the components in the application package.

- **ProviderTestCase2**

It is a class that is used for isolated testing of a single ContentProvider. This test case has its own internal map that it uses to resolve providers that are given an authority. This enables injecting test providers that are to be used and at the same time nulling out providers that are unused. The base class for this class is AndroidTestCase class. The template parameter T represents the ContentProvider under test and uses an IsolatedContext and a MockContentResolver for implementation of this test.

A few mock objects that are set up by this class are as follows:

- **IsolatedContext**
  - Allows tests to run file and perform database work.
  - Stubs out Context methods that affect functioning of the rest of the system.
- **MockContentResolver**
  - Uses IsolatedContext to function like a regular content resolver.
  - Stubs out the method notifyChange(Uri, ContentObserver, boolean) to prevent the test from affecting the functioning of the rest of the system.
- An instance of the provider case under test, which runs in an IsolatedContext.

The base class' setUp() method automatically sets up the framework for the ProviderTestCase class. In case there is a need to override this method, the super() method is invoked and should be the first statement that needs to be invoked in the overridden method.

For successful running of these tests, concrete subclasses must support their own constructor with no arguments.

Figure 2.17 shows the UML diagram of ProviderTestCase2 and the closest related classes.

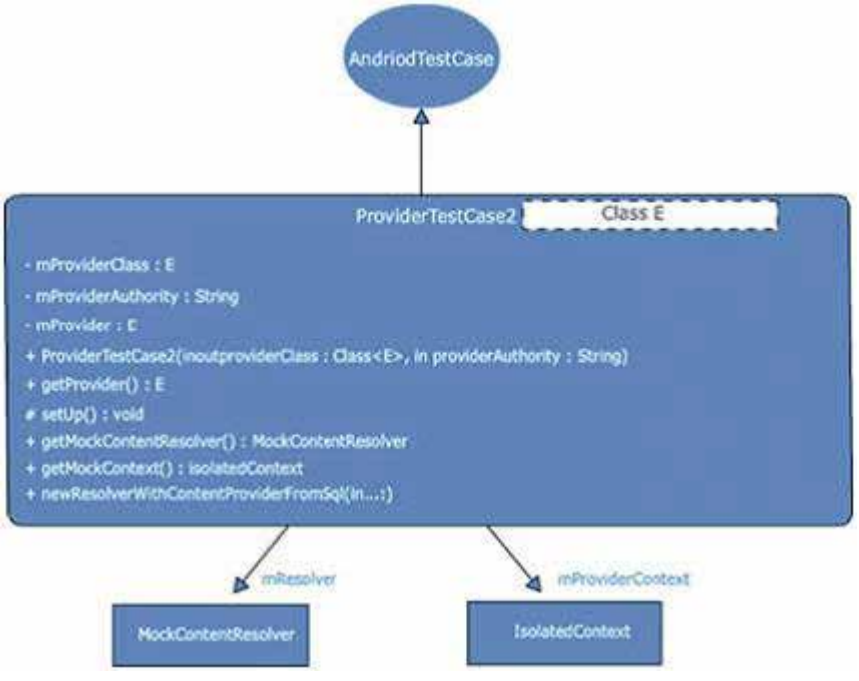


Figure 2.17: ProviderTestCase2

The only public non-deprecated constructor of this class is:

```
ProviderTestCase2(Class<E>providerClass, String providerAuthority).
```

It is invoked using an instance of the ContentProvider class which is also used as the first parameter. The authority for the provider, defined as a constant named AUTHORITY in the ContentProvider class is the second parameter.

Code Snippet 5 demonstrates an example of testing the Content Provider.

Code Snippet 5:

```
...
public void testCheckQuery(){
   Uri uri=Uri.withAppendedPath(MyProvider.CONTENT_URI, "sample");
   final Cursor ap=mProvider.query(uri, null, null, null, null);
   final int anticipated=4;
   final int original=ap.getCount();
   assertEquals(anticipated, original);
}
...
```

In Code Snippet 5, the Cursor object is anticipated to return a result containing four rows. This test can be asserted for more than four rows. In the setUp() method, the getProvider() method is invoked to obtain a reference of the provider named, mProvider. An interesting point to observe is that since, these tests are using MockContentResolver and IsolatedContext, the content of the real database is not affected. This allows running tests as shown in Code Snippet 6.

Code Snippet 6:

```
...
public void testCheckDelete(){
   Uri uri = Uri.withAppendedPath(MyProvider.CONTENT_URI, "sample");
      final int original = mProvider.delete(uri, "_id = ?", new
String[] { "1" });
    final int anticipated = 1;
    assertEquals(anticipated, original);
}
…
```

Though this test deletes some content of the database, it restores the initial content so that other tests are not affected.

- **ServiceTestCase<T>**

This test case tests services. In other words, it tests the Service class. Service lifecycle methods such as setupService(), startService(), bindService(), and shutDownService() are also included in this class.

The only public non-deprecated constructor of this class is:

```
ServiceTestCase(Class<T>serviceClass)
```

This test provides the basic support to test Service classes in a controlled environment. It provides the framework for the lifecycle of the service. It also serves links for injecting various dependencies to test the Service class in a controlled environment. Figure 2.18 shows that this class extends the AndroidTestCase class and also depicts its UML class diagram representation.

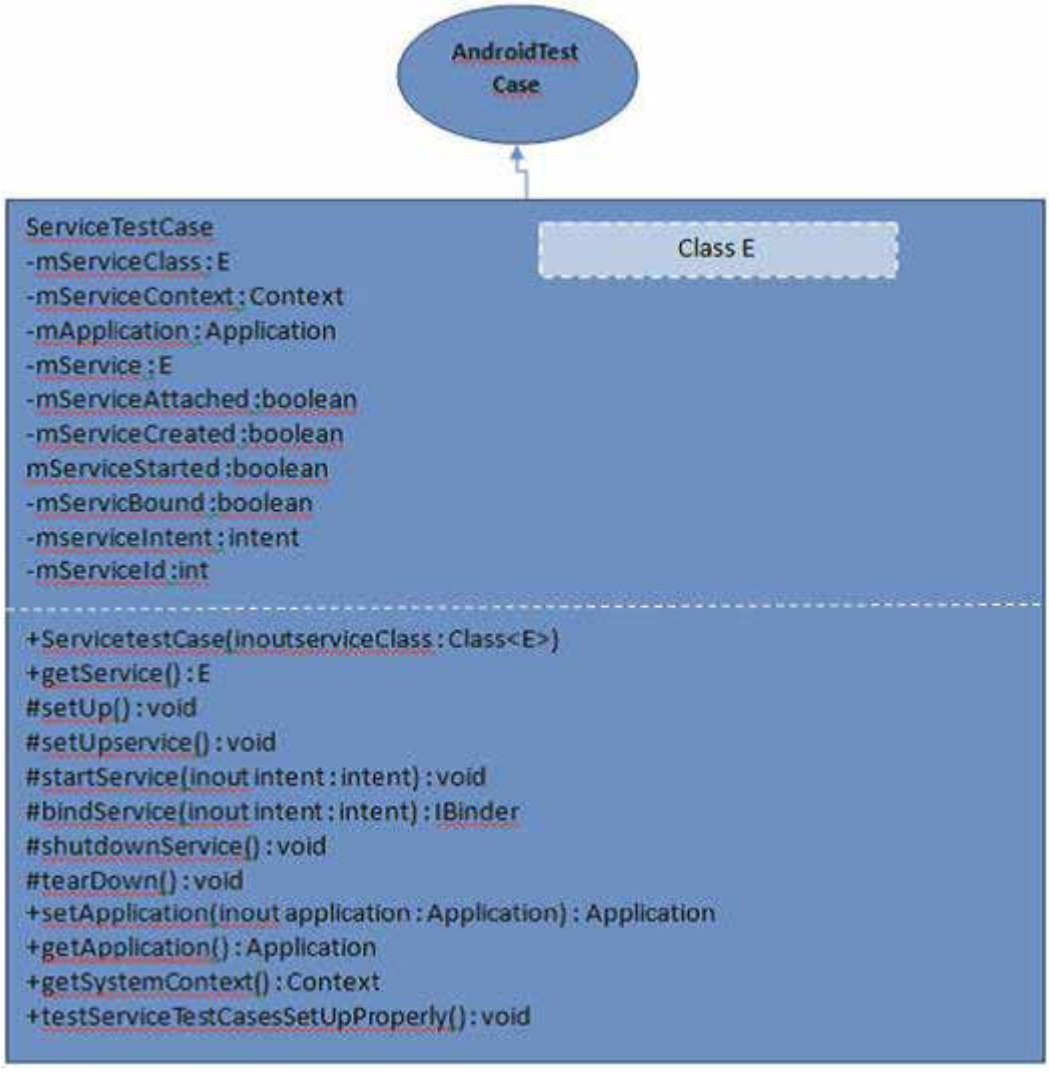


Figure 2.18: UML Class Diagram of ServiceTestCase<E>

2.4.2 Android Instrumentation

The Android system has a set of control methods or 'hooks' called Android instrumentation.

The hooks function as a control:

- for an Android component to run independently of its normal cycle.
- on how an Android loads an application/applications.

Generally, the system determines the lifecycle of an Android component. For example, the lifecycle of an Activity object begins when it is activated by an Intent. The object of the Activity class invokes the onCreate() method followed by the onResume() method. When another application is started, the onPause() method is invoked. The onDestroy() method is invoked when the Activity code invokes the finish() method. These callback methods cannot be invoked directly with the use of the Android framework API but can be done using instrumentation.

Normally, the system runs all components of an application in the same process or sometimes for some components, such as content providers, it runs as a separate process. However, it is not possible for the system to run an application in the same process as another application that is already running. Such a constraint can be overrun with Android instrumentation. Android instrumentation runs step-by-step through the lifecycle of a component by invoking callback methods in the test code.

Following example shows the use of Android Instrumentation in Eclipse. The example will create a new project in Eclipse where two numbers will be added and will be tested with the help of Android Instrumentation.

1. Start Eclipse IDE and create a project named Add Numbers.
2. Navigate to res → layout folder.
3. Open activity_main.xml file and modify the code as demonstrated in Code Snippet 7.

Code Snippet 7:

```
<RelativeLayout    xmlns:android="http://schemas.android.com/
apk/res/android"
 xmlns:tools="http://schemas.android.com/tools"
 android:layout_width="match_parent"
 android:layout_height="match_parent"
 android:paddingBottom="@dimen/activity_vertical_margin"
 android:paddingLeft="@dimen/activity_horizontal_margin"
 android:paddingRight="@dimen/activity_horizontal_margin"
 android:paddingTop="@dimen/activity_vertical_margin"
```

```
tools:context="com.example.addnumbers.MainActivity">
<EditText
 android:id="@+id/editText2"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:layout_alignLeft="@+id/editText1"
 android:layout_below="@+id/editText1"
 android:layout_marginTop="22dp"
 android:ems="10"
 android:hint="Enter second number"
 android:singleLine="true" />
<TextView
 android:id="@+id/textView5"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:layout_alignLeft="@+id/editText2"
 android:layout_below="@+id/button1"
 android:layout_marginTop="30dp"
 android:text="Output" />
<EditText
 android:id="@+id/editText1"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:layout_alignParentLeft="true"
 android:layout_below="@+id/textView1"
 android:layout_marginLeft="16dp"
 android:layout_marginTop="17dp"
 android:ems="10"
 android:hint="Enter first number"
```

```
  android:singleLine="true" />
 <Button
  android:id="@+id/button1"
  android:layout_width="wrap_content"
  android:layout_height="wrap_content"
  android:layout_below="@+id/editText2"
  android:layout_marginTop="14dp"
  android:layout_toRightOf="@+id/textView5"
  android:text="ADD" />
 <TextView
  android:id="@+id/textView1"
  android:layout_width="wrap_content"
  android:layout_height="wrap_content"
  android:layout_alignLeft="@+id/button1"
  android:layout_alignParentTop="true"
  android:layout_marginTop="20dp"
  android:gravity="center"
  android:text="Add Two Numbers" />
</RelativeLayout>
```

4. Navigate to src → com.example.addnumbers folder.

5. Open MainActivity.java file and modify the code as demonstrated in Code Snippet 8.

Code Snippet 8:

```
import android.app.Activity;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
```

```
import android.widget.TextView;
import android.view.View.OnClickListener;
public class MainActivity extends Activity {
EditText first, second;
Button res;
TextView tx;
@Override
protected void onCreate(Bundle savedInstanceState) {
     super.onCreate(savedInstanceState);
     setContentView(R.layout.activity_main);
     first = (EditText) findViewById(R.id.editText1);
     second = (EditText) findViewById(R.id.editText2);
     tx = (TextView) findViewById(R.id.textView5);
     res = (Button) findViewById(R.id.button1);
     res.setOnClickListener(new OnClickListener() {
          @Override
          public void onClick(View v) {
               // TODO Auto-generated method stub
               float  result  =  Float.parseFloat(first.getText().
toString())
                    +         Float.parseFloat(second.getText().
toString());
               tx.setText(Float.toString(result));
               }
          });
}
@Override
public boolean onCreateOptionsMenu(Menu menu) {
     // Inflate the menu; this adds items to the action bar if it is
present.
```

```
        getMenuInflater().inflate(R.menu.main, menu);
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
        // Handle action bar item clicks here. The action bar will
        // automatically handle clicks on the Home/Up button, so long
        // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();
        if (id == R.id.action_settings) {
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
}
```

Once you execute the application, the output will be as shown in figure 2.19.



Figure 2.19: Add Number Application

Next a project, AddNumberTest will be created which will use Instrumentation framework to test the method of application.

1. Click File → New → Other to display the New dialog box.
2. Select Android → Android Test Project.
3. Click Next.
4. Type AddNumbersTest in Project Name box as shown in figure 2.20.

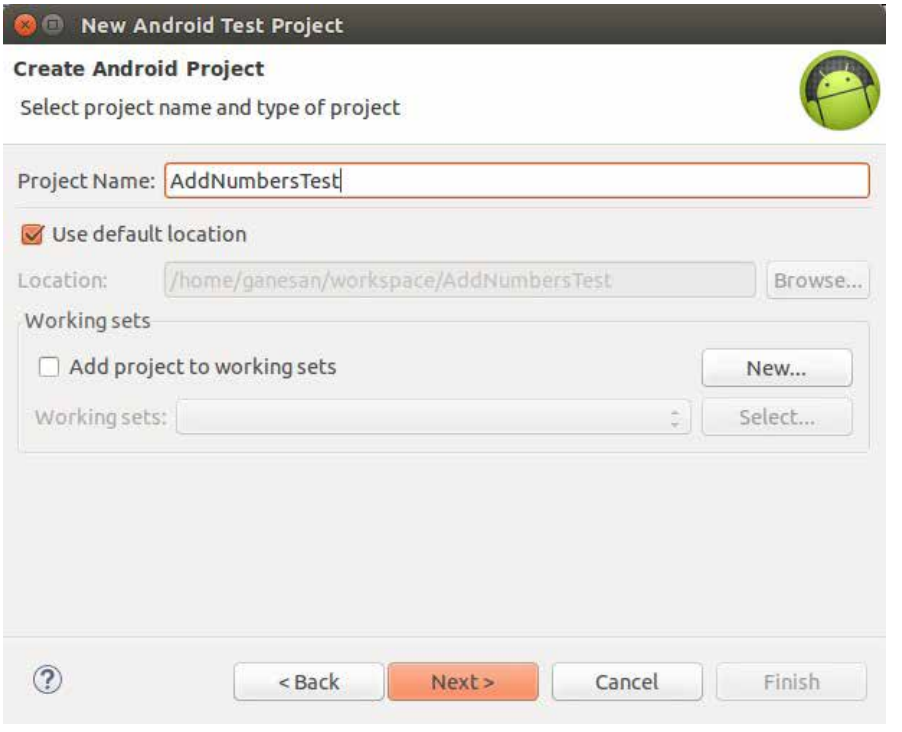


Figure 2.20: New Android Test Project

5. Click Next.

6. Select AddNumbers as the project for which the test project is being created as shown in figure 2.21.

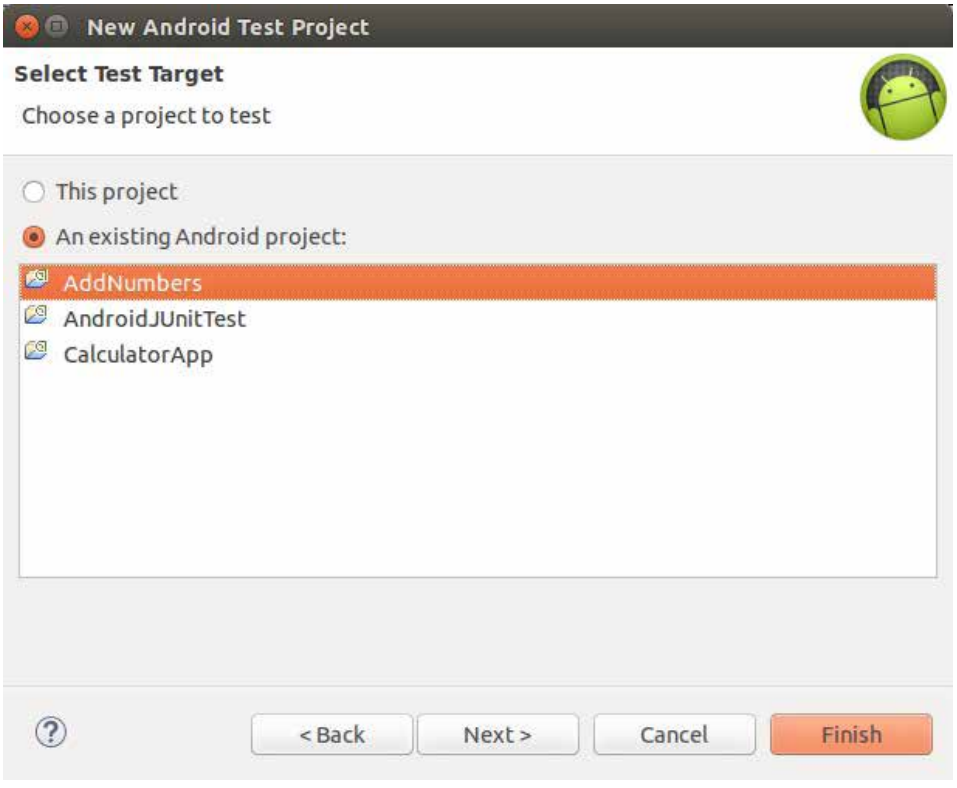


Figure 2.21: Select Test Target Pane

7. Click Next.
8. Select Android 6.0 as the Build Target and click Finish as shown in figure 2.8.
9. Navigate to src → com.example.addnumbers.test folder.
10. Right-click the folder to display the context menu.
11. Click New → Class.
12. Type AddNumbersTestCase in the Name box and click Finish.

13. Modify the code in the class as demonstrated in Code Snippet 9.

Code Snippet 9:

```
package com.example.addnumbers.test;

import com.example.addnumbers.MainActivity;

import android.test.ActivityInstrumentationTestCase2;

import android.test.TouchUtils;

import android.test.suitebuilder.annotation.SmallTest;

import android.view.KeyEvent;

import android.widget.Button;

import android.widget.EditText;

import android.widget.Spinner;

import android.widget.TextView;

public class AddNumbersTestCase extends
ActivityInstrumentationTestCase2<MainActivity> {

MainActivity mActivity;

/** Operator of the Calculator activity */

Spinner mOperator;

/** Operand1 */

EditText mNumber1;

/** Operand2 */

EditText mNumber2;

/** Result */

TextView mResult;

/** Button to perform calculation */

Button mBtnResult;

@SuppressWarnings("deprecation")

public AddNumbersTestCase() {

     super("com.example.addnumbers", MainActivity.class);

}
```

```
@Override
protected void setUp() throws Exception {
      super.setUp();
      /** Starting Calculator Activity */
      mActivity = getActivity();
      /** Getting reference to Operand1 */
      mNumber1 = (EditText) mActivity.findViewById(com.example.
addnumbers.R.id.editText1);
      /** Getting reference to Operand2 */
      mNumber2 = (EditText) mActivity.findViewById(com.example.
addnumbers.R.id.editText2);
      /** Getting reference to Result */
      mResult = (TextView) mActivity.findViewById(com.example.
addnumbers.R.id.textView5);
      /**
       * Getting reference to the calculation performing button
       */
      mBtnResult = (Button) mActivity.findViewById(com.example.
addnumbers.R.id.button1);
}
@Override
protected void tearDown() throws Exception {
      super.tearDown();
}
/** Testing Add Operation */
@SmallTest
public void testAddition() {
      String strNum1 = "1 0 0 PERIOD 5 ";
      String strNum2 = "5 0";
      float expected = 150.5f;
```

```
        /** Inputting Operand1 */
        TouchUtils.tapView(this, mNumber1);
        sendKeys(strNum1);
        /** Inputting Operand2 */
        TouchUtils.tapView(this, mNumber2);
        sendKeys(strNum2);
        /** Performs calculation */
        try {
            runTestOnUiThread(new Runnable() {
                @Override
                public void run() {
                    mBtnResult.performClick();
                }
            });
        } catch (Throwable e1) {
            e1.printStackTrace();
        }
            /** Getting the result shown */
        float actual = Float.parseFloat(mResult.getText().
toString());
            /** Asserts, expected and actual are same */
        assertEquals(expected, actual, 0);
}
}
```

14. Right-click AddNumbersTestCase.java to display the context menu.

15. Select Run As → Android JUnit Test. The output will be as shown in figure 2.22, which shows successful execution of the test.



Figure 2.22: Test – Output

Note - While running the test ensure that the virtual device is not locked. Unlock the device by sliding the lock on the virtual device. Then the test code runs smoothly without setting up any permission or security error.

The key method used in Code Snippet 9 is getActivity(). This method is part of the instrumentation API. The Activity class which is being tested does not begin unless this method is invoked. It is possible to set the test fixture in advance and later call this method to start the activity.

Also, Instrumentation uses the same process to load a test package and the application under check. As a result of this, tests can invoke methods and modify and examine fields in the units of the application being tested.

The various test cases that have access to Instrumentation have the InstrumentationTestCase class as the base class, either directly or indirectly. The most important subclasses for which InstrumentationTestCase class may be a direct or indirect base class are as follows:

- **ActivityTestCase** - base class for activity test classes.
- **SingleLaunchActivityTestCase** - tests a single activity.
- **SyncBaseInstrumentation** - tests synchronization of a content provider.
- **ActivityUnitTestCase** - isolated test of a single activity.
- **ActivityInstrumentationTestCase2** - tests a single activity within a normal system environment.

Using the ActivityTestCase class

This class stores common code for other test cases. This class can be used in cases where the existing alternatives do not fit requirements. Other classes available for meeting the requirements are as follows:

- ActivityInstrumentationTestCase2<E extends Activity>
- ActivityUnitTestCase<E extends Activity>

Figure 2.23 is an UML class diagram of ActivityTestCase and its closely related classes.

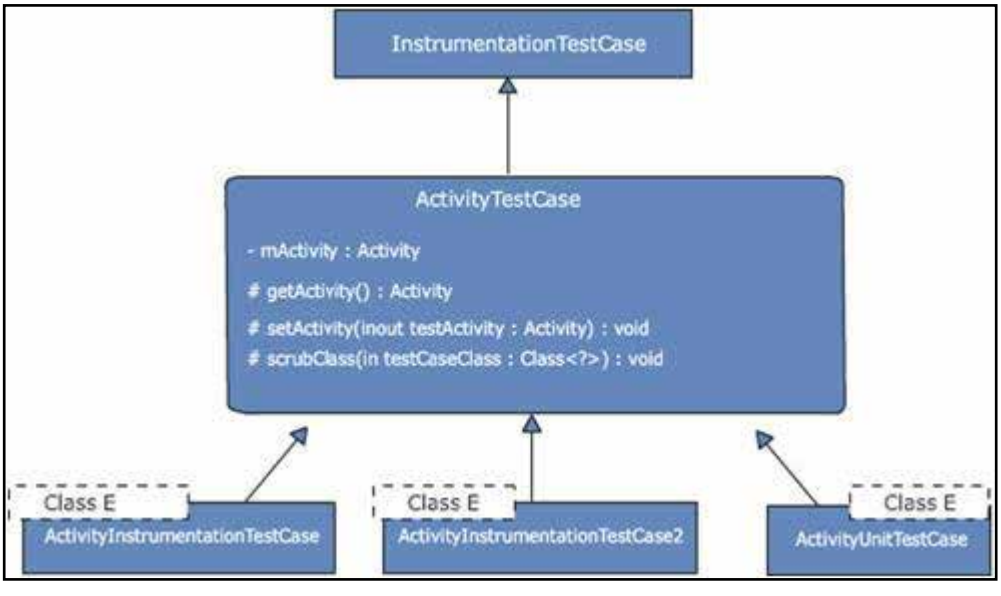


Figure 2.23: UML Class Diagram of ActivityTestCase

It can be seen from figure 2.23 that the ActivityTestCase class extends from the android.test.InstrumentationTestCase class and is the base class for other classes such as ActivityInstrumentationTestCase, ActivityInstrumentationTestCase2, and ActivityUnitTestCase.

- **scrubClass method**

One of the protected methods in this class is the scrubClass() method. The syntax of this method is as follows:

Syntax:

```
protected void scrubClass(Class<?>testCaseClass)
```

where,

testCaseClass: represents the class of the derived TestCase implementation.

This is invoked from the tearDown() method while implementing various test cases to clean up class variables. These may be manifested as non-static inner classes to avoid the need to have references to them.

In case there is a problem in accessing these variables, IllegalAccessException is raised.

- **ActivityInstrumentationTestCase2 class**

This is the most commonly used class for writing Android test cases because it allows functional testing of a single Activity class. The class invokes the InstrumentationTestCase. launchActivity() method and uses the system infrastructure to create the Activity class to be tested. Such action is possible because this class has access to Instrumentation. Figure 2.24 shows the UML class diagram of ActivityInstrumentationTestCase2 and its closely related classes.

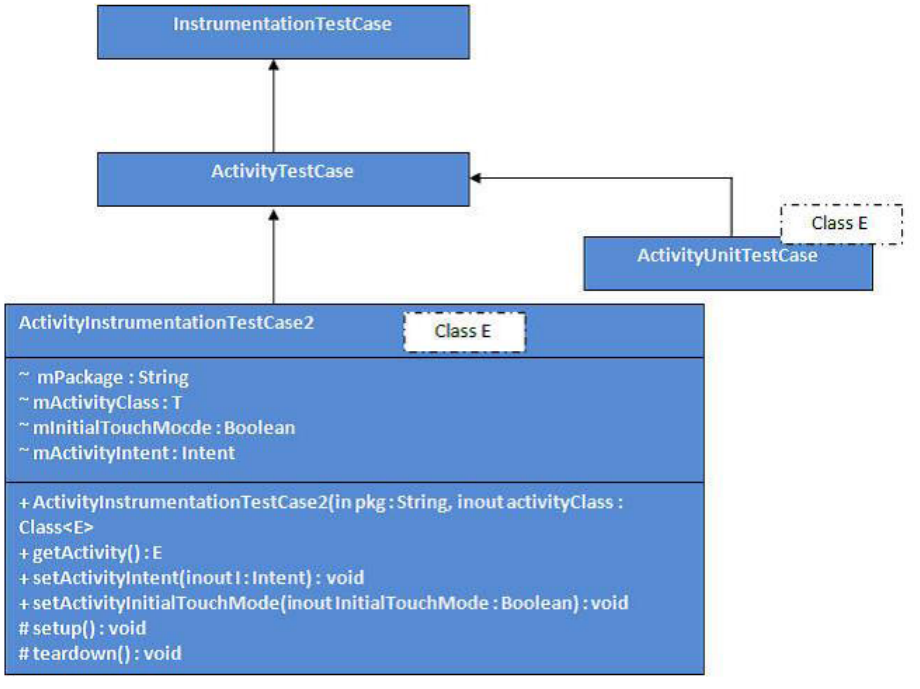


Figure 2.24: UML Class Diagram of ActivityInstrumentationTestCase2

Figure 2.24 shows that the ActivityInstrumentationTestCase2 and ActivityUnitTestCase classes are derived from the ActivityTestCase class. Class template parameter E represents the Activity class. This enables the Activity to be manipulated and monitored after creation.

Suppose there is a requirement to provide a custom Intent to start an Activity, then setActivityIntent(Intent intent) method should be introduced before invoking getActivity() method. This test is very useful for testing user interaction through the interface as events can be introduced to simulate user behavior.

The syntax of the public non deprecated constructor for this class is as follows:

```
ActivityInstrumentationTestCase2(Class<E> activityClass)
```

The constructor can be invoked with the same Activity class that is used as the class template parameter.

- **setUp method**

This method is used for initialization of test case fields and other fixture components that need initialization. Code Snippet 10 demonstrates the use of the setUp() method.

Code Snippet 10:

```
...
protected void setUp() throws Exception {
junitTestClass = new JunitTestClass();
mArg1 = 6;
mArg2 = 3;
super.setUp();
}
...
```

- **tearDown method**

This method is used for clean-up of all fields that were initialized in the setup() method. Code Snippet 11 demonstrates the use of tearDown() method.

Code Snippet 11:

```
…
protected void tearDown() throws Exception {
super.tearDown();
}
...
```

- **testPreconditions method**

This method checks the preliminary conditions so that the test could be executed correctly. Though it is not mandatory that this test will be executed before all other tests, it is advisable to group all preconditions tests under this custom name. Code Snippet 12 demonstrates the use of the testPreconditions() method.

Code Snippet 12:

```
...
public void testPreconditions() {
assertNotNull(indexActivity);
assertNotNull(indexInstrumentation);
assertNotNull(indexLink);
assertNotNull(indexMessage);
assertNotNull(indexCapitalize);
}
...
```

In the Code Snippet, only the Not Null values are checked. By testing the Not Null values, it can be confirmed that the Views were acquired using the specific IDs with the correct types. When Not Null values are unchecked they are assigned using the setup() method.

2.5 Testing Applications Using Android Studio

Android Studio 0.1.8 onwards, support for running Instrumentation tests was added. A test application is integrated into the application project. There is no need to create a separate test project.

Version 1.1 of Android Studio and the Android gradle plugin (Gradle is a build system that takes the best features from other build systems and combines them into one) brought support for unit testing your code on your development computer. However, this feature is still considered experimental.

Along with the main sourceSet, the androidTest sourceSet is created which is located by default in src/androidTest/ folder. From this sourceSet, a test apk is built that can be deployed to a device to test the application using the Android testing framework. This can contain unit tests, instrumentation tests, and later uiautomator tests.

The <instrumentation> node of the manifest for the test app is generated. However, you can create a src/androidTest/AndroidManifest.xml file to add other component(s) to the test manifest. To test an application in Android Studio, perform the following steps:

1. Start Android Studio and click File → New → New Project.
2. Enter the application name as CalculatorApplication and other details as shown in figure 2.25.

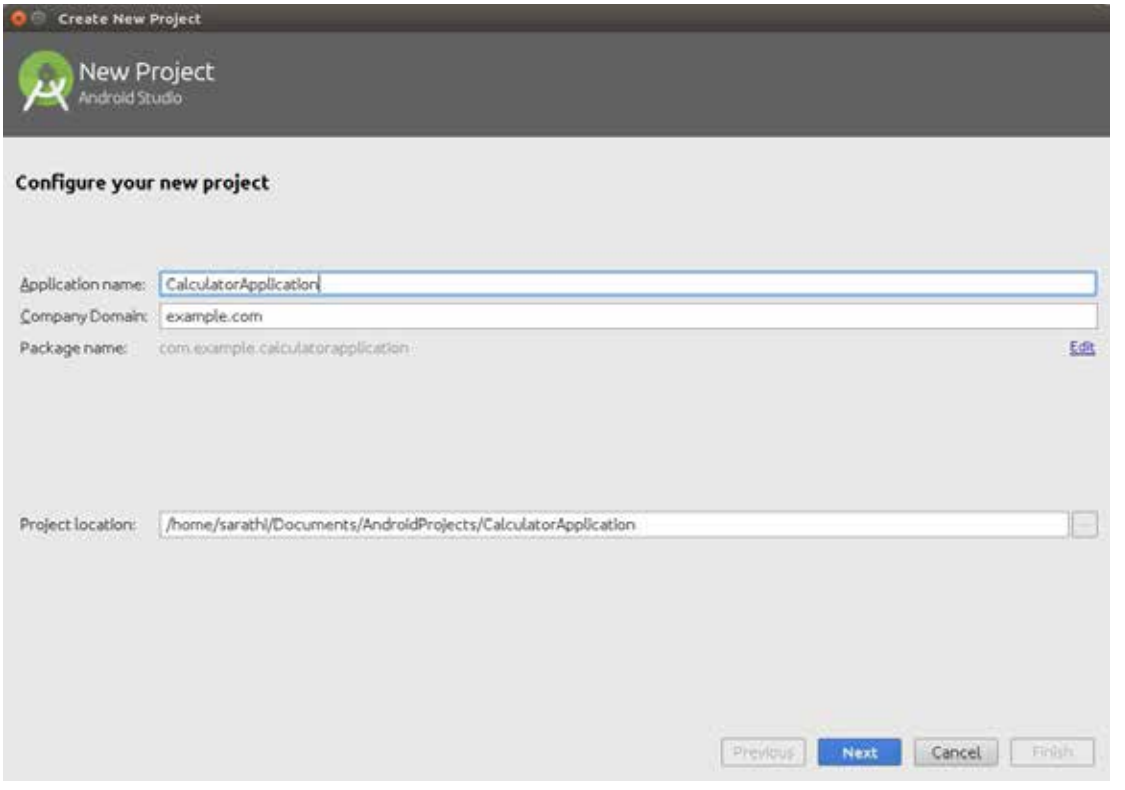


Figure 2.25: New Project Details

3. Click Next.

4. Select API 23: Android 6.0 (Marshmallow) from the Minimum SDK drop-down list as shown in figure 2.26.

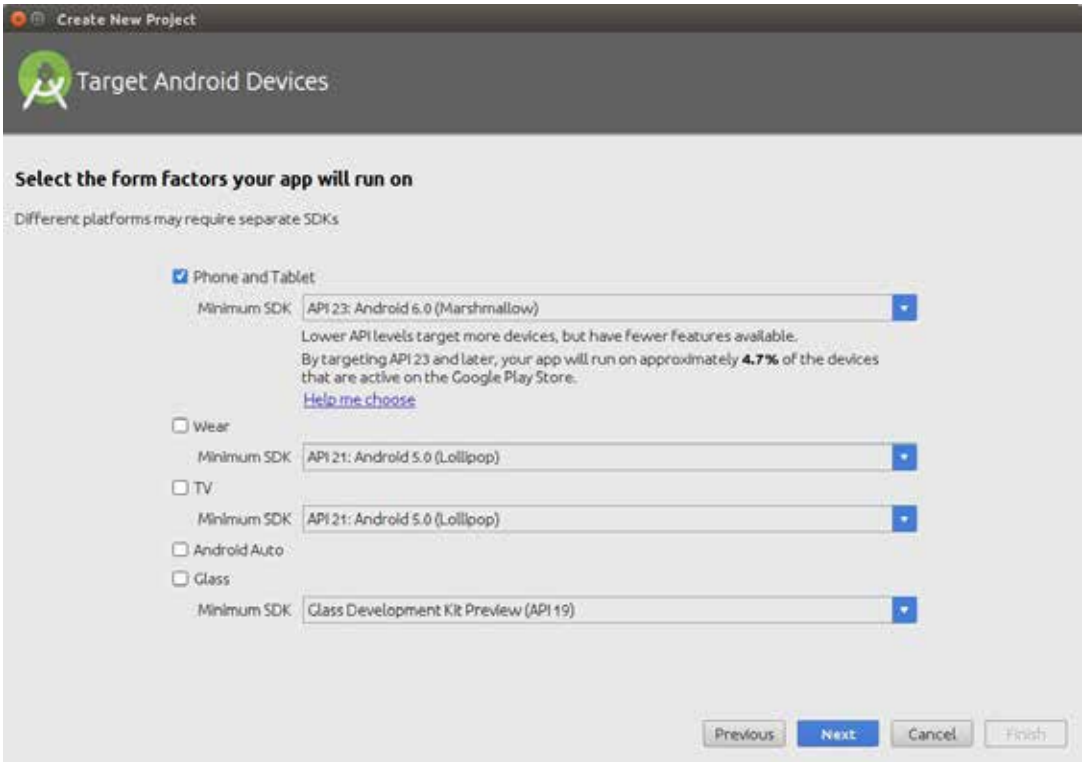


Figure 2.26: Selecting Minimum SDK

5. Click Next.

6. Select Blank Activity and click Next.

7. Specify a new Activity Name or leave the defaults and click Finish as shown in figure 2.27.

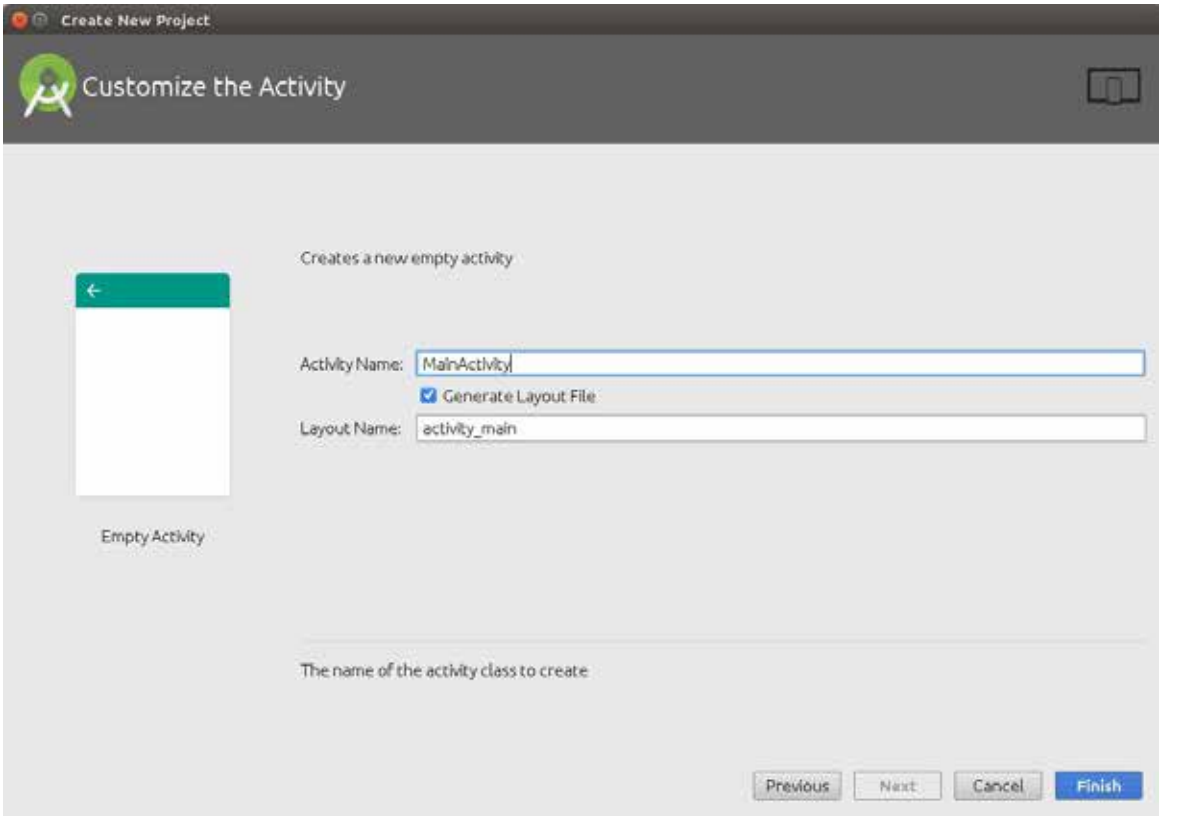


Figure 2.27: Providing Activity Name

8. Navigate to com.example.calculatorapplication.
9. Right-click the folder and select New → Java Class.
10. Specify class name as Calculator and click OK in the Create New Class dialog box.
11. Add the code demonstrated in Code Snippet 13 to the Calculator class.

Code Snippet 13:

```
public int multiply(int a, int b){
        return (a*b);
   }
```

12. Navigate to com.example.calculatorapplication (androidTest).
13. Right-click the folder and select New → Java Class.
14. Specify class name as Test1 and click OK in the Create New Class dialog box.

15. Modify the code in the Test1 class as demonstrated in Code Snippet 14.

Code Snippet 14:

```
import android.test.InstrumentationTestCase;
public class Test1 extends InstrumentationTestCase {
Calculator obj1;
public void test() throws Exception{
        final int a = 4;
        final int b = 5;
        int expected = 24;
        obj1 = new Calculator();
        int reality = obj1.multiply(a, b);
        assertEquals(expected, reality);
    }
}
```

In the code, the Test1 class extends InstrumentationTestCase. This indicates that it is a test class. Note that any method that must run as a test should be prefixed with the word 'test'. Within the test() method an object of Calculator class is created. Further, the multiply() method of the class is invoked and the output is tested against an assertion.

16. Click Run → Edit Configurations.

17. Expand Defaults and select Android Tests in the Run/Debug Configurations dialog box as shown in figure 2.28.

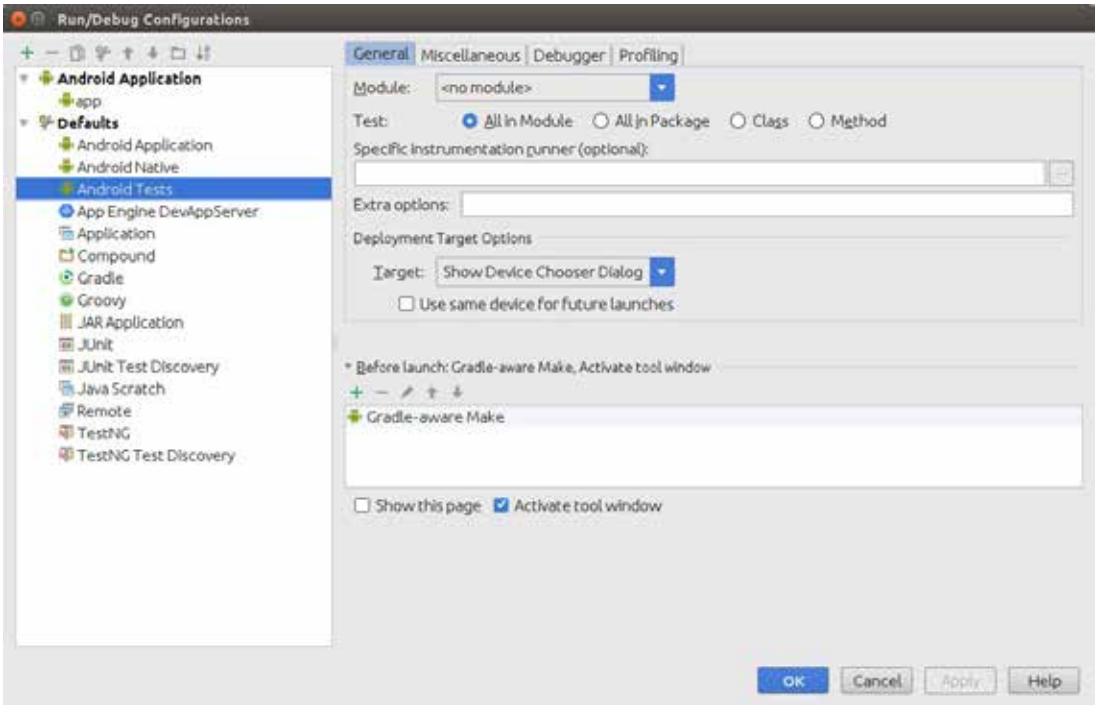


Figure 2.28: Creating Android Test Configuration

18. Click the + sign and select Android Tests as shown in figure 2.29.

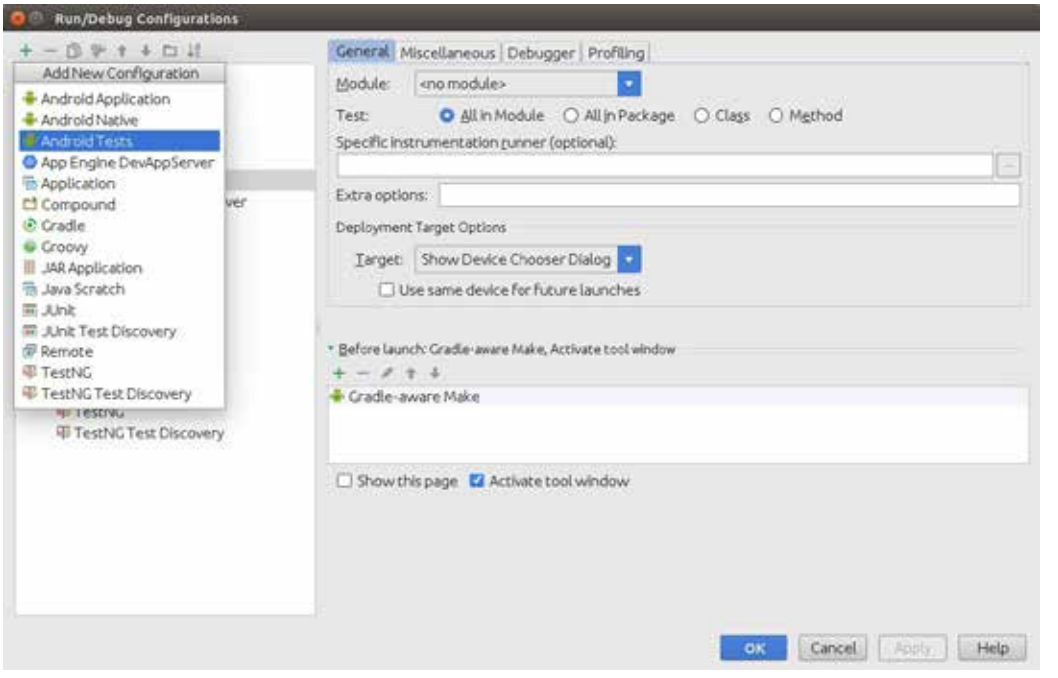


Figure 2.29: Creating New Run Configuration

19. Specify the name and other configuration information as shown in figure 2.30.

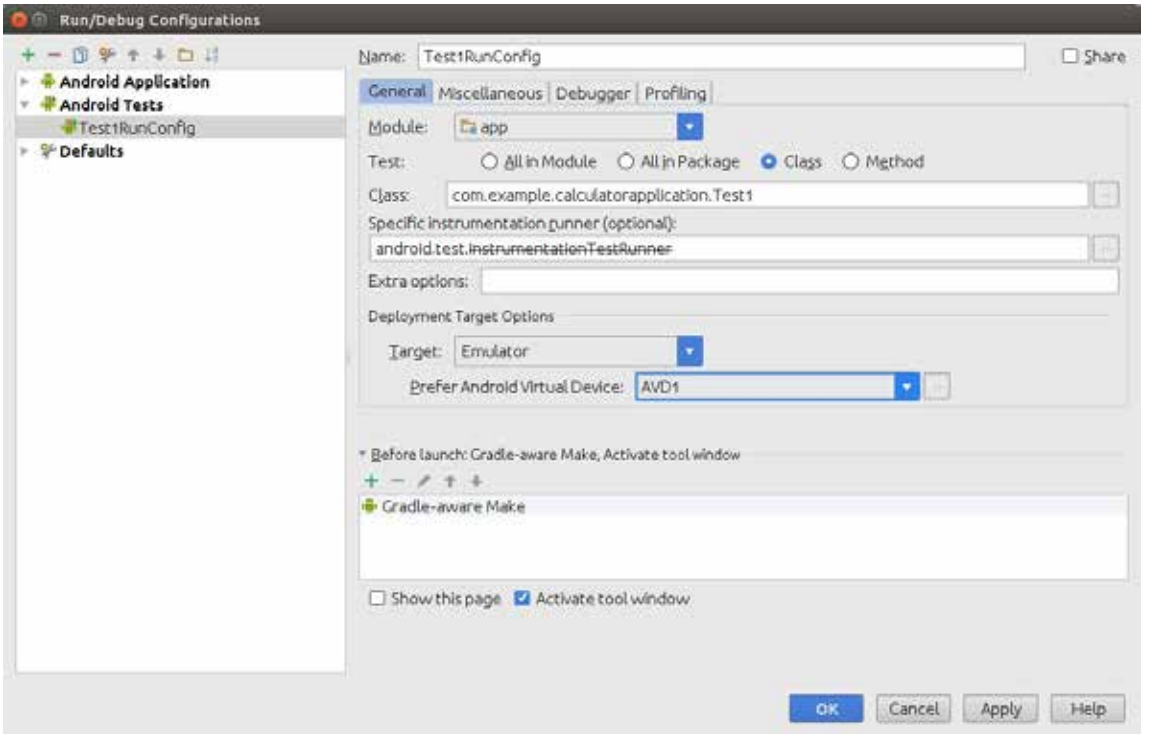


Figure 2.30: Specifying Run Configuration Information

20. Click OK.

21. Click the Run symbol against Test1RunConfig as shown in figure 2.31.

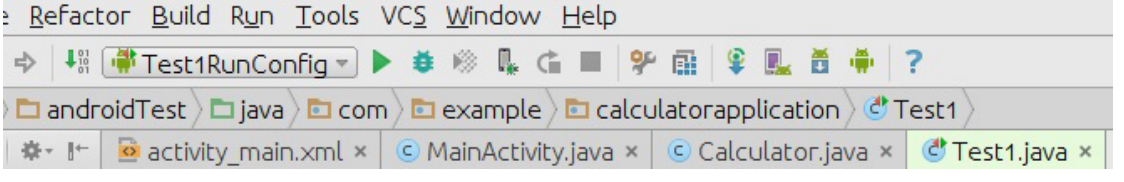


Figure 2.31: Running the Test

The output of the test run is shown in figure 2.32.

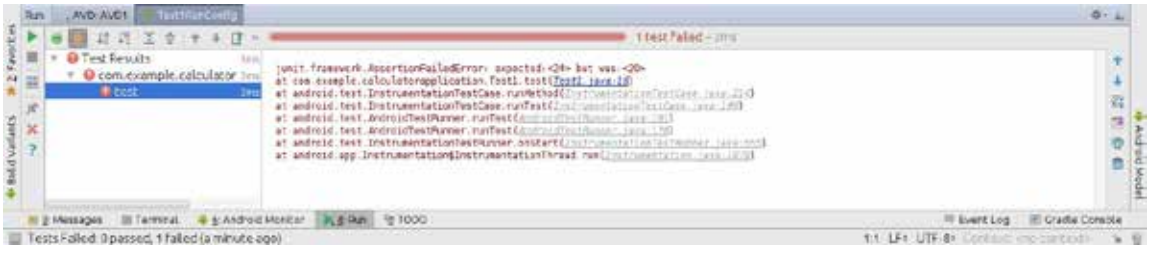


Figure 2.32: Test Run Output

Since, the value returned by the multiply() method did not match the expected value, the assertion fails and leads to AssertionFailedError exception.

2.5.1 Testing from Gradle

Running tests on an Android emulator or device is slow because building, deploying, and launching the app often takes a minute or more. This makes testing applications very time consuming and is not a correct approach for Test-driven Development (TDD).

Unit tests can be run on a local JVM on your development machine using the gradle plugin. This plugin compiles source code present in src/test/java and executes it using the usual Gradle testing mechanisms. At runtime, tests get executed against a modified version of android.jar. In this jar, all final modifiers are stripped off. This allows usage of popular mocking libraries such as Mockito.

For testing using gradle, testing dependencies of the application must be specified in the build.gradle file of the android module. For example:

```
dependencies {
  testCompile 'junit:junit:4.12'
  testCompile "org.mockito:mockito-core:1.9.5"
}
```

- **Setting Up Android Studio**

  To utilize the unit testing support in Android Studio, the following settings need to be done:

  1. Update build.gradle to use the android gradle plugin version 1.1.0-rc1 or later.

  This can be done manually in build.gradle file or in the UI in File → Project Structure as shown in figure 2.33.

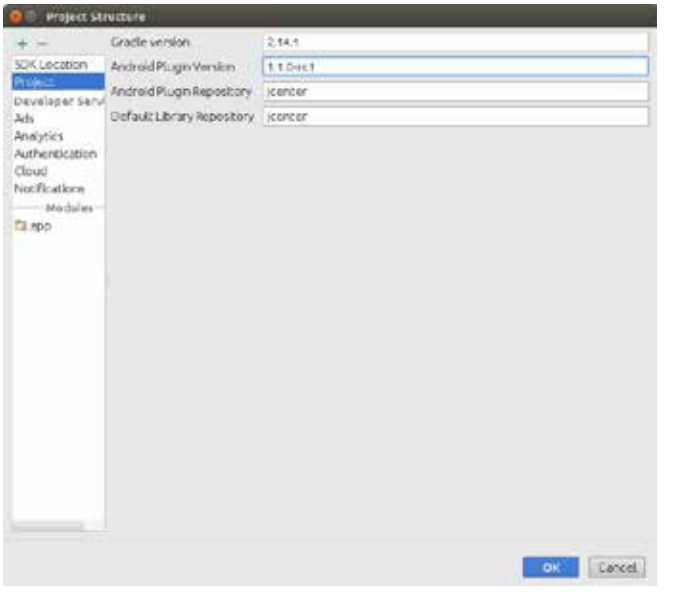


Figure 2.33: Setting Gradle Version

2. Add necessary testing dependencies to app/build.gradle as shown in figure 2.34.



Figure 2.34: Adding Dependencies

3. Sync your project by right-clicking app and selecting Synchronize 'app'.

4. Open the Build Variants tool window (on the left) and change the Test Artifact to 'Unit Tests' as shown in figure 2.35.

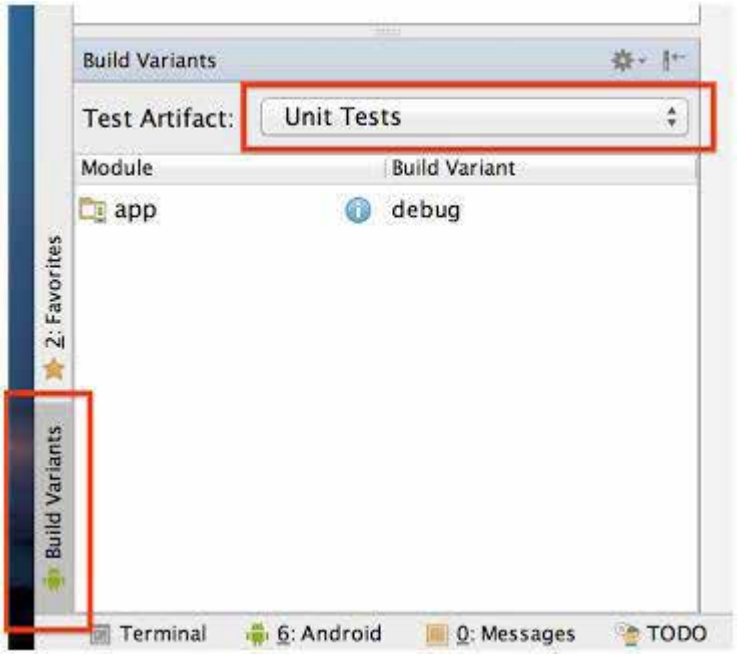


Figure 2.35: Setting Build Variants

5. Create a directory for your testing source code, that is, src/test/java. This can be done from the command line or using the Project view in the Project tool window.

> **Note** - Test source directories names are determined by the gradle plugin based on a convention.

6. Create a new test. This can be done by opening a class, right-clicking its name and selecting Go to → Test.

7. Add some test cases.

8. Right-click a test class or method and select 'Run'.

The compilation time can be reduced by using Gradle directly. This can be done as follows:

1. Go to the Run menu and select 'Edit configurations'.
2. In the default JUnit template, remove the 'Make' Beforelaunch step and add a 'Gradle aware make' step instead.
3. Leave the task name empty.

Android Studio allows usage of different third party testing frameworks to create and run unit tests. Some of the popular frameworks are as follows:

- **Robolectric**

Robolectric is a unit test framework that modifies the Android SDK jar to allow you to test-drive the development of your Android app. With Robolectric, tests run inside the JVM on your workstation in just few seconds. Using Robolectric tests can be written as demonstrated in Code Snippet 15.

Code Snippet 15:

```
@RunWith(RobolectricTestRunner.class)
public class NewActivityTest {
  @Test
   public void clickingButton_shouldChangeResultsViewText()
throws Exception {
 NewActivity activity = Robolectric.setupActivity(NewActivity.
class);
       Button btnClick = (Button) activity.findViewById(R.
id.button);
    TextView output = (TextView) activity.findViewById(R.
id.results);
    btnClick.performClick();
  assertThat(output.getText().toString()).isEqualTo("Android
Rocks!");
  }
}
```

This is possible using Robolectric because it rewrites the Android SDK classes as they are being loaded and makes it possible for them to run on a regular JVM.

SDK, Resources, and Native Method Emulation

Robolectric handles inflation of views, resource loading, and lots of other features that are implemented in native C code on Android devices. Due to this, tests can perform most of the actions that are done on a real device. It allows to use your our own implementation for specific SDK methods.

Therefore, you can simulate error conditions or real-world sensor behavior, for example:

- **Run Tests Outside of the Emulator**

With Robolectric, tests can be run on your workstation or on your Continuous Integration environment in a regular JVM, without an emulator. Due to this, the dexing, packaging, and installing-on-the emulator steps are not required. This reduces test cycles from minutes to seconds leading to quick iteration and refactoring of code.

Note - The compiled Java code of Android is called dex (Dalvik Executable).

- **No Mocking Frameworks Required**

Instead of Robolectric, you can also use mock frameworks such as Mockito. This is also a valid approach however; the tests yielded from this are often reverse implementations of the application code.

On the other hand, Robolectric testing style is similar to black box testing. This makes the tests more effective for refactoring. Also, the tests can focus more on the behavior of the application instead of the implementation of Android. However, you can still use a mocking framework along with Robolectric.

- **Mockito**

Mockito is an open source testing framework for Java released under the MIT License. It allows testing double objects (mock objects) in automated unit tests for the purpose of Test-driven Development (TDD) or Behavior Driven Development (BDD). Mockito library enables mocks creation, verification, and stubbing.

Building Local Unit Tests

Dependencies and Prerequisites: Android Plug-in for Gradle 1.1.0 or higher.

If the unit test does not have any dependencies or only few simple dependencies on Android, the test should be run on a local development machine.

This is an efficient testing methodology as it helps to avoid loading if target app as well as unit test code on an emulator or physical device every time the test is run. This further leads to reduced test execution time. For this approach, you can use a mocking framework such as Mockito to fulfill any dependency relationships.

The Android Plug-in for Gradle version 1.1.0 and higher allows creation of a source directory src/test/java in the application. This directory stores JUnit tests that are to be run on a local machine. This leads to improved project organization as the unit tests are grouped together into a single source set. The tests can be executed from Android Studio or the command-line. The plugin executes the tests on the local Java Virtual Machine (JVM) on your development machine.

Set Up Your Testing Environment

Before building local unit tests, perform the following steps:

- Set up your project structure - In a Gradle project, the source code for the target app is placed in the app/src/main/java folder. The source code for your local unit tests must be placed under the app/src/test/java folder.
- Specify your Android testing dependencies - To use JUnit 4 and Mockito with the local unit tests, certain libraries must be specified as dependencies in the build. gradle file of your Android app module. This is demonstrated in Code Snippet 16.

Code Snippet 16:

```
dependencies {
    // Unit testing dependencies
    testCompile 'junit:junit:4.12'
    // Set this dependency if you want to use Mockito
    testCompile 'org.mockito:mockito-core:1.10.19'
   // Set this dependency if you want to use Hamcrest matching
    androidTestCompile 'org.hamcrest:hamcrest-library:1.1'
}
```

Code Snippet 16 shows the junit 4.12 and mockito library dependencies added to the build.gradle file.

Create a Local Unit Test Class

The local unit test class should be written as a JUnit 4 test class. Unlike the earlier approach to Android unit testing based on JUnit 3, with JUnit 4, you do not need to extend the junit.framework.TestCase class. Also, a test method need to be prefixed with the 'test' keyword. No classes need to be used in the junit.framework or junit. extensions package.

To create a basic JUnit 4 test class, create a Java class that contains one or more test methods. A test method begins with the @Test annotation and contains the code to test a single functionality in the component of your application that needs to be tested.

Code Snippet 17 demonstrates implementation of a local unit test class.

Code Snippet 17:

```
import org.junit.Test;

import java.util.regex.Pattern;

import static org.junit.Assert.assertFalse;

import static org.junit.Assert.assertTrue;

public class EmailValidatorTest {

@Test

public void emailValidatorTestMethod() {

assertThat(EmailValidator.isValidEmail("sample@email.com"),
is(true));

 }

 ...

}
```

The test method emailValidatorTestMethod verifies that the isValidEmail() method in the application under test returns the correct result.

The junit.Assert methods are used to test that components of the application under test, return the expected results. These methods allow performing validation checks (or assertions) to compare the status of the component under test against some expected value. The tests can be made more meaningful by using Hamcrest matchers such as the is() and equalTo() methods, to match the returned result against the expected result.

Several annotations can be used in the JUnit 4 test class to call out sections in the test code for special processing. Some of the annotations are as follows:

- **@Before**: This annotation is used to specify a block of code with test setup operations. It will be invoked before each test. There can be multiple @Before methods but the order which these methods are called is not fixed.
- **@After**: This annotation is used to specify a block of code with test tear-down operations. It will be called after every test method. There can be multiple @ After operations in the test code. This annotation is used to release any resources from memory.

- **@Test**: This annotation is used to mark a test method. There can be multiple test methods in a single test class. Each method should be prefixed with this annotation.
- **@BeforeClass**: This annotation is used to specify static methods to be invoked only once per test class. It is mainly useful for expensive operations such as connecting to a database.
- **@AfterClass**: This annotation is used to specify static methods to be invoked only after all tests in the class have been run. It is useful for releasing any resources allocated in the @BeforeClass block.
- **@Test(timeout=<milliseconds>)**: This annotation specifies a timeout period for the test. If the test starts but does not complete within the given timeout period, it automatically fails. The timeout period must be specified in milliseconds, for example: `@Test(timeout=4000)`.

Check Your Progress

1. Which of the following test method statements test a part of the application under test?

(A)	JUnit test	(B)	Instrumentation
(C)	AndroidTestCase	(D)	ApplicationTestCase

2. A _____ is a directory or an Eclipse project that creates the source code and manifest files for a test package.

(A)	test project	(B)	command-line tools
(C)	SDK tools	(D)	JUnit tests

3. One of the functions of Android instrumentation is to _____.

(A)	run an Android component independently of its normal cycle	(B)	build files and manifests files and directory structure
(C)	automatically assign InstrumentationTestRunner	(D)	test each part of an application

4. Which class is used to test a single activity within a normal system environment?

(A)	ActivityUnitTestCase	(B)	SingleLaunchActivityTestCase
(C)	ActivityTestCase	(D)	ActivityInstrumentationTestCase2

5. The class that is used for isolated testing of a single ContentProvider is _____.

(A)	ProviderTestCase2	(B)	ActivityUnitTestCase
(C)	ServiceTestCase	(D)	ApplicationTestCase

Check Your Progress

6. Which of the following testing frameworks can be used with Android Studio for unit testing?

(1)	Mockito	(2)	JSF
(3)	Spring	(4)	Robolectric

(A)	2, 3	(B)	1, 3
(C)	1, 4	(D)	1, 2

Answers

1.	A
2.	A
3.	A
4.	D
5.	A
6.	C

Summary

- Android has a built-in testing framework that helps in testing every unit of an application.
- Android build and test tools assume that the test projects consists of an organized and standard structure of tests, test case classes, test packages, and test projects.
- The Android testing API is modeled on the JUnit API. It has an instrumentation framework and Android-specific testing classes.
- A JUnit can be called as a method that allows testing of each part of an application.
- The Android system has a set of control methods that enables an Android component to run independently of its normal cycle and determines how Android loads an application(s).
- ActivityInstrumentationTestCase2 is the most commonly used class for writing tests. It tests a single activity within a normal system environment.
- Android Studio added support for running Instrumentation tests whereby building a test application is integrated into the application project.

Onlinevarsity

GO DIGITAL



03 Components Used for Creating Tests



Welcome to the Session, **Components Used for Creating Tests**.

This session describes the various components that are required to create a test. These can be broadly divided into various classes namely, Assert, TouchUtils, Mock Object, and TestCase base class. This session will describe these classes along with their subclasses and examples.

In this Session, you will learn to:

- Identify the different classes used for creating tests
- Explain in detail the classes used for creating tests
- Describe the methods under each of these classes

Session

03

Components Used for Creating Tests

3.1 Introduction

Assert, TouchUtils, Mock Object, classes, and the TestCase base class are some of the building blocks required to create tests. Assert class is the base class for all test case classes that are used for writing tests. TouchUtils class extends from the InstrumentationTestCase and consists of a number of reusable methods for generating touch events.

Mock Object classes stub out the corresponding system object class and override methods to provide mock dependencies. Mock objects are duplicate objects that are called instead of the real domain objects to allow testing of units in isolation. TestCase class is the base class for all test cases and provides the basic methods in the JUnit framework.

3.2 Android Assertion Class

Assertion is a method that evaluates a condition and depending on the result of evaluation, throws an exception and aborts the program or normally executes the program. Assertion methods can be used to display the result of tests as Android test case classes extend JUnit. It is more convenient to use and provides improved test performance.

The characteristics of the Android Assert class are as follows:

- Is a part of the JUnit API base class for all of the test case classes.
- Has several assertion methods that are used for writing tests. Assertion methods evaluate the actual results to an expected result value.
- Presents the possible types of comparisons.
- Tests the UI.
- Tests a variety of conditions.
- Supports different parameter types.
- Displays messages only when assertion fails.

Depending on the conditions that they check, they can be grouped into different sets as listed in table 3.1.

Methods	Description
assertEquals()	Affirms that both objects passed as parameters are equal by invoking the equals() method of the object.
assertFalse()	Affirms that the condition is false.
assertNotNull()	Affirms that an object is not null.
assertNotSame()	Affirms that both the objects do not refer to the same object.
assertNull()	Affirms that an object is null.
assertSame()	Affirms that both the objects refer to the same object.
assertTrue()	Affirms that the condition is true.
fail()	Fails a test with a given/no message.

Table 3.1: Assert Methods and their Description

As the name indicates, each condition tested is identified by the method name. When any one of these conditions fails, the execution of the test is aborted and an AssertionFailedError exception is thrown.

The fail() method is used under the following two conditions:

- When there is a need to create a test that is not implemented immediately but is flagged for later use. In this condition, the fail() method always fails and indicates the condition with a custom message as demonstrated in Code Snippet 1.

Code Snippet 1:

```
…
public void testNotdone() {
      fail("Not done yet");
}
…
```

- When there is a need to test if a method has thrown an exception. In such a case, the method invocation code is surrounded by a try-catch block and the fail() method is forcefully invoked if the exception is not thrown. Code Snippet 2 demonstrates the use of this condition.

Code Snippet 2:

```
…
public void testCheckShouldThrowException() {
try {
HelloProjectActivity.hellomethodThatShouldThrowException();
fail("Exception not thrown");
} catch ( Exception ex ) {
// do nothing
}
}
…
```

3.2.1 Using Custom Messages

All methods of the Assert class have an overloaded version which includes a custom String message. When an assertion fails, the custom messages are printed by the test runner instead of a default message. Custom messages help in identifying the failure and are often prescribed as a best practice. Code Snippet 3 demonstrates the use of a custom message.

Code Snippet 3:

```
…
public void testMax() {
  final int x = 1;
  final int y = 2;
  final int expected = y;
  final int actual = Math.max(x, y);
   assertEquals(" Exception " + expected + " but was " + actual,
expected, actual);
}
…
```

3.2.2 Importing Static Methods

Some assertion methods need specific import statements to improve the readability of the tests, though the basic assertion methods extend from the Assert base class. A specific practice is followed to import assert methods from the corresponding classes.

The developer can either use the static assert method prefixed with the package and class name or use the static import statements. Code Snippet 4 demonstrates the use of the static assert method prefixed with the package and the class name.

Code Snippet 4:

```
…
public void testAlignment() {
  final int edge = 0;
  ...
```

```
        android.test.ViewAsserts.assertRightAligned(indexMessage,
indexCapitalize, edge);
}
…
```

Code Snippet 5 demonstrates the use of static import statement for importing the static method and then, using the static method within another method in the class.

Code Snippet 5:

```
import static android.test.ViewAsserts.assertRightAligned;
public void testAlignment() {
  final int edge = 0;
  assertRightAligned(indexMessage, mindexCapitalize, edge);
}
…
```

To work with static import methods, add it to the Favorites list of Eclipse IDE. To add the static imports method, perform the following steps:

1. Click Window → Preferences as shown in figure 3.1.

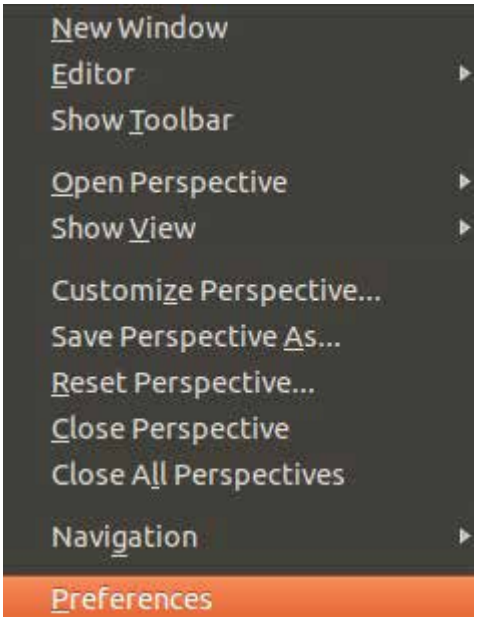


Figure 3.1: Selecting Preferences

This will display the Preferences dialog box.

2. In the left pane of the Preferences dialog box, select Java → Editor → Content Assist → Favorites to view the Favorites pane as shown in figure 3.2.

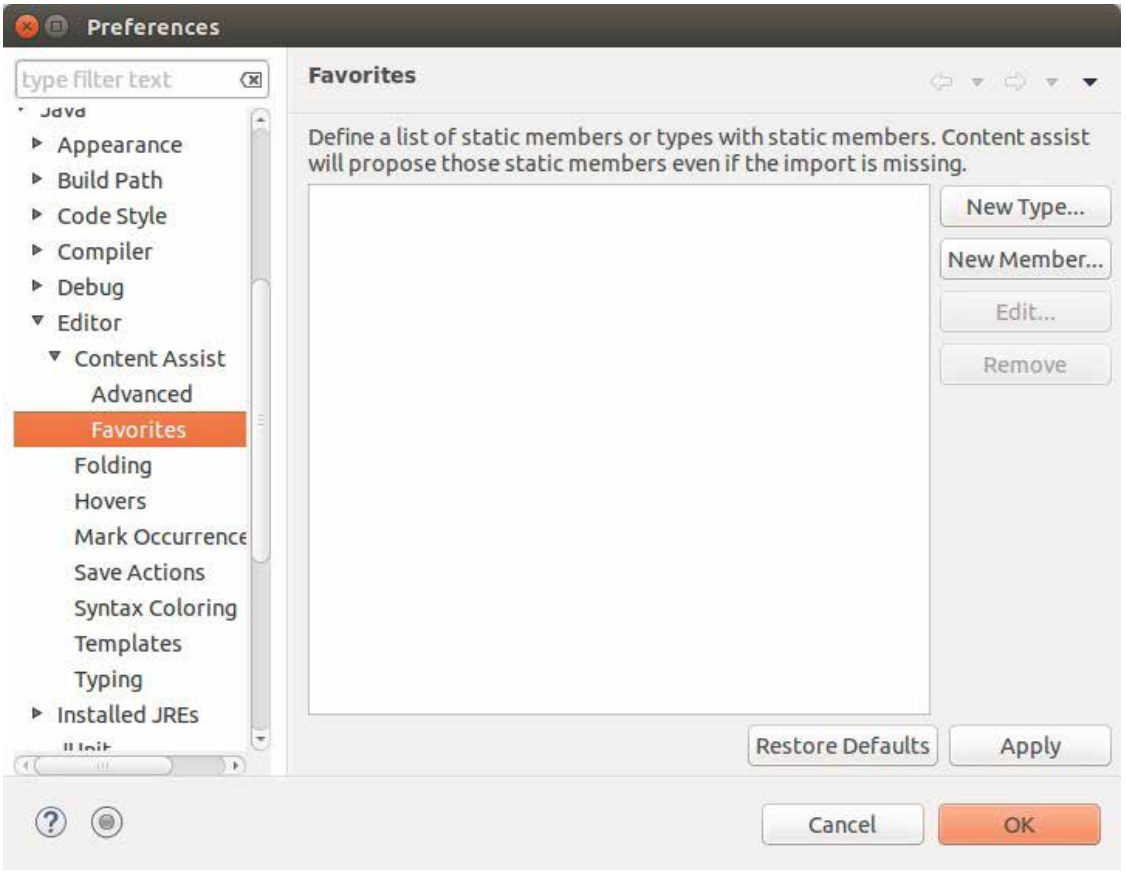


Figure 3.2: Favorites Pane in the Preferences Dialog Box

3. Click New Type to display the New Type Favorite dialog box as shown in figure 3.3.

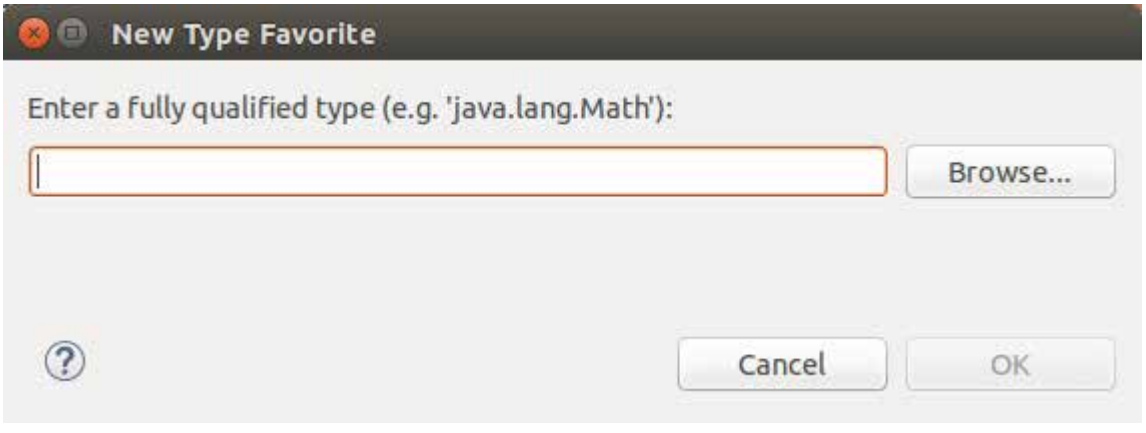


Figure 3.3: New Type Favorite Dialog Box

4. Type android.test.MoreAsserts to add the assert class and click OK.

5. Repeat steps 3 and 4 to add android.test.ViewAsserts class as shown in figure 3.4.

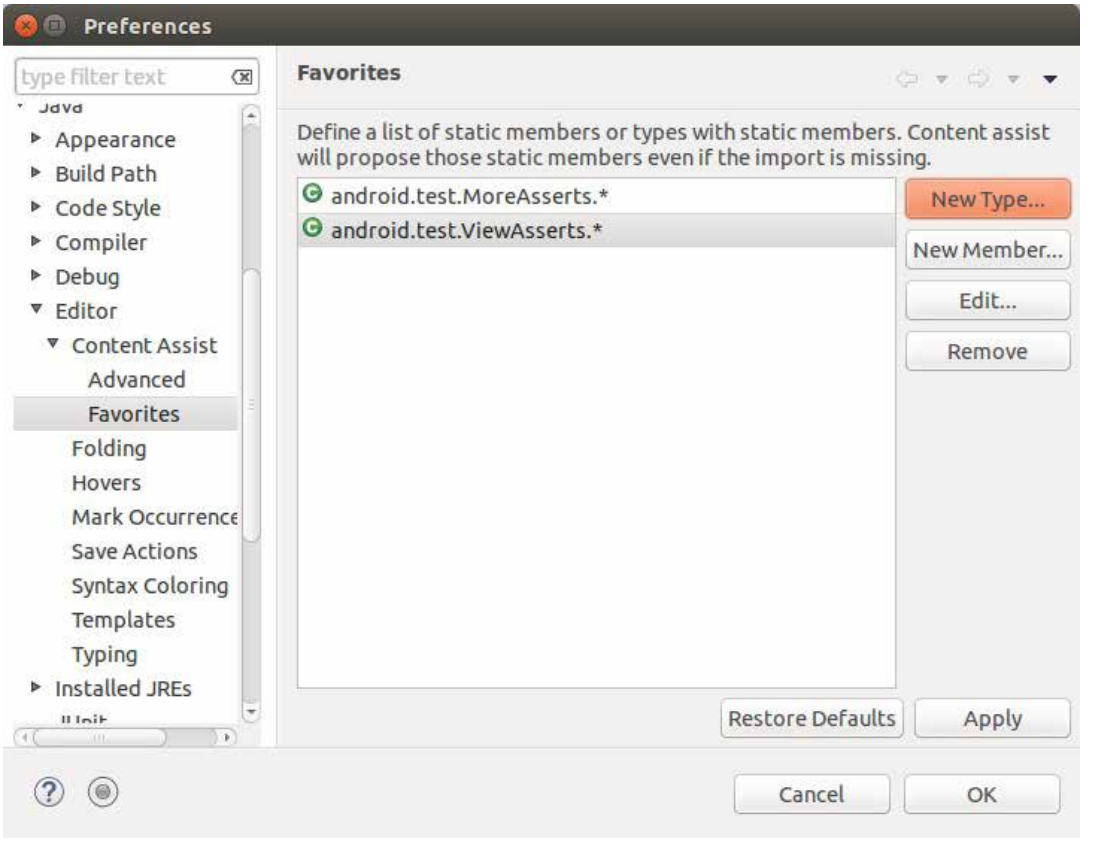


Figure 3.4: Adding Classes to the Favorites List

6. Click OK.

3.2.3 ViewAsserts Class Methods

Besides the JUnit Assert class that is used to test simple conditions or objects, Android provides the ViewAsserts and MoreAsserts classes for testing complex conditions. These classes are present in the android.test package. ViewAsserts class provides advanced methods that are used for testing user interfaces. The android.test.ViewAsserts class has different assertion methods that test not only the views on screen but also their absolute and relative positions on the screen.

Table 3.2 lists some of the methods of the class along with their description.

Methods	Description
`static void assertBaselineAligned (View v1, View v2)`	The method checks that the two views are baseline aligned with their baselines on the same y location.
`assertBottomAligned (View v1, View v2)`	The method checks that the two views have their bottom aligned with their bottom edges on the same y location.
`assertGroupContains (ViewGorup Parent, View child)`	The method affirms that the specified group contains a specific child once and only once.
`AssertGroupNotContains (ViewGorup Parent, View child)`	The method checks that the specified group does not contain a specific child.
`assertHasScreenCoordinates (View orgin,View view, int x, int y)`	The method tests whether there is a particular x and y position for each view on the visible screen.
`assertHorizontalCenterAligned (View reference, View Test)`	The method checks that the test view is horizontally center aligned in relation to the reference view.
`assertOffScreenAbove (View orgin, View view)`	The method affirms that the specified view is visible above the visible screen.
`assertOffScreenBelow (View orgin, View view)`	The method affirms that the specified view is visible below the visible screen.
`assertOnScreen (View orgin, View view)`	The method affirms that a view is visible on the screen.
`assertRightAligned (View first, View Second, int margin)`	The method asserts that two views are right-aligned with their right edges on the same x location. An optional margin can also be given.
`assertTopAligned (View First, view Second, int margin)`	The method tests that two views are top-aligned with their top edges on the same y location. An optional margin can also be specified.
`assertVerticalCenterAligned (View reference, View Test)`	The method checks whether the test view is vertically center aligned in relation to the reference view.

Table 3.2: ViewAsserts Methods for Different Conditions

For further understanding of the role of ViewAsserts class, consider Code Snippet 6 where ViewAsserts class is used to test the user interface layout.

Code Snippet 6:

```
…
@Test
public final void testFontSizes() {
  final float expected = 24.0f;
  assertEquals(expected, textView1.getTextSize(), 0);
  assertEquals(expected, textView2.getTextSize(), 0);
}

@Test
public final void testMargins() {
  LinearLayout.LayoutParams lp;
  final int expected = 6;
  lp = (LinearLayout.LayoutParams) Layout1.getLayoutParams();
  assertEquals(expected, lp.leftMargin);
  assertEquals(expected, lp.rightMargin);
  lp = (LinearLayout.LayoutParams) Layout2.getLayoutParams();
  assertEquals(expected, lp.leftMargin);
  assertEquals(expected, lp.rightMargin);
}
…
```

3.2.4 MoreAsserts Class Methods

MoreAsserts class is responsible for regular expression matching as they contain methods such as assertContainsRegex (String, String). Similar to ViewAsserts, this class contains additional assertion methods.

Table 3.3 lists some of the methods of this class along with their description.

Methods	Description
`assertAssignableFrom`	The method check that class expected is assignable from the object actual.
`assertContainsRegex`	The method tests the match between an expected Regex and any substring of the specified String. When there is no match the specified message is failed.
`assertContainsInAnyOrder`	The method reviews whether the specified Iterable contains the expected elements, but in any order.
`assertContainsInOrder`	The method affirms that the specified Iterable contains the expected elements, in the same order.
`assertEmpty`	The method asserts that an Iterable is empty.
`assertEquals`	The method confirms that the JUnit asserts does not cover some Collections.
`assertMatchesRegex`	The method checks whether there is an exact match between the specified Regex and the String. When there is no match the provided message is failed.
`assertNotContainsRegex`	The method affirms that there is no match between the specified Regex and any substring of the specified String, fails with provided message if it does.
`assertNotEmpty`	The method confirms that some Collections not covered in JUnit asserts are not empty.
`assertNotMatchesRegex`	The method tests that there is no exact match between the specified Regex and the specified String, fails with the provided message if it does.
`checkEqualsAndHashCodeMethods`	The method tests `equals()` and `hashCode()` results at once. Tests that equals() applied to both objects matches the specified result.

Table 3.3: Methods of MoreAsserts Class

3.3 TouchUtils Class

The TouchUtils class provides one of the simplest ways to stimulate different kinds of touch events in the UI. This class is used with the InstrumentationTestCase or ActivityInstrumentationTestCase2 for interaction with the application through a touch screen. It consists of a number of reusable methods for generating touch events.

TouchUtils class supports the following reusable methods:

- Clicking a View and releasing
- Tapping on a View and releasing
- Long click a View
- Dragging the screen
- Dragging Views

Table 3.4 lists the methods of the TouchUtils class.

Methods	Description
`clickView(InstrumentationTestCase test, View v)`	The method is invoked to simulate touching at the center of a view and releasing it.
`Drag(InstrumentationTestCase test, float fromX, float toX, float fromY, float toY, int stepCount)`	The method is invoked to simulate touching at a specific location and dragging it to a new location.
`dragQuarterScreenDown(InstrumentationTestCase test, Activity activity)`	The method is invoked to simulate touching at the center of the screen and dragging it to one quarter of the way down.
`dragQuarterScreenUp(InstrumentationTestCase test, Activity activity)`	The method is invoked to simulate touching at the center of the screen and dragging it to one quarter of the way up.
`dragViewTo(InstrumentationTestCase test, View v, int gravity, int toX, int toY)`	The method is invoked to simulate touching a view to drag to a specified location such as dragViewToX or dragViewToY for viewing.
`dragViewToBottom (InstrumentationTestCase test, Activity activity, View v)`	The method is invoked to simulate touching the center of a view and drags it to the bottom of the screen.
`dragViewToTop(InstrumentationTestCase test, View v)`	The method is invoked to simulate touching the center of a view and drags it to the top of the screen.

Methods	Description
`longClickView(InstrumentationTestCase test, View v)`	The method is invoked to simulate touching the center of a view, holding for a long press, and then releasing.
`scrollToTop((InstrumentationTestCase test, Activity activity, ViewGroup v)`	The method is invoked to simulate scrolling of a ViewGroup to the top by repeatedly invoking dragQuarterScreenDown.
`tapView(InstrumentationTestCase test, View v)`	The method is invoked to simulate touching the center of a view and releasing quickly (before the tap timeout).
`touchAndCancelView (InstrumentationTestCase test, View v)`	The method is invoked to simulate touching the center of a view to cancel.

Table 3.4: Methods of TouchUtils Class

3.4 Using Mock Objects

Mock Objects, as the name suggests are duplicate objects that are called instead of the real domain objects to allow testing of units in isolation. They isolate the tests by stubbing out or overriding them. Some of the Mock Objects available in Android are as follows:

- Context objects
- ContentProvider objects
- ContentResolver objects
- Service objects
- Mock Intent objects

Mock Objects are used to:

- detach unit from dependencies.
- perform repetitive testing.
- monitor interactions.
- run tests independently.

For successfully using Mock Objects in tests, there is a need to provide dependencies to compile tests. The Android testing frameworks supports a number of the Mock Object classes in the android.test.mock and the android.test packages.

3.4.1 Simple Mock Object Classes

Simple Mock Object classes exhibit a simple and useful mock strategy. Their objective is to provide stubbed-out versions of the corresponding system object class. When implementing the methods, override only those methods that are to be used to provide mock dependencies. The methods throw an UnsupportedOperationException when invoked and need to be overwritten.

Table 3.5 lists the simple Mock Object classes.

Class	Description
MockApplication	The methods in MockApplication are non-functional and throw UnSupportedOperationException. This class contains methods such as onConfigurationChanged(), onCreate(), onTerminated().
MockContext	A mock Context class is used to force mocks or monitors into the classes that are under testing.
MockContentProvider	A Mock Object for ContentProvider.
MockCursor	MockCursor class is used to separate the test code from the implementation of real cursor.
MockDialogInterface	It is a mock DialogInterface class implementation.
MockPackageManager	A mock implementation of PackageManager class.
MockResources	A mock Resources class.

Table 3.5: Simple Mock Objects Class

3.4.2 Resolver Mock Objects/MockContentResolver Class

MockContentResolver class isolates the test code by stubbing the normal system resolver framework. MockContentResolver uses its own internal table to find a content provider for a given authority string. All methods in the MockContentResolver class have a non-functional approach and throw the UnsupportedOperationException when used. The reason for this is that the MockContentResolver isolates the test code from the real content system.

Providers can be added to the MockContentResolver using the method addProvider(String, ContentProvider). Using the addProvider(String, ContentProvider) method, it is possible to perform the following:

- Add a mock content provider with an authority.
- Make a representative of the real provider but use test data in it.
- Set a provider with an authority to null.

3.4.3 Android Context Class

There are two Context classes in Android namely, IsolatedContext and RenamingDelegatingContext. These classes are present in the android.test package and both are used for testing.

Table 3.6 shows the two Context class and their functions.

Class	Description	Function
`IsolatedContext`	Associates an isolated Context, file, directory, and database operations.	Uses the Context in a test area by creating a stub code to respond to system calls.
`RenamingDelegating Context`	Associates a Context where most functions are run by an existing Context except for file and database functions which is taken care by the IsolatedContext.	Uses test directory to name special files and directories. Name can be assigned automatically by the constructor or can be controlled by the tester.

Table 3.6: Context Class

3.5 Using the TestCase Base Class

TestCase base class is the base class for all test cases in the JUnit framework. It provides the basic methods in the JUnit framework. Figure 3.5 shows that the test cases extend either directly from TestCase or from one of the subclasses of TestCase.

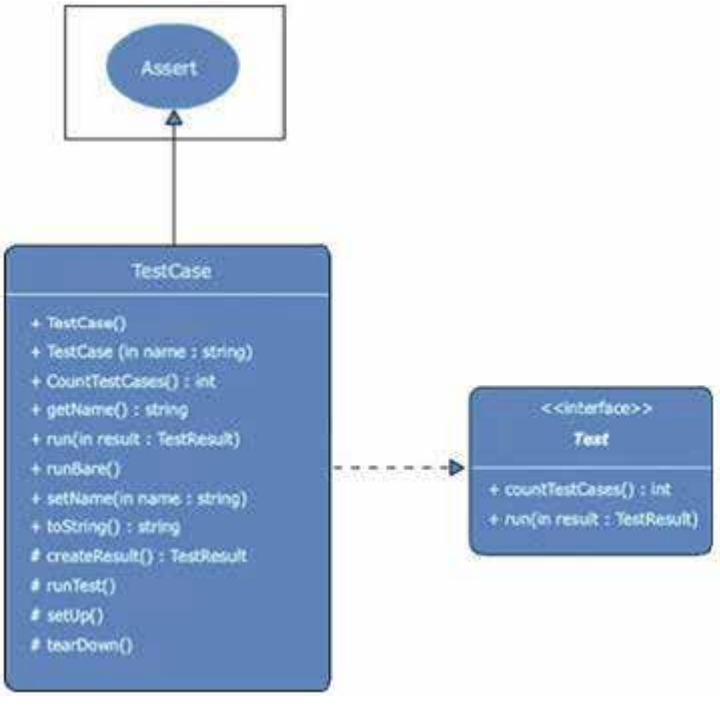


Figure 3.5: UML Diagram for TestCase Base Class

Sometimes, based on the test runner that is used, a constructor is invoked. The no argument constructor is the only default constructor that is required for all tests. The constructor is also used for serialization. Code Snippet 7 demonstrates the use of the default constructor.

Code Snippet 7:

```
public class AddNumberTestCase extends TestCase {
     public AddNumberTestCase() {
          this("AddNumberTestCase Default Name");
     }
     public AddNumberTestCase(String name) {
           super(name);
     }
}
```

The function of this constructor is to name itself as an argument to provide it to the other constructor. This constructor appears in test reports and is helpful to identify failed tests.

- **setName() method**

  In cases where some classes extending the test case do not provide a given name to the constructor the setName(String name) method is used.

3.6 Methods of AndroidTestCase

As discussed in Session 2, the AndroidTestCase is the base class for general purpose Android test cases. Some of the methods of this class are discussed here in detail.

- **assertActivityRequiresPermission()**

  The assertActivityRequiresPermission() is an assertion method that checks whether the launch of a particular Activity is permission protected.

  The syntax of this method is as follows:

  Syntax:

  ```
  public void assertActivityRequiresPermission (String packageName, String className, String permission)
  ```

  where,

  packageName - represents a String that accepts the package name of the Activity to launch.

  className - refers to a String variable that accepts the classname of the Activity to launch.

  permission - represents a string variable that contains the name of the permission to query.

This method affirms that a particular permission protects the start of a given activity. Such action takes places when an activity is started and validated by invoking the exception, SecurityException that displays the permission in its error message. Code Snippet 8 checks the need for the android.Manifest.permission. WRITE_EXTERNAL_STORAGE permission which is required to write to external storage in the Activity class named MyFirstProjectActivity.

Code Snippet 8:

```
…
public void testActivityPermission() {
  final String AP = "com.example.myfirstproject";
  final String ACTION = AP + ".MyFirstProjectActivity";
  final String CONSENT =
  android.Manifest.permission.WRITE_EXTERNAL_STORAGE;
  assertActivityRequiresPermission (AP, ACTION, CONSENT);
}
…
```

- **assertReadingContentUriRequiresPermission()**

This assertion method checks whether reading from a specific URI has the required permission as stated by the parameters.

The syntax of the method is:

Syntax:

```
public void assertReadingContentUriRequiresPermission (Uri uri,
String permission)
```

where,

uri - represents the URL where permission is required to query.

permission - represents a string containing the permission to query.

The assertion method is validated when a SecurityException is thrown containing the specified permission.

Code Snippet 9 demonstrates the use of this method.

Code Snippet 9:

```
...
public void testReadingContacts() {
  final Uri URI = ContactsContract.AUTHORITY_URI;
  final String CONSENT =
  android.Manifest.permission.READ_CONTACTS;
  assertReadingContentUriRequiresPermission(URI, CONSENT);
}
…
```

In the code, the test method reads the contacts and verifies whether the correct SecurityException is generated.

- **assertWritingContentUriRequiresPermission()**

This assertion methods checks whether inserting into a specific URI has the required permission as stated by the parameters. The syntax of this method is as follows:

Syntax:

```
public void assertWritingContentUriRequiresPermission (Uri uri, String permission)
```

where,

uri - represents the URI that requires the permission to query.

permission - represents a string variable with the permission to query.

The assertion method is validated when a SecurityException is thrown containing the specified permission. Code Snippet 10 demonstrates the use of the method.

Code Snippet 10:

```
…
public void testWritingContacts() {
  final Uri URI = ContactsContract.AUTHORITY_URI;
  final String CONSENT = android.Manifest.permission.WRITE_
CONTACTS;
assertWritingContentUriRequiresPermission(URI, CONSENT);
}
…
```

In the code, a test method is created that writes to contacts and verifies whether the correct SecurityException is generated.

3.7 Component-specific Test Cases

The component-specific test classes are responsible for testing specific components with methods for fixture setup and teardown and component lifecycle control. They are also required to provide methods for setting up of Mock Objects. The component-specific test classes are as follows:

- **Activity Testing**: This testing relies on the Android instrumentation framework.

  InstrumentationTestCase is the base class for activity testing. It supports the following functions:

  a. Instrumentation helps in controlling the lifecycle of an activity being tested. It can start, pause, and destroy the activity using methods of the test case classes.

  b. Instrumentation helps in controlling the test environment and separates it from the production systems by creating Mock Objects and using them to run the activity being tested. It is also possible to customize Intents and begin an activity using them.

  c. Instrumentation can be used for user interface interaction by sending keystrokes or touch events directly to the UI of the activity being tested.

  The two main subclasses of activity testing are as follows:

  - **ActivityInstrumentationTestCase2** - is used for functional testing of one or more Activities in an application. It runs the activities in a normal system infrastructure. It uses the standard system context and runs the activities in a normal instance of the application under test.
  - **ActivityUnitTestCase** - is used for testing a single activity in isolation. Use it to perform unit testing of methods that do not interact with Android.

  Along with these, following are the different test cases:

  - SingleLaunchActivityTestCase, used to test a single activity in an environment that do not change from test to test.
  - Mock Objects and activity testing, creates a hidden mock application object. This object is used as an application under test.
  - Assertions for activity testing, is used to verify the positions and alignments of view objects.

  When there is a need to test an activity that runs in non-standard mode, the SingleLaunchActivityTestCase is invoked. This class is a convenience class that is used for testing a single activity where the test environment need not be reset from test to test.

- **Content Provider Testing**: Content providers are an essential part of the Android API. The main focus of the provider testing API is to supply an isolated test environment. Their function is to store and retrieve data and make them available across applications.

  ProviderTestCase2 is the base test class for content providers. The primary function of this class is its initialization to create an isolated environment for testing. It uses the IsolatedContext and MockContentResolver, to complete its function.

  - **IsolatedContext**: This object allows file and database operations and these operations take place in directory, local to the device.
  - **MockContentResolver**: Used as a resolver for the test.

- **Service Testing**: Service objects run in isolation in a separate testing framework provided by Android.

  ServiceTestCase is the test case class for Service objects. It is a part of the JUnit TestCase class. It has methods to test application permissions and control the application and Service being tested. ServiceTestCase delays initialization until the ServiceTestCase.startService() or ServiceTestCase.bindService() is invoked. By doing this, it sets up the Mock Objects before Service is started.

  The setUp() method for ServiceTestCase is called before each test to set up the test fixture. It makes a copy of the current system Context before being inspected by any test methods. The methods setApplication() and setContext(Context) methods are used to set a mock Context or mock Application (or both) for the Service, to isolate the test from the rest of the system.

3.8 Function of the ApplicationTestCase

The ApplicationTestCase tests the setup and teardown of applications. The main function of this class of objects is to store the global information. All units in an application package use the stored information. Also, it verifies whether the <application> element in the manifest file is correctly set up.

3.9 The InstrumentationTestCase Class

The system initiates instrumentation before running any application code. Instrumentation examines the interaction between the system and the application. The interaction is specified in the AndroidManifest.xml file under the <instrumentation> tag.

Code Snippet 11 demonstrates the use of AndroidManifest.xml file in a test.

Code Snippet 11:

```
<instrumentation android:targetPackage="com.example.helloandroidproject"
android:name="android.test.InstrumentationTestRunner"
android:label="HelloAndroidProjectTests"/>
```

The InstrumentationTestCase class may be a direct or indirect base class for the different test cases that have an approach to Instrumentation. This class is present in the android.test package. This class extends from junit.framework.TestCase class which in turn extends from junit.framework.Assert class. Figure 3.6 displays the UML class diagram of the InstrumentationTestCase class.

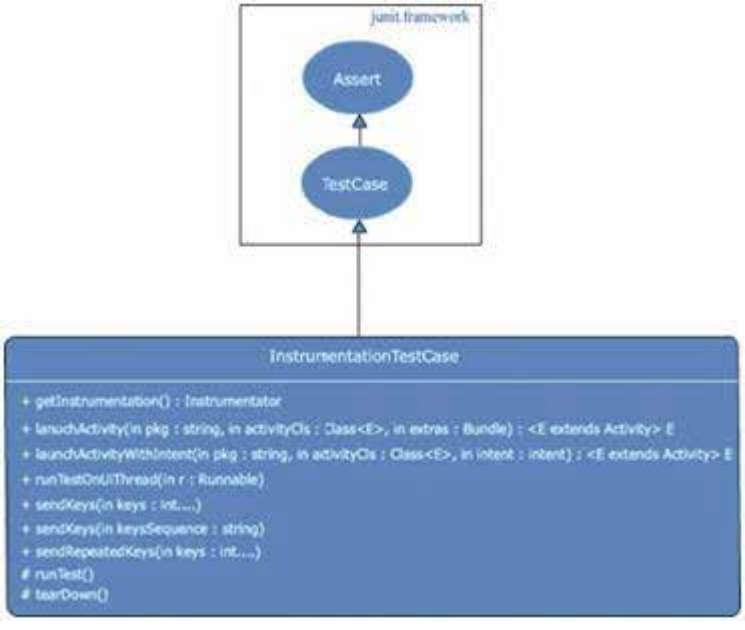


Figure 3.6: UML Class Diagram of InstrumentationTestCase Class

Related Classes

Some of the most important direct and indirect subclasses that have access to Instrumentation are as follows:

- ActivityTestCase
- ProviderTestCase2<E extends ContentProvider>
- SingleLaunchActivityTestCase<E extends Activity>
- SyncBaseInstrumentation
- ActivityInstrumentationTestCase2<E extends Activity>
- ActivityUnitTestCase<E extends Activity>

3.9.1 Methods of InstrumentationTestCase Class

Some of the methods of the class are as follows:

- **launchActivity**

  This is an utility method that is used to launch an Activity. The syntax of this method, when an intent is not specified is as follows:

  Syntax:

  ```
  public final T launchActivity( String pkg, Class<E> activityCls,
  Bundle extras)
  ```

  where,

  pkg – refers to the package name hosting the activity that is to be launched which is mentioned in the AndroidManifest.xml file.

  activityCls – refers to the activity class name to be launched.

  extra – refers to the optional stuff to pass to the activity.

- **launchActivityWithIntent**

  This method will launch an Activity with a custom Intent. The syntax of this method, is as follows:

  Syntax:

  ```
  public final T launchActivityWithIntent( String pkg, Class<E>
  activityCls, Intent intent)
  ```

  where,

  pkg – refers to the package name hosting the activity that is to be launched which is mentioned in the AndroidManifest.xml file.

  activityCls – refers to the activity class name to be launched.

  intent – refers to the intent to launch with.

- **sendKeys and sendRepeatedKeys**

  The sendKeys and sendRepeatedKeys methods are both utility methods that are used to interact with qwerty-based keyboards or DPAD buttons while testing the UI of the Activity class. They are used to:

  - send keys.
  - complete fields.
  - select shortcuts or.
  - navigate between different components.

Consider an example of simple Calculator where a developer uses sendKeys and sendRepeatedKeys.

To begin with, create an Android project named Calculator and test it through Android Test Project. To create an Android project, perform the following steps:

1. Create an Android Project named Calculator.
2. Specify the project details as shown in figure 3.7.

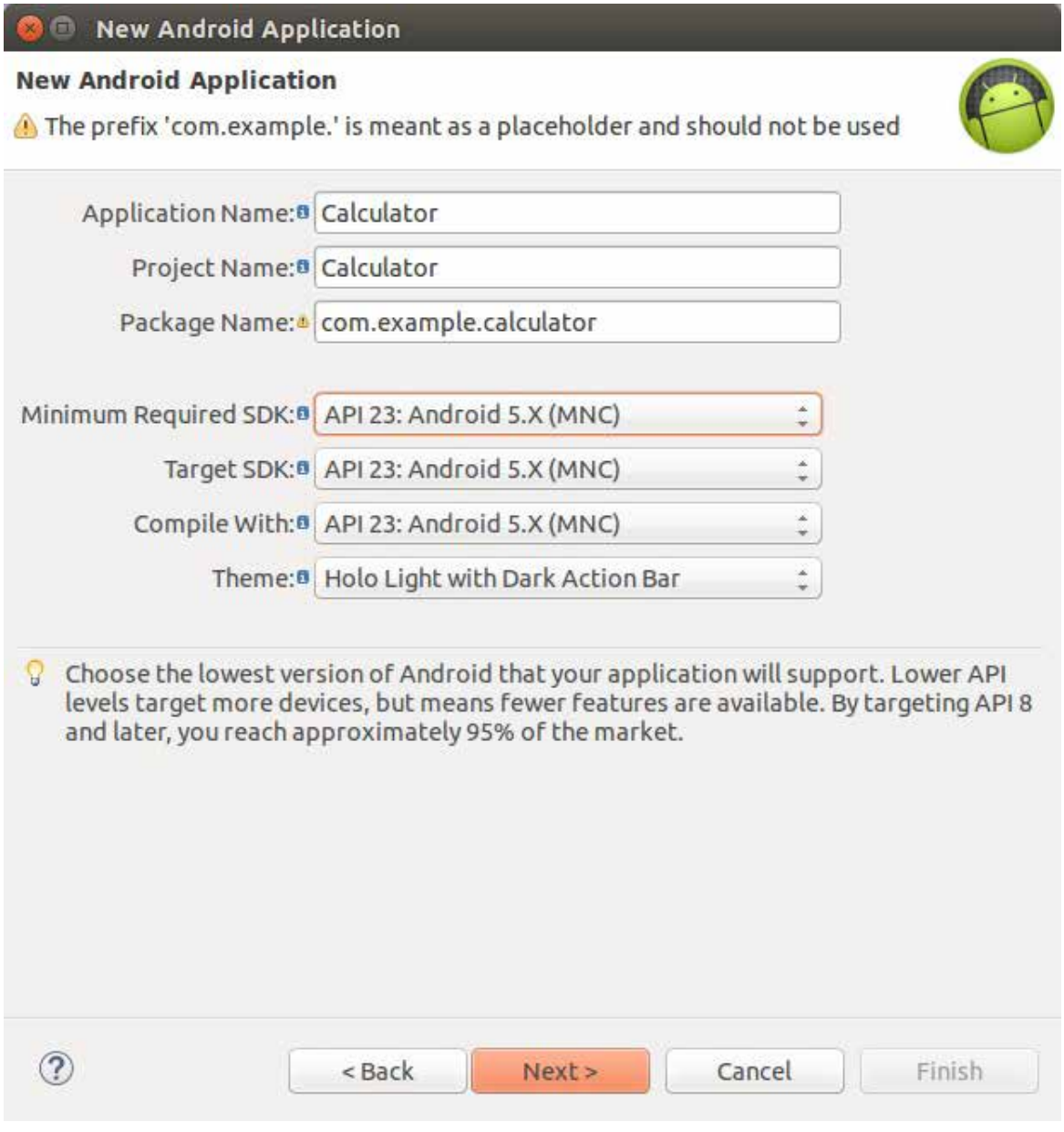


Figure 3.7: New Android Application

3. Click Next till you reach the Blank Activity screen.

4. Enter the Activity Name and Layout name as shown in figure 3.8 and click Finish.

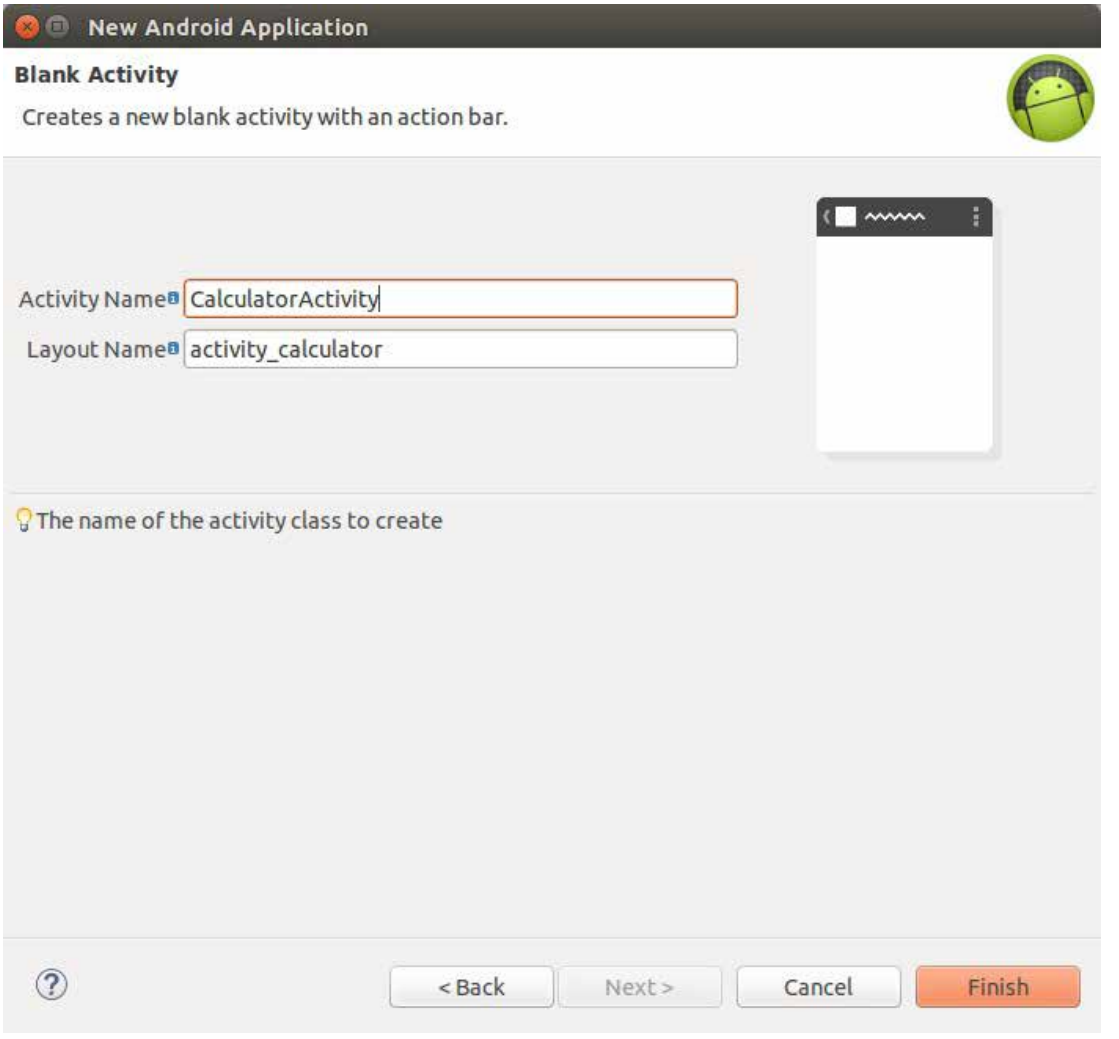


Figure 3.8: Activity Settings

5. Create a class named Calculator within the package, com.example.calculator.
6. Modify the code in the Calculator class as demonstrated in Code Snippet 12.

Code Snippet 12:

```
package com.example.calculator;
public class Calculator {
	/**
	 * Left operand of the operation to be performed.
	 */
	private double leftOperand;
	/**
	 * Right operand of the operation to be performed.
	 */
	private double rightOperand;
	/**
	 * Constructs a Calculator object by initializing the
leftOperand and
 * rightOperand with the given values.
	 *
	 * @param leftOperand
	 *            Left operand.
	 * @param rightOperand
	 *            Right operand.
	 */
		public Calculator(double leftOperand, double
rightOperand) {
		this.leftOperand = leftOperand;
		this.rightOperand = rightOperand;
	}
	/**
	 * Adds the leftOperand to the rightOperand.
	 *
```

```
        * @return The sum of the two operands.
        */
      public double add() {
            return leftOperand + rightOperand;
      }
      /**
        * Subtracts the rightOperand from the leftOperand.
        *
        * @return The subtraction of the two operands.
        */
      public double subtract() {
            return leftOperand - rightOperand;
      }
      /**
* Multiply the leftOperand to the rightOperand.
        *
        * @return The multiplication of the two operands.
        */
      public double multiply() {
            return leftOperand * rightOperand;
      }

      /**
      * Divides the leftOperand to the rightOperand.
      *
      * @return The division of the two operands.
      */
      public double divide() {
            if (rightOperand == 0) {
```

```
        throw new ArithmeticException("Right operand should not be
zero!");
        }
        return leftOperand / rightOperand;
        }
}
```

7. Modify the code in activity _ calculator.xml file as demonstrated in Code Snippet 13.

Code Snippet 13:

```
<LinearLayout     xmlns:android="http://schemas.android.com/apk/
res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/linear"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
android:orientation="vertical"
    tools:context="com.example.calculator.CalculatorActivity" >
   <TableLayout
        android:id="@+id/table"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
       android:orientation="vertical"
        android:stretchColumns="1" >
        <TableRow>
            <TextView
    android:id="@+id/leftOperand_label"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:padding="3dip"
                android:text="Left Operand"
```

```
                                    android:textStyle="bold" >
            </TextView>
            <EditText
                android:id="@+id/leftOperand"
                android:nextFocusDown="@+id/rightOperand"
                android:numeric="decimal"
                android:padding="3dip"
                android:scrollHorizontally="true"
                android:singleLine="true" >
            </EditText>
        </TableRow>
        <TableRow>
             <TextView
                android:id="@+id/rightOperand_label"

               android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:padding="3dip"
                android:text="Right Operand"
                android:textStyle="bold" >
            </TextView>
            <EditText
                android:id="@+id/rightOperand"
                android:nextFocusDown="@+id/plus"
                android:padding="3dip"
                android:scrollHorizontally="true"
                android:singleLine="true" >
      </EditText>
        </TableRow>
```

```
    </TableLayout>
      <LinearLayout
       android:id="@+id/buttons_linear"
       android:layout_width="wrap_content"
       android:layout_height="wrap_content"
       android:layout_below="@+id/table"
      android:layout_marginTop="20dp"
       android:orientation="horizontal" >
       <Button
            android:id="@+id/plus"
            android:layout_width="46dp"
            android:layout_height="wrap_content"
            android:layout_gravity="center_horizontal"
            android:layout_weight="0.04"
            android:nextFocusRight="@+id/minus"
          android:padding="6dip"
            android:text="+" />
       <Button
            android:id="@+id/minus"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_gravity="center_horizontal"
            android:nextFocusRight="@+id/multiply"
            android:padding="6dip"
            android:text="-" >
       </Button>
       <Button
       android:id="@+id/multiply"
            android:layout_width="0px"
```

```
                    android:layout_height="wrap_content"
            android:layout_gravity="center_horizontal"
            android:layout_weight="0.46"
            android:nextFocusRight="@+id/divide"
            android:padding="6dip"
            android:text="*" >
        </Button>
        <Button
            android:id="@+id/divide"
            android:layout_width="0px"
            android:layout_height="wrap_content"
            android:layout_gravity="center_horizontal"
            android:layout_weight="0.54"
            android:nextFocusRight="@+id/divide"
            android:padding="6dip"
    android:text="/" >
        </Button>
        <Button
            android:id="@+id/clear"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_gravity="center_horizontal"
            android:nextFocusRight="@+id/divide"
            android:padding="6dip"
            android:text="C" >
        </Button>
    </LinearLayout>
<TextView
        android:id="@+id/result"
```

```
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:layout_alignLeft="@+id/buttons_linear"
        android:layout_below="@+id/buttons_linear"
        android:layout_marginTop="22dp"
        android:gravity="right"
        android:padding="20dip"
        android:text="Result"
        android:textStyle="bold" />
</LinearLayout>
```

8. Modify the code in CalculatorActivity class as demonstrated in Code Snippet 14.

Code Snippet 14:

```
package com.example.calculator;
import android.app.Activity;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
public class CalculatorActivity extends Activity implements
OnClickListener{
      private EditText leftOperand;
      private EditText rightOperand;
      private TextView result;
      private Calculator calculator;
```

```
	@Override
	protected void onCreate(Bundle savedInstanceState) {
		super.onCreate(savedInstanceState);
		setContentView(R.layout.activity_calculator);
		leftOperand = (EditText) findViewById(R.
id.leftOperand);
		rightOperand = (EditText) findViewById(R.
id.rightOperand);
		result = (TextView) findViewById(R.id.result);
		Button plus = (Button) findViewById(R.id.plus);
		plus.setOnClickListener(this);
		Button minus = (Button) findViewById(R.id.minus);
		minus.setOnClickListener(this);
		Button multiply = (Button) findViewById(R.
id.multiply);
		multiply.setOnClickListener(this);
		Button divide = (Button) findViewById(R.
id.divide);
		divide.setOnClickListener(this);
		Button clear = (Button) findViewById(R.id.clear);
		clear.setOnClickListener(this);
	}
	@Override
	public boolean onCreateOptionsMenu(Menu menu) {
		// Inflate the menu; this adds items to the
action bar if it is present.
		getMenuInflater().inflate(R.menu.calculator,
menu);
		return true;
	}
```

```
        @Override
        public void onClick(View view) {
       double leftOp = Double.parseDouble(leftOperand.
       getText().toString());
       double rightOp = Double.parseDouble(rightOperand.
       getText().toString());
       calculator = new Calculator(leftOp, rightOp);
       if (view.getId() == R.id.plus) {
       result.setText("" + calculator.add());
       } else if (view.getId() == R.id.minus) {
       result.setText("" + calculator.subtract());
       } else if (view.getId() == R.id.multiply) {

result.setText("" + calculator.multiply());
       } else if (view.getId() == R.id.divide) {
       result.setText("" + calculator.divide());
       } else if (view.getId() == R.id.clear) {
             leftOperand.setText("");
             rightOperand.setText("");
             result.setText("");
             leftOperand.requestFocus();
             }
             }
       @Override
       public boolean onOptionsItemSelected(MenuItem item)
{
             // Handle action bar item clicks here. The
action bar will
                  // automatically handle clicks on the
Home/Up button, so long
```

```
            // as you specify a parent activity in
AndroidManifest.xml.
            int id = item.getItemId();
            if (id == R.id.action_settings) {
                  return true;
            }
            return super.onOptionsItemSelected(item);
      }
}
```

9. Clean the project to rebuild and execute it: right-click Calculator and select Run As → Android Application. Figure 3.9 displays the output of the application.



Figure 3.9: Calculator Application

Figure 3.10 displays the output after the values are entered and the button '+' is clicked.



Figure 3.10: Addition Output

To create the Android Test Project for Calculator application, perform the following steps:

1. Create an Android Test Project named CalculatorTest.
2. Navigate to com.example.calculator.test package.
3. Create a test case class named CalculatorActivityTest within the package.
4. Modify the code in the CalculatorActivityTest class as demonstrated in Code Snippet 15.

Code Snippet 15:

```
package com.example.calculator.test;

import com.example.calculator.CalculatorActivity;

import junit.framework.TestCase;
```

```
import android.test.ActivityInstrumentationTestCase2;
import android.test.TouchUtils;
import android.test.suitebuilder.annotation.SmallTest;
import android.view.KeyEvent;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;


public class CalculatorActivityTest extends
ActivityInstrumentationTestCase2<CalculatorActivity> {
      private CalculatorActivity calculatorInstance;
      private TextView result;
      Button plus;
      EditText ed1, ed2;
      public CalculatorActivityTest() {
            super("com.example.calculator", CalculatorActivity.
class);
      }


      protected void setUp() throws Exception {
            super.setUp();
            calculatorInstance      =      (CalculatorActivity)
getActivity();
            result     =     (TextView)     calculatorInstance.
findViewById(com.example.calculator.R.id.result);
            plus = (Button) calculatorInstance.findViewById(com.
example.calculator.R.id.plus);
            ed1 = (EditText) calculatorInstance.findViewById(com.
example.calculator.R.id.leftOperand);
            ed2 = (EditText) calculatorInstance.findViewById(com.
example.calculator.R.id.rightOperand);
```

```
        }
        /**
         * Test the addition operation of the CalculatorActivity
         *
         * @throws Throwable
         */
        public void testAdd() throws Throwable {
                // First field value
                sendKeys(KeyEvent.KEYCODE_3);
                sendKeys(KeyEvent.KEYCODE_PERIOD);
                sendKeys(KeyEvent.KEYCODE_5);
                // Move to the second field
                sendKeys(KeyEvent.KEYCODE_DPAD_DOWN);
                sendKeys(KeyEvent.KEYCODE_2);
                sendKeys(KeyEvent.KEYCODE_PERIOD);
                sendKeys(KeyEvent.KEYCODE_1);
                // Move to the '+' button
                sendKeys(KeyEvent.KEYCODE_DPAD_DOWN);
                sendKeys(KeyEvent.KEYCODE_DPAD_CENTER);
                // Wait for the activity to finish all of its processing.
                getInstrumentation().waitForIdleSync();
                // Use assertion to make sure the value is correct
                assertTrue(result.getText().toString().
equals("5.6"));
        }
        /** Testing Add Operation */
        @SmallTest
        public void testAddition() {
                String strNum1 = "1 0 0 PERIOD 5 ";
```

```
            String strNum2 = "5 0";
            float expected = 150.5f;
            /** Inputting Operand1 */
            TouchUtils.tapView(this, ed1);
            sendKeys(strNum1);
            /** Inputting Operand2 */
            TouchUtils.tapView(this, ed2);
            sendKeys(strNum2);
            /** Selects Add operator from the button Widget */
            TouchUtils.tapView(this, plus);
            this.sendRepeatedKeys(1,     KeyEvent.KEYCODE_DPAD_
CENTER, 1, KeyEvent.KEYCODE_DPAD_DOWN, 1,
                        KeyEvent.KEYCODE_DPAD_CENTER);
            /** Performs calculation */
            try {
                  runTestOnUiThread(new Runnable() {
                        @Override
                        public void run() {
                              plus.performClick();
                        }
                  });
            } catch (Throwable e1) {
                  e1.printStackTrace();
            }
            /** Getting the result shown */
            float  actual  =  Float.parseFloat(result.getText().
toString());
            /** Asserts, expected and actual are same */
            assertEquals(expected, actual, 0);
      }
```

```
        protected void tearDown() throws Exception {
                super.tearDown();
        }
}
```

In the code, a method named testAdd() has been created to test the addition of the numbers in the emulator. Similarly, you can create other methods, such as testSubstract(), testMultiply(), and testDivide() for testing the subtraction, multiplication, and division functionality of the Calculator class.

The method named testAddition() has been created to use sendKeys() and sendRepeatedKeys() methods.

5. Select CalculatorTest and click Run → Run Configuration to display the Run Configurations dialog box.
6. Right-click Android JUnit Test in the left pane and select New.
7. Type CalculatorTest in the Name box.
8. Select 'Run a single test'.
9. Type CalculatorTest in the Project box.
10. Click Search and select Test class as CalculatorActivityTest to display com.example.calculator.test.CalculatorActivityTest in the Test class box.
11. Select android.test.InstrumentationTestRunner from the Instrumentation runner list as shown in figure 3.11.

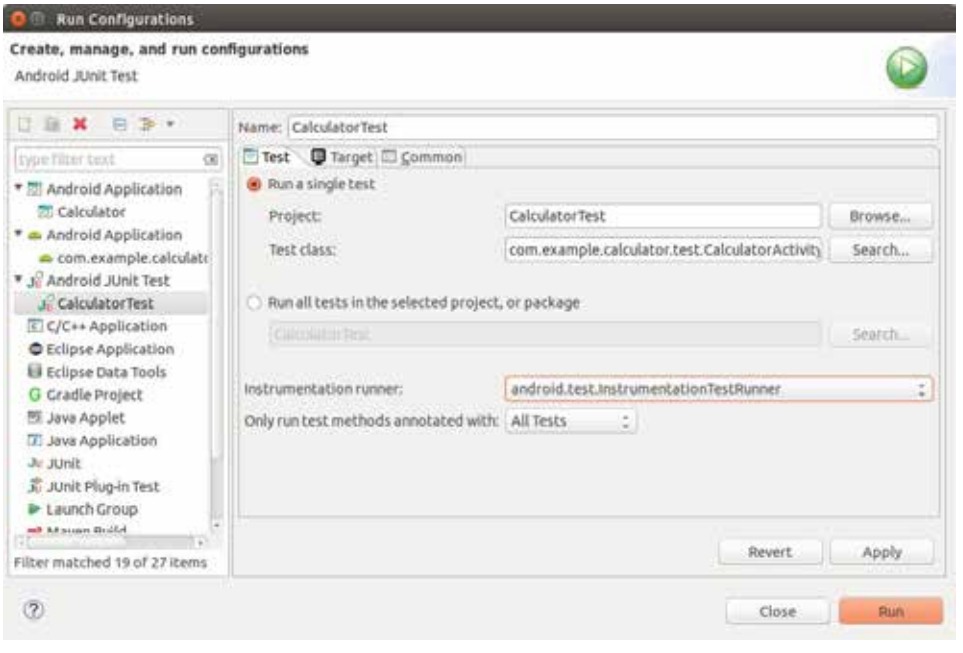


Figure 3.11: Run Configuration Settings

12. Click Run.

Figure 3.12 displays the output after execution of the test class.



Figure 3.12: CalculatorTest Class – Output

The test case methods automatically add values to the textboxes in the application and simulate the click of the add button '+'. The result is then compared with the expected value. In both the tests, the expected and actual values match. So the test shows successful execution for both test methods as shown in figure 3.13.

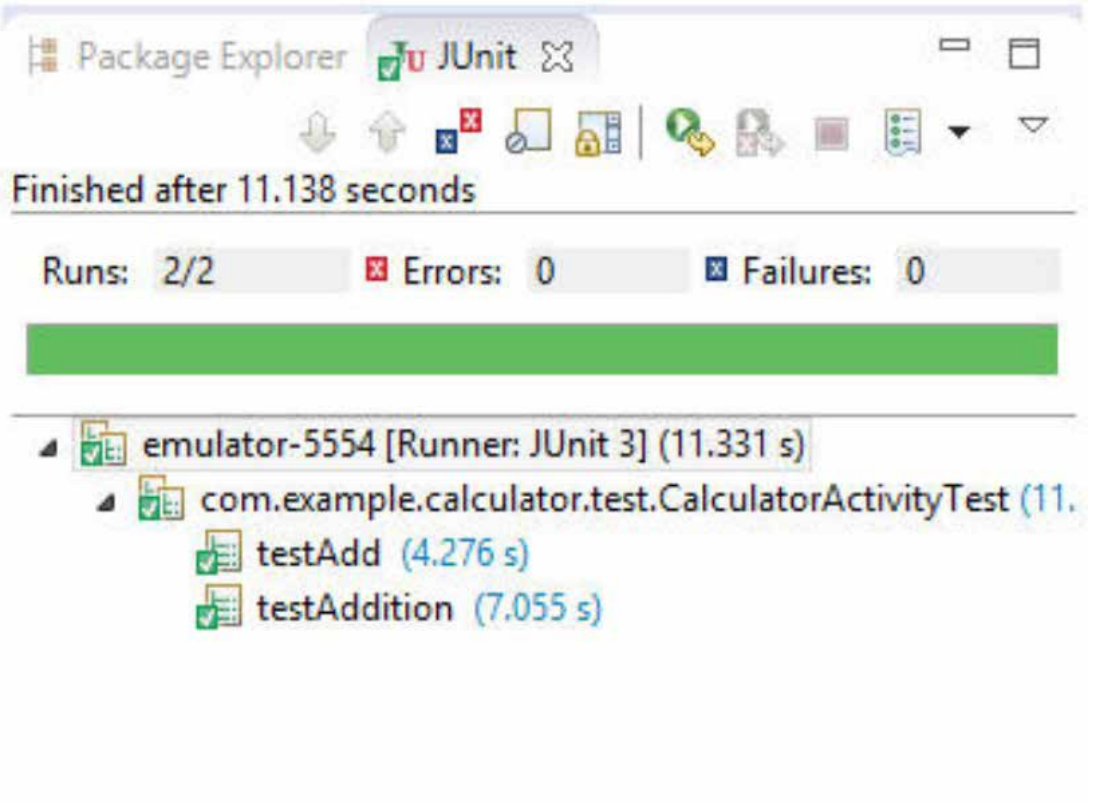


Figure 3.13: Test Execution Result

- **runTestOnUiThread helper method**

  This is a helper method that is used for running portions of a test on the UI thread. The syntax of the method is as follows:

  Syntax:

  ```
  public void runTestOnUiThread (Runnable r)
  ```

  where,

  r – represents the runnable containing the test code in the run() method.

Check Your Progress

1. Match the following keywords with their respective description.

Keyword		Description	
a.	Assert Class	1.	Tests the setup and teardown of applications.
b.	Mock Objects	2.	Direct or indirect base class for the different test cases.
c.	TouchUtils	3.	Base class for all test cases.
d.	ApplicationTestCase	4.	Extends from the InstrumentationTestCase.
e.	InstrumentationTestCase	4.	Duplicate real domain objects.

(A)	a-2, b-3, c-4, d-5, e-1	(B)	a-4, b-5, c-1, d-2, e-3
(C)	a-3, b-5, c-4, d-1, e-2	(D)	a-5, b-1, c-2, d-3, e-4

2. Which of the following class is the base class for all test cases in the JUnit framework?

(A)	TestCase class	(B)	ApplicationTestCase class
(C)	Assert class	(D)	JUnit tests

3. Which of the following assertion methods checks the required permission for the reading from a specific URI?

(A)	assertReadingContentUri RequiresPermission() method	(B)	assertActivityRequires Permission() Method
(C)	assertWritingContentUri RequiresPermission() method	(D)	CodeAccessPermission. Assert method

4. Which of the following options states the characteristics of Mock Objects?

(A)	Support different parameter types	(B)	Set a provider with an authority to null
(C)	Enables tapping on a View and releasing	(D)	To detach unit from dependencies

Check Your Progress

5. _____ class has advanced methods that are used for testing user interfaces.

(A)	ViewAsserts	(B)	MoreAssert
(C)	TouchUtils	(D)	Context

Answers

1.	C
2.	A
3.	A
4.	D
5.	A

Summary

- The Assert class is the base class for all of the test case classes. It is a part of the JUnit API and consists of several assertion methods that are used for writing tests.
- The android.test.ViewAsserts test the views on screen and their absolute and relative positions on the screen. MoreAsserts are responsible for regular expression matching.
- TouchUtils class stimulates different kinds of touch events in the UI. It extends from the InstrumentationTestCase.
- Mock Objects are duplicate objects that are called instead of the real domain objects to allow testing of units in isolation. Simple Mock Object classes stub out the corresponding system object class and override methods to provide mock dependencies. MockContentResolver isolates the test code.
- IsolatedContext and RenamingDelegatingContext classes are used for testing. IsolatedContext prevents user from talking to the rest of the devices. RenamingDelegatingContext allows replacing of the context used when calling file system methods.
- TestCase base class is the base class for all test cases in the JUnit framework. They provide the basic methods in the JUnit framework. AndroidTest case is the base class for general purpose Android test cases.
- Component-specific classes are responsible for testing specific components with methods for fixture setup and teardown and component lifecycle control. They are also needed to provide methods to set Mock Objects.
- ApplicationTestCase tests the setup and teardown of applications.
- The InstrumentationTestCase class may be a direct or indirect base class for the different test cases that have an approach to Instrumentation.

Onlinevarsity



LEARN @ YOUR COMFORT

04 Android Testing Environment



Welcome to the Session, **Android Testing Environment**.

This session explains the Android testing environment. It begins with creation of the Android Virtual Device (AVD) that will pave the way for different conditions and configurations required to run tests with the available options. Finally, it describes a test-driven application that will simulate generated events and use it for testing.

In this Session, you will learn to:

- Explain the process to create an AVD with AVD Manager
- Explain the process to run the AVD from command line
- Explain how to simulate network conditions
- Explain a test-driven application

Session

04

Android Testing Environment

4.1 Introduction

An essential part of creating Android applications is to run and test them. It is important to test the code and application on real devices to assess the drawbacks and rectify them. Learning to create an AVD, simulate network conditions, and develop a test-driven application are some of the fundamental steps to build an Android testing environment.

4.2 Android Virtual Device (AVD)

An Android Virtual Device (AVD) is an emulator configuration. It is made up of the following components:

- Hardware that defines whether the device has a camera or the storage memory, use of a dialing pad or a physical QWERTY keyboard, and so on.
- A mapping to the system image that provides details about what Android version to run the virtual device. It also provides a choice of the version of the standard Android platform or the system image with the SDK add-on.
- It provides options to choose and specify the emulator skin, emulator SD card, screen dimensions, appearance, and so on.
- A dedicated storage area that stores the device's user data and emulated SD card.

The main objective of the AVD is to design a prototype of an actual device by specifying the hardware and software options to be emulated by the Android emulator.

4.2.1 Create AVD with AVD Manager

Using the AVD Manager is one of the simplest ways of creating an AVD. The AVD Manager has options to edit, delete, repair, and view details of each AVD.

It is possible to create many AVDs depending on the need and types of device that are to be modeled. However, in order to thoroughly test the application, it is advisable to test the application on all API levels higher than the target API level of the application.

To create an AVD, perform the following steps:

1. Click Window → Android Virtual Device Manager to launch the AVD Manager from Eclipse as shown in figure 4.1.

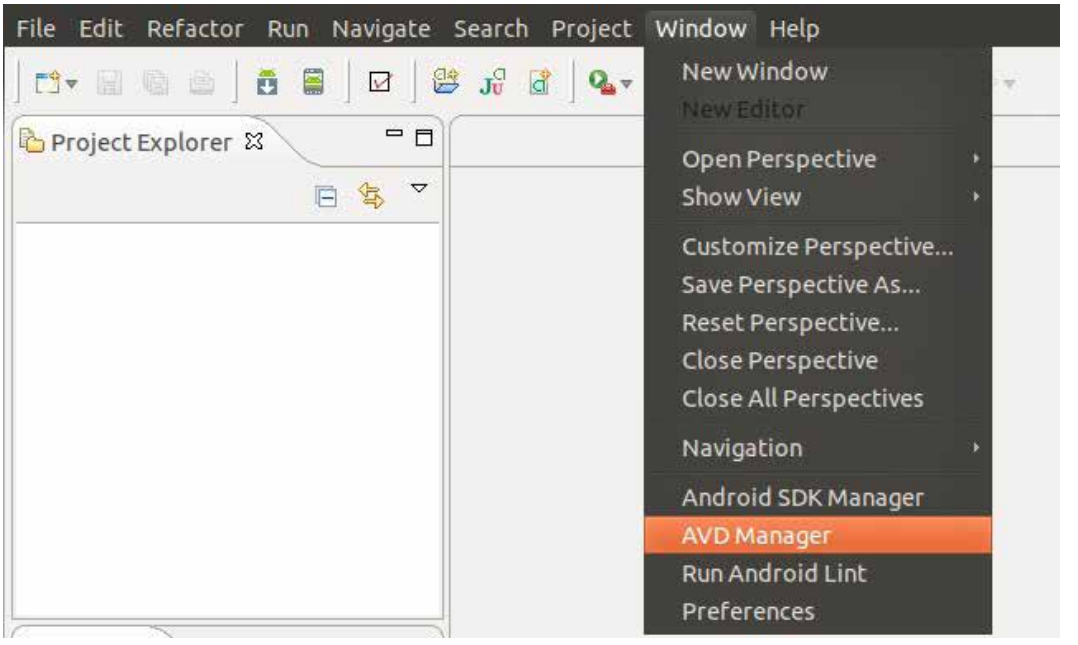


Figure 4.1: Launch AVD Manager from Eclipse

Alternately, click AVD Manager icon in the Eclipse toolbar as shown in figure 4.2.

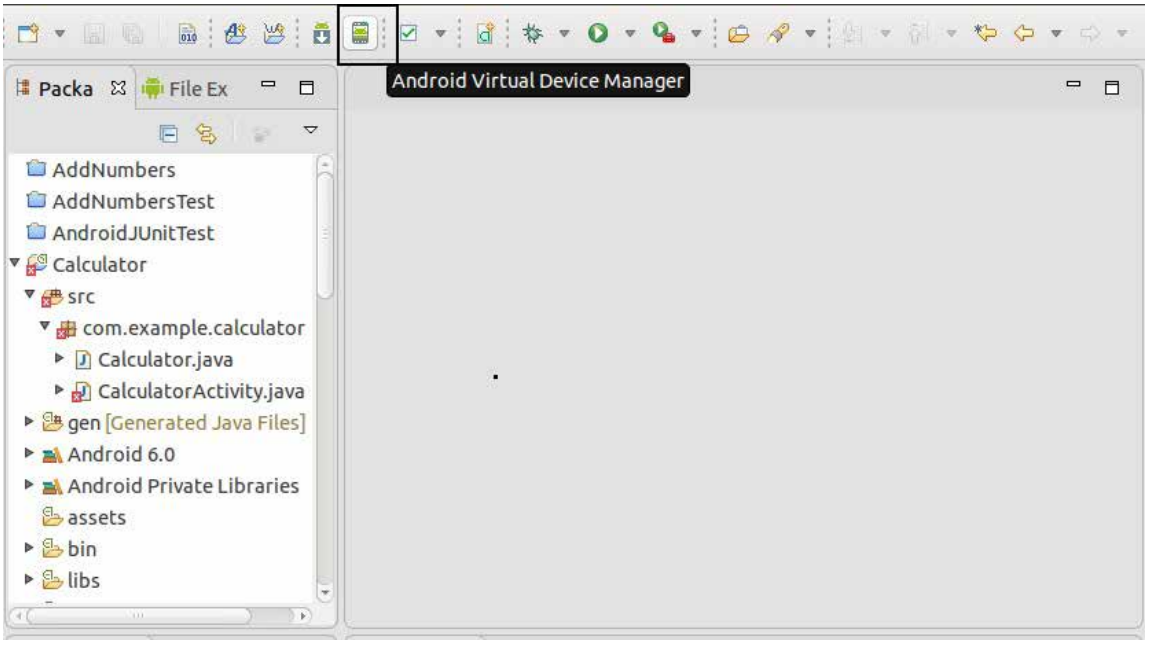


Figure 4.2: Launch AVD Manager Icon from the Eclipse Toolbar

Note - To start AVD Manager in other IDEs, navigate to SDK's tools/ directory and execute the android tool with no arguments.

To start the AVD Manager from the command line, go to the sdk/tools/ directory and invoke the android tool with avd options. Browse to this directory, press Shift, and right-click inside the folder. Select 'Open command window here' from the context menu. Type './android' and press the Enter key. The AVD Manager dialog box will open.

To get help on different avd options, ./android –h and press the Enter key. The list of options with description will be displayed as shown in figure 4.3.

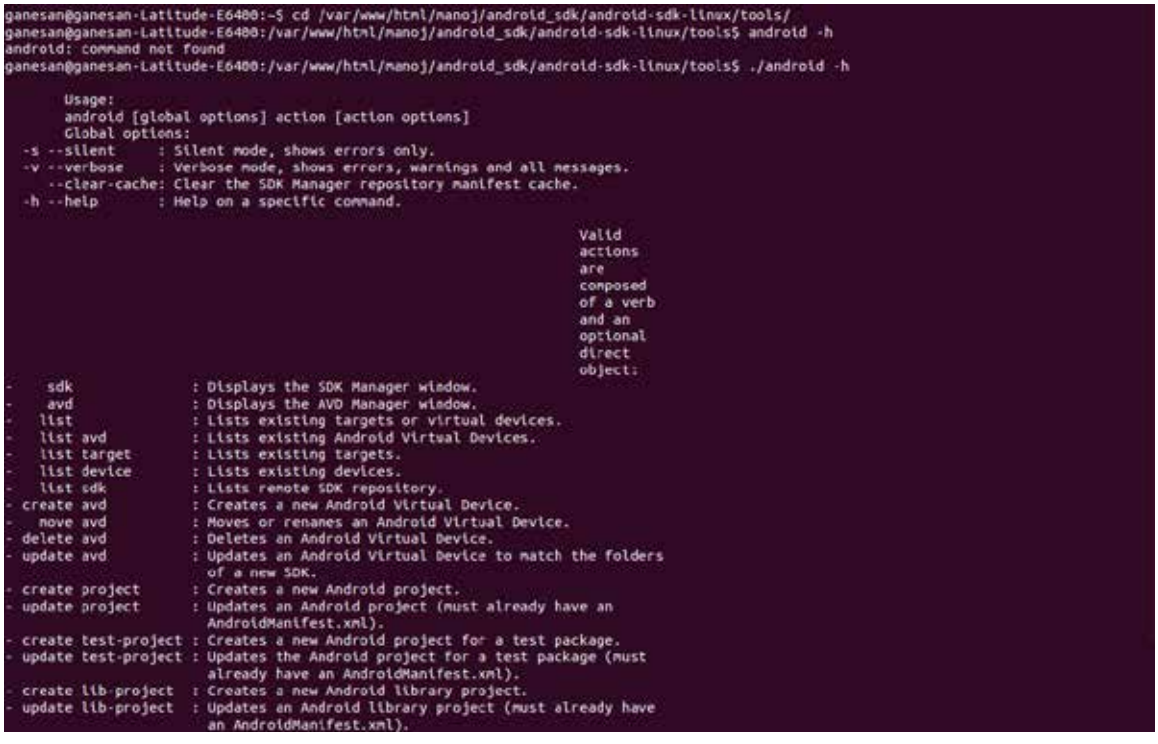


Figure 4.3: Using android Tool from Command Line

Virtual Devices panel of the AVD Manager lists the existing AVDs.

2. To create a new AVD, click Create as shown in figure 4.4.

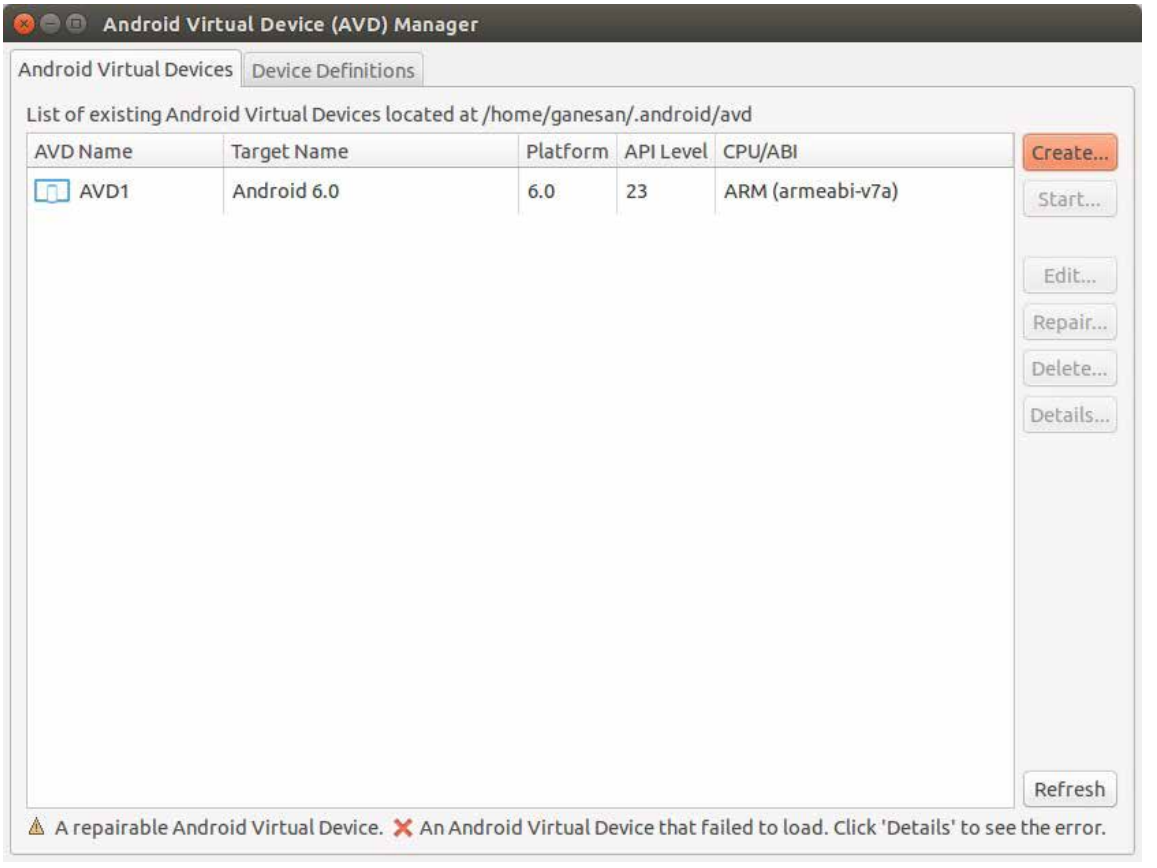


Figure 4.4: Virtual Devices Panel of the AVD Manager

The Create new Android Virtual Device (AVD) dialog box is displayed.

3. Enter details for AVD such as name, platform target, SD card size, and a skin as shown in figure 4.5.

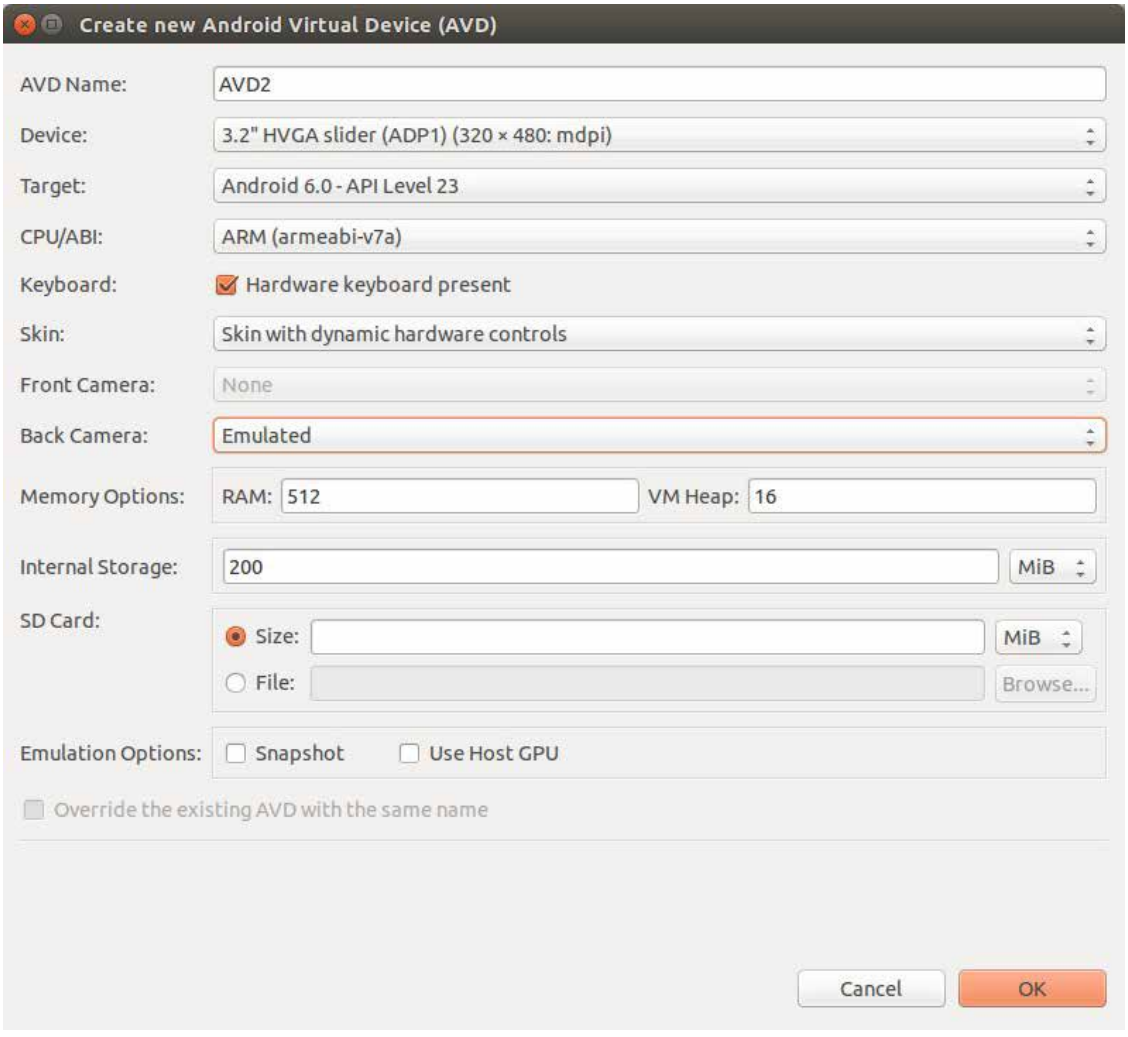


Figure 4.5: Enter Details for AVD

Give a descriptive name and use 200 MB RAM for basic testing AVDs. Leave the rest of the options as default. Remember to define a target for the new AVD that meets the application's Build Target. The AVD platform target should be equal or greater than API Level of the application.

4. Click OK to create a new AVD. The Virtual Devices panel shows the newly created AVD as shown in figure 4.6. It is possible to create more AVDs using this method.

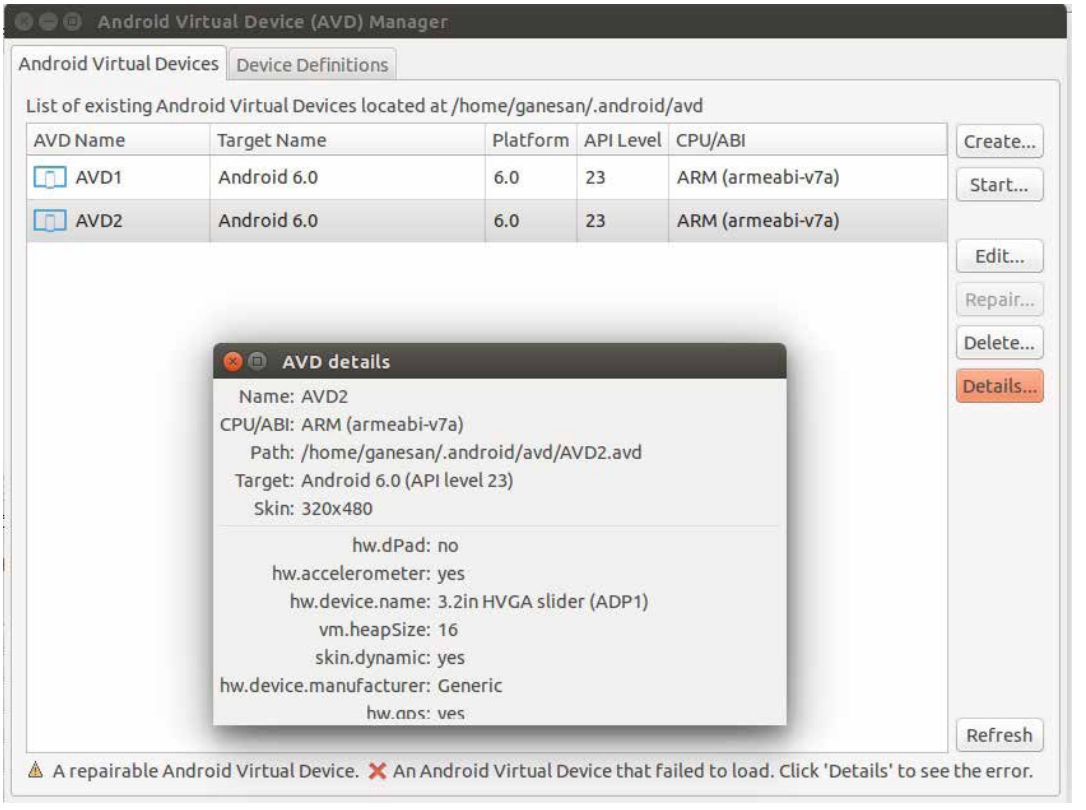


Figure 4.6: Newly Created AVD

5. Close the AVD Manager, or continue to create more AVDs, or launch an emulator. Follow steps 1 to 4 to create more AVDs.

6. To launch an emulator with the AVD, select the AVD and click Start.

7. It will launch the AVD by displaying the Launch Options dialog box as shown in figure 4.7. Your AVD is now ready and you can close the AVD Manager, create more AVDs, or launch an emulator with the AVD by selecting a device.

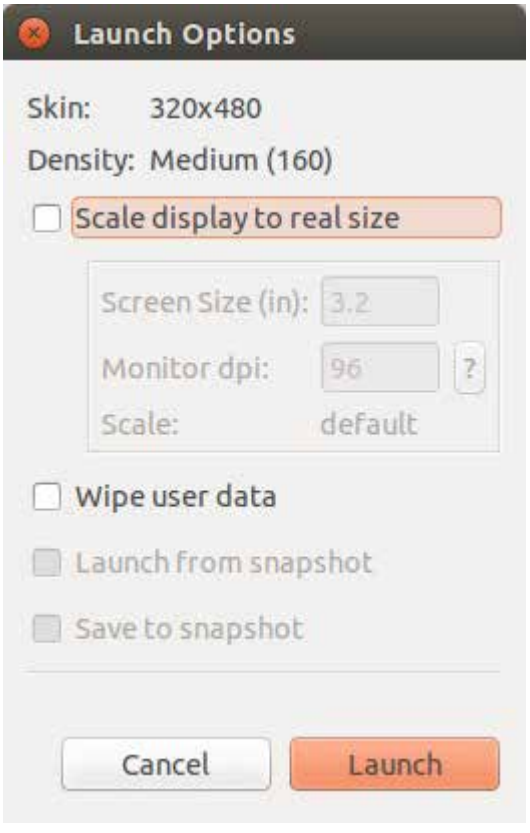


Figure 4.7: Launch Options Dialog Box

The Launch Options dialog box shows the skin type and density along with other launch options such as:

- **Scale display to real size**: Refers to the resolution of the emulator's display scaled to the actual screen size of the physical Android device that is emulated. The scale display screen size is provided in terms of inches and dots per inch.
- **Wiper user data**: Results in deletion of the AVD's user-data partition. The emulated device tends to store apps and state data across AVD when it restarts in a user-data partition.
- **Launch from snapshot**: Refers to the emulated device being started from a previously saved snapshot. This checkbox is checked by default when an AVD is created with snapshot enabled. Snapshot refers to a stored file of emulator state.
- **Save to snapshot**: Refers to saving a snapshot of the state of the emulated device on exit from the device. This checkbox is checked by default when an AVD is created. Launching an emulator is fast when it is launched from a snapshot.

8. Click Launch to see the emulator window as shown in figure 4.8.



Figure 4.8: Emulator Window

The emulator window has a device screen on the left and the phone controls on the right. The title bar of the emulator window shows 5554:AVD2. Value 5554 refers to the value of the console port that can be used to query and control the AVD's environment.

The Android logo, as shown in figure 4.9, is displayed few minutes later after the initial Android screen and is shown while the Android platform associated with the AVD is initializing.



Figure 4.9: Graphical Android Logo

If the ANDROID logo is displayed for more than 15–30 minutes, then there may be an error. Restart the IDE, start AVD Manager, delete the previous AVD, create a new one, and relaunch the new AVD. After some time, the lock screen is displayed as shown in figure 4.10.



Figure 4.10: Android Lock Screen

9. Slide the lock icon to display the home screen as shown in figure 4.11.

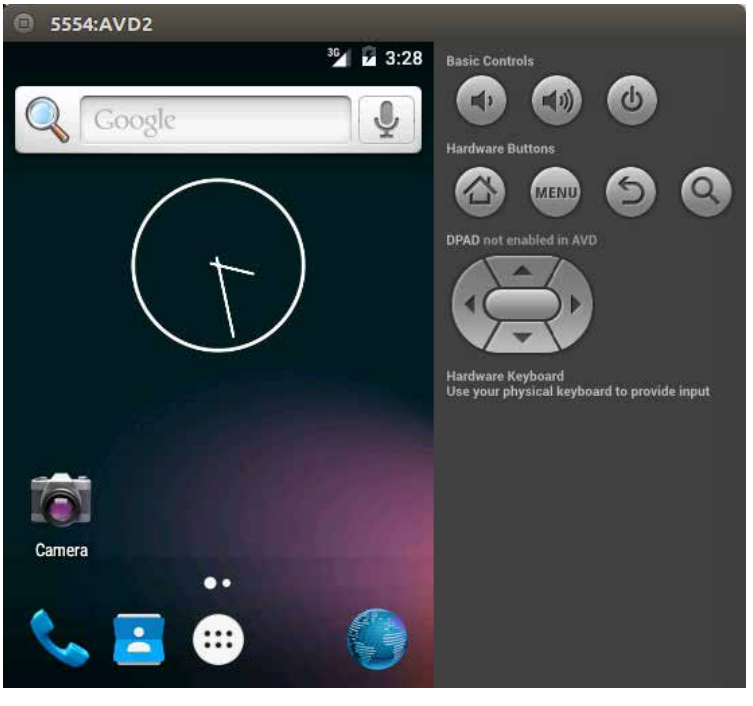


Figure 4.11: Android Home Screen

4.2.2 Hardware Options

The different hardware properties that can be specified for the AVD to emulate are listed in table 4.1.

Characteristic	Description	Property	Type
Device RAM size	Physical RAM size on the device measured in megabytes. Default value is 96.	hw.ramSize	integer
Touch-screen support	Presence/absence of a touch screen facility on the device. Default value is 'yes'.	hw.touchScreen	boolean
Trackball support	Presence/absence of a trackball on the device. Default value is 'yes'.	hw.trackBall	boolean
Keyboard support	Presence of a QWERTY keyboard on device. Default value is 'yes'.	hw.keyboard	boolean
DPad support	Presence of DPad keys on device. Default value is 'yes'.	hw.dPad	boolean
GSM modem support	Presence of GSM modem in the device. Default value is 'yes'.	hw.gsmModem	boolean
Camera support	Presence of camera in the device. Default value is 'no'.	hw.camera	boolean
Maximum horizontal camera pixels	Default value is 640.	hw.camera. maxHorizontalPixels	integer
Maximum vertical camera pixels	Default value is 480.	hw.camera. maxVerticalPixels	integer
GPS support	Presence of a GPS in the device. Default value is 'yes'.	hw.gps	boolean
Battery support	Whether the device can run on a battery. Default value is 'yes'.	hw.battery	boolean
Accelerometer	Presence of an accelerometer in the device. Default value is 'yes'.	hw.accelerometer	boolean
Audio recording support	Availability of audio record in the device. Default value is 'yes'.	hw.audioInput	boolean
Audio playback support	Availability of audio play in the device. Default value is 'yes'.	hw.audioOutput	boolean

Characteristic	Description	Property	Type
SD Card support	Whether the device supports insertion/removal of virtual SD Cards. Default value is 'yes'.	hw.sdCard	boolean
Cache partition support	Use a /cache partition on the device. Default value is 'yes'.	disk.cachePartition	boolean
Cache partition size	Default value is 66 MB.	disk.cachePartition.size	integer
Abstracted LCD density	Sets the generalized density characteristic used by the AVD's screen. Default value is 160.	hw.lcd.density	boolean

Table 4.1: Hardware Property Specification for an AVD

4.3 Run AVD from Command Line

As mentioned earlier, it is also possible to run the AVD from the command line. This section describes how the AVD can be manipulated from the command line.

4.3.1 Headless Emulator

A headless (run without display) emulator is useful to run automated tests without any supervision or when the interaction between the test runner and application is too fast to decipher. However, it needs to be understood that sometimes observing the interaction is necessary to assess the reason for failure of tests.

Emulators are named based on the communication ports and its serial number. While running AVDs, their communication ports are provided at runtime, starting with 5554 and increasing by 2 from the last used port. So, an emulator using a port 5554 is given a name such as emulator-5554. Such naming is useful when AVDs are run during the development process as the port assignment can be ignored. However, it may be confusing when more than one emulator is running simultaneously. For such cases, it is advisable to assign known ports to the communication ports to keep the specific AVD under control. Also, when running tests on more than one emulator at a time, it is best to avoid sound output as well as detach the window.

4.3.2 Disable Keyguard

Tests can be executed using a standard approach such as a standard emulator launched from Eclipse. However, running them using the standard emulator may generate some test errors. For example, in case the screen is emulator locked while running a test.

To disable the lock screen programmatically, the test-related code is required to be added in the application but needs to be removed or disabled when the application is ready to be shipped. To disable the lock screen programmatically, add the permission to the manifest file, AndroidManifest.xml and then disable the screen lock for the application under test.

Code Snippet 1 demonstrates the code to add permission in the manifest file.

Code Snippet 1:

```
<manifest>

<uses-permissionandroid:name="android.permission.DISABLE_
KEYGUARD"/>

</manifest>
```

Code Snippet 2 demonstrates the code to be added to disable the key in the onResume() method of the Activity class.

Code Snippet 2:

```
mKeyGuardManager =

(KeyguardManager) getSystemService(KEYGUARD_SERVICE);

appLock = mKeyGuardManager.newKeyguardLock("com.example.tc");

appLock.disableKeyguard();
```

In the code, the KeyGuardManager instance is first retrieved and then the KeyguardLock is obtained. Customize the package name to enable debugging. Disable the keyguard from displaying using appLock.disableKeyguard() method. The keyguard will be shown again when the reenableKeyguard() method is invoked.

4.3.3 Clean Up

Sometimes, it is necessary to clean up services and processes. This prevents the results of the processes to be influenced by the ending conditions of the previous tests. Warm-boot the emulator and start from a known condition to free all the used memory, stop services, reload resources, and restart processes. The command to open the emulator shell of the emulator and run the start and stop commands is demonstrated in Code Snippet 3.

Code Snippet 3:

```
adb -s emulator-5580 shell 'stop; sleep 5; start'
```

Note - The Android Debug Bridge (ADB) tool can be accessed on command line from the android-sdk/platform-tools folder. Browse to this directory, press Shift and right-click inside it, and select Open command window here.

The logcat option can be used with adb to monitor the evolution of the start and stop commands as demonstrated in Code Snippet 4.

Code Snippet 4:

```
adb -s emulator-5580 logcat
```

To test whether ADB is working properly, connect your Android device to your computer using a USB cable and run the following command:

```
adb devices
```

The output will be as shown in figure 4.12.

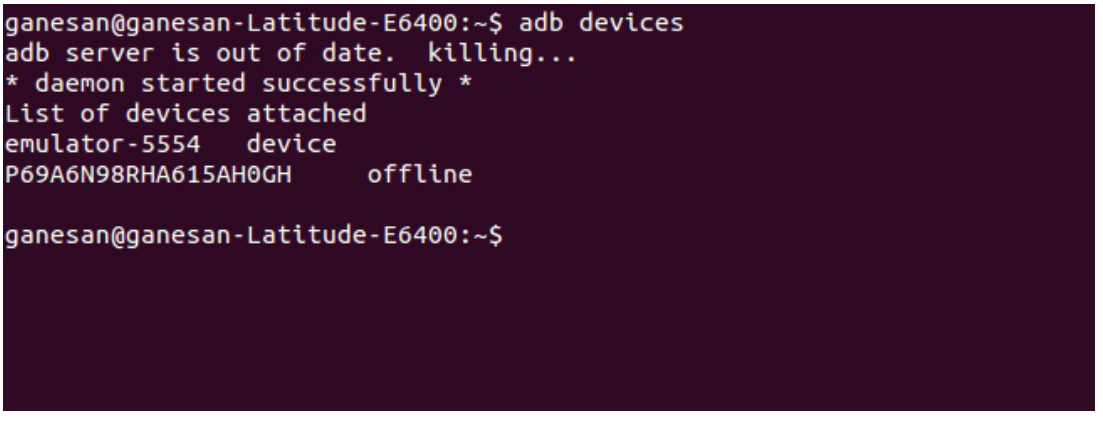


Figure 4.12: List of Attached Devices

The connected device appears in the list.

Note - If the device is connected but does not appear in the list, you will have to install the appropriate drivers. Device manufacturers may provide a downloadable driver package for their devices. Also, try installing the Google USB Driver from the Extras folder in the SDK Manager window.

4.4 Use Emulator Configurations

Sometimes, it is necessary to test the AVD outside its set options. For example, there may be a need to test the application under different locales. In such a case, consider testing the application on an emulator where the language is set to Canadian French and country to Canada.

The '–prop' command line option is used to introduce these properties as demonstrated in Code Snippet 5.

Code Snippet 5:

```
emulator -avd test -no-window -no-audio -no-boot-anim -port 5580
-prop persist.sys.language=fr -prop persist.sys.country=CA
```

Use the getprop command to verify whether the settings are successful as demonstrated in Code Snippet 6.

Code Snippet 6:

```
adb -s emulator-5580 shell "getproppersist.sys.language"
fr
adb -s emulator-5580 shell "getproppersist.sys.country"
CA
```

Use the command from the command line as demonstrated in Code Snippet 7 to kill all user data after completing the task with persistent setting and starting the emulator afresh.

Code Snippet 7:

```
adb -s emulator-5580 emu kill
emulator -avd test -no-window -no-audio -no-boot-anim -port 5580\
-wipe-data
```

4.5 Simulate Network Conditions

Host networks have different speed and latency. To ensure that the Android emulator supports network throttling, it is important to test under different network conditions.

For this, use the emulator command line options –netspeed <speed> and –netdelay <delay>.

The emulator uses default values for these options such as:

- Default network speed is 'full'.
- Default network latency is 'none' when values are not specified.

4.6 Develop Test-Driven Applications

Test-driven Development (TDD), unlike other approaches, is a strategy to write tests prior to writing code in the development process. Initially a single test is added, followed by the code to satisfy the compilation of the test, and finally, the full set of test cases is written to verify the results.

The advantages of using this method are as follows:

- Helps to avoid writing test cases, especially when present at the end of application development.
- Makes developers take responsibility for their work quality.

Test-driven development, as shown in the UML in figure 4.13 has a set of individual activities such as:

- understanding requirements
- writing a test case
- running all tests
- refactoring code

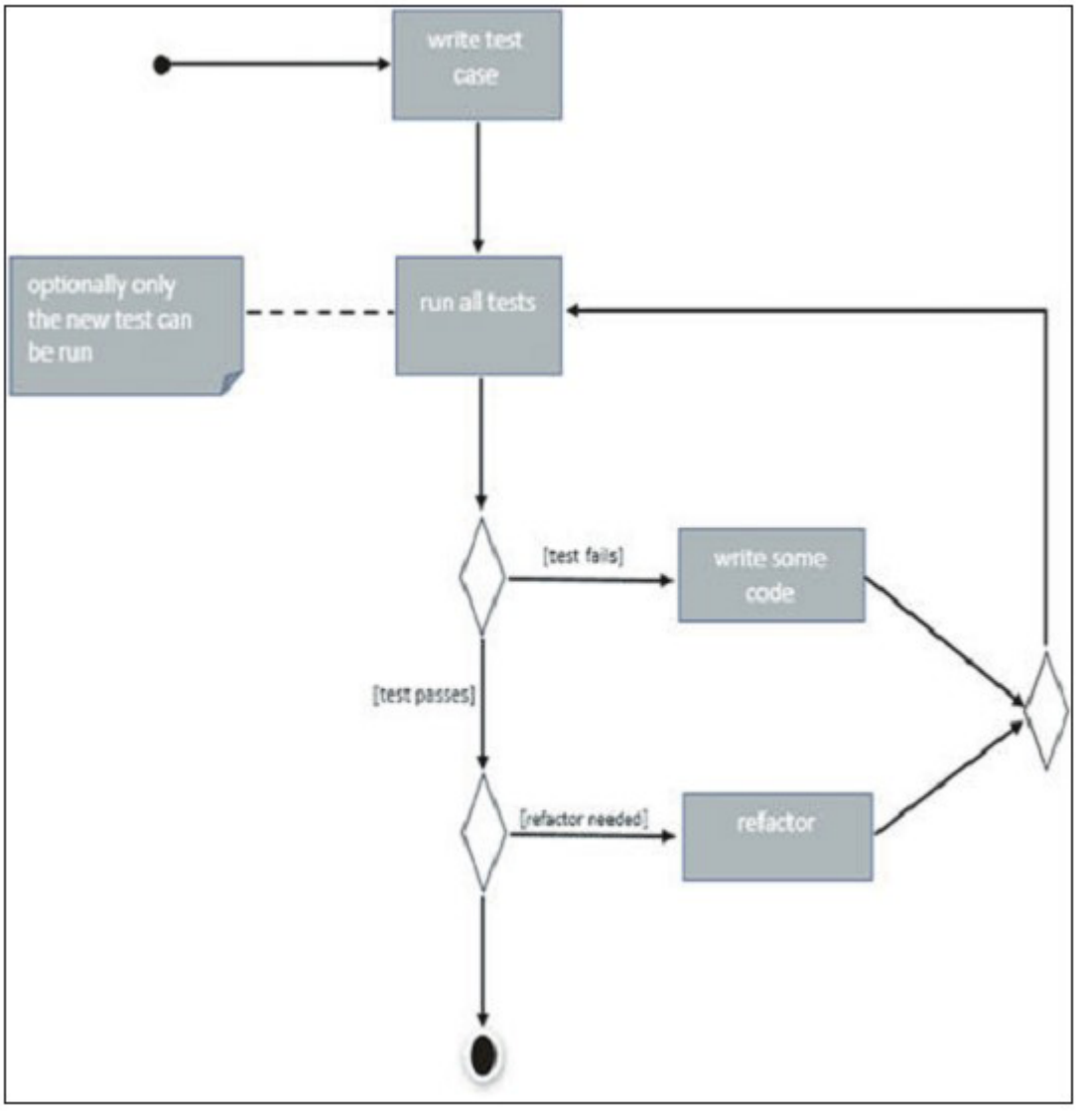


Figure 4.13: UML Diagram of the Test-Driven Development Process

4.6.1 Understand Requirements

One of the essential criteria for writing test cases is to understand the requirements. The process would be to first understand the subject under test, translate the requirements into tests, and finally, to verify the tests against the requirements with an implementation and a verification process. Any change in the requirements will change the test cases verifying the requirements as well as change the implementation code to check that everything was clearly and correctly understood and mapped to the code.

4.6.2 Write a Test Case

It is necessary to understand the problem domain and its details before writing a code. The code is written based on the problem that is at hand.

4.6.3 Run All Tests

The next step is to run the tests along with the other tests that have been written until now. An IDE with a built-in support of the testing environment cuts the development time considerably. In this stage, it is expected that the test will fail since there is no code written.

Therefore, to complete the test, additional codes are written and design decisions are taken into considerations. The additional code should be adequate to compile. The test is then compiled and executed. If the test fails again, then a minimum amount of required code is written to make the test successful. Also, the newly added test is executed to save time before running the whole suite to verify everything is still working properly.

4.6.4 Refactor the Code

When the test succeeds refactoring of the code that has been added is required to keep it tidy, clean, and minimal. Run all tests again to check if refactoring has not broken any sequence. When the test results are satisfactory, further refactoring is not necessary.

Running tests after refactoring is a safe cover to verify every case used by the application and expressed as a test case.

4.6.5 Advantages of Test-Driven Applications

The advantages of test-driven applications are as follows:

- Reduction in the development time as the output is known.
- There is a safety cover for every change of code. It is assured that the other parts of the system are not affected as long as every change of code is tested and verified.
- Less bugs.
- Few bugs found later during development results in less time spent on debugging.
- Helps in development of modular code.

- Uses stubs and mock objects.
- Has a clear approach to development process and does not require extensive prototyping or experimentation.

4.6.6 Disadvantages of Test-Driven Applications

The disadvantages of test-driven applications are as follows:

- Test-driven development relying on unit tests does not perform sufficient testing in situations where full functional tests are required to determine the success or failure.
- Developer is responsible for writing test cases and thus, the tests may have some blind spots within the code.
- Proper knowledge is required for writing test cases.
- Tests become part of the maintenance overhead cost of a project.
- Prototyping can be very difficult.
- Testing abstraction changes from time to time.

Check Your Progress

1. Which one of the following is the correct method to start AVD Manager from Eclipse?

(A)	Window → AVD Manager	(B)	Project → AVD Manager
(C)	File → AVD Manager	(D)	Navigate → AVD Manager

2. In which dialog box, can an AVD with wipe user data be enabled?

(A)	Virtual Device Dialog Box	(B)	Emulator Window
(C)	Android Virtual Device Manager	(D)	Launch Options Dialog Box

3. Which of the following is a disadvantage of test-driven applications?

(A)	Testing abstraction changes from time to time	(B)	Uses stubs and mock objects
(C)	Less bugs	(D)	Reduction in the development time as the output is known

4. Which of the following commands is used to monitor the evolution of the start and stop commands?

(A)	start	(B)	getprop
(C)	prop	(D)	logcat

5. Which permission is added to the manifest file to disable the lock screen programmatically?

(A)	AndroidManifest.xml	(B)	android.permission.DISABLE_KEYGUARD
(C)	numLock	(D)	disableKeyguard()

Answers

1.	A
2.	D
3.	A
4.	D
5.	B

Summary

- An Android Virtual Device (AVD) is an emulator configuration that enables you to model an actual device by defining various options to be emulated by the Android Emulator.
- An AVD can be created with AVD Manager and from the command line.
- Different hardware options can be specified for the AVD to emulate.
- Emulator configurations can be used to test the AVD outside its set options.
- The Android emulator should be tested under different network conditions to ensure that it supports network throttling.
- In test-driven development, tests are written, compiled, and executed prior to the development of full application code.

Onlinevarsity



LEARN @

YOUR TIME AND SPACE

05

Testing Android Projects



Welcome to the Session, **Testing Android Projects**.

This session introduces the various steps required for creating an Android application. After explaining how to create the application, the session will describe the process to run and debug the tests. Finally, the session will provide the details about building a sample test project called Currency Converter.

In this Session, you will learn to:

- Explain the process to create an Android main project using Eclipse with ADT
- Explain the process to create an Android main project using command line tools
- Explain the process to create an Android test application and a test case
- Explain the process to run a test from Eclipse with ADT
- Explain the process to run a test from Emulator and from command line
- Explain the process to debug a test
- Explain the process to create a sample test project

Session

05

Testing Android Projects

5.1 Introduction

An Android project has all the source codes required for the development of an Android application. With the help of Android SDK tools it is easy to start a new Android project with default project directories and files.

The session will explain how to create an Android project using Eclipse and command line tools. It further discusses the process of creating an Android test project and then running the tests from Eclipse and emulator. It will also describe how to debug tests. Later, the session will give a step-by-step guidance on creating a sample Android project.

5.2 How to Create an Android Main Application Project?

As mentioned earlier, all the files required for an Android application are stored in the Android project. A new project can be created using Eclipse with the ADT plugin or using the SDK tools from a command line. However, before beginning the application development, the development environment must be set up. To work in Eclipse, Eclipse needs to be installed with the ADT plug-in. The Android SDK tools are also required to be installed to enable the developer for working from the command line.

5.2.1 Using Eclipse

To create an Android project using Eclipse, perform the following steps:

1. Start Eclipse.
2. Click File → New → Project to display the New Project dialog box.
3. Navigate to Android → Android Application Project.

4. Enter details in the New Android Application dialog box as shown in figure 5.1.

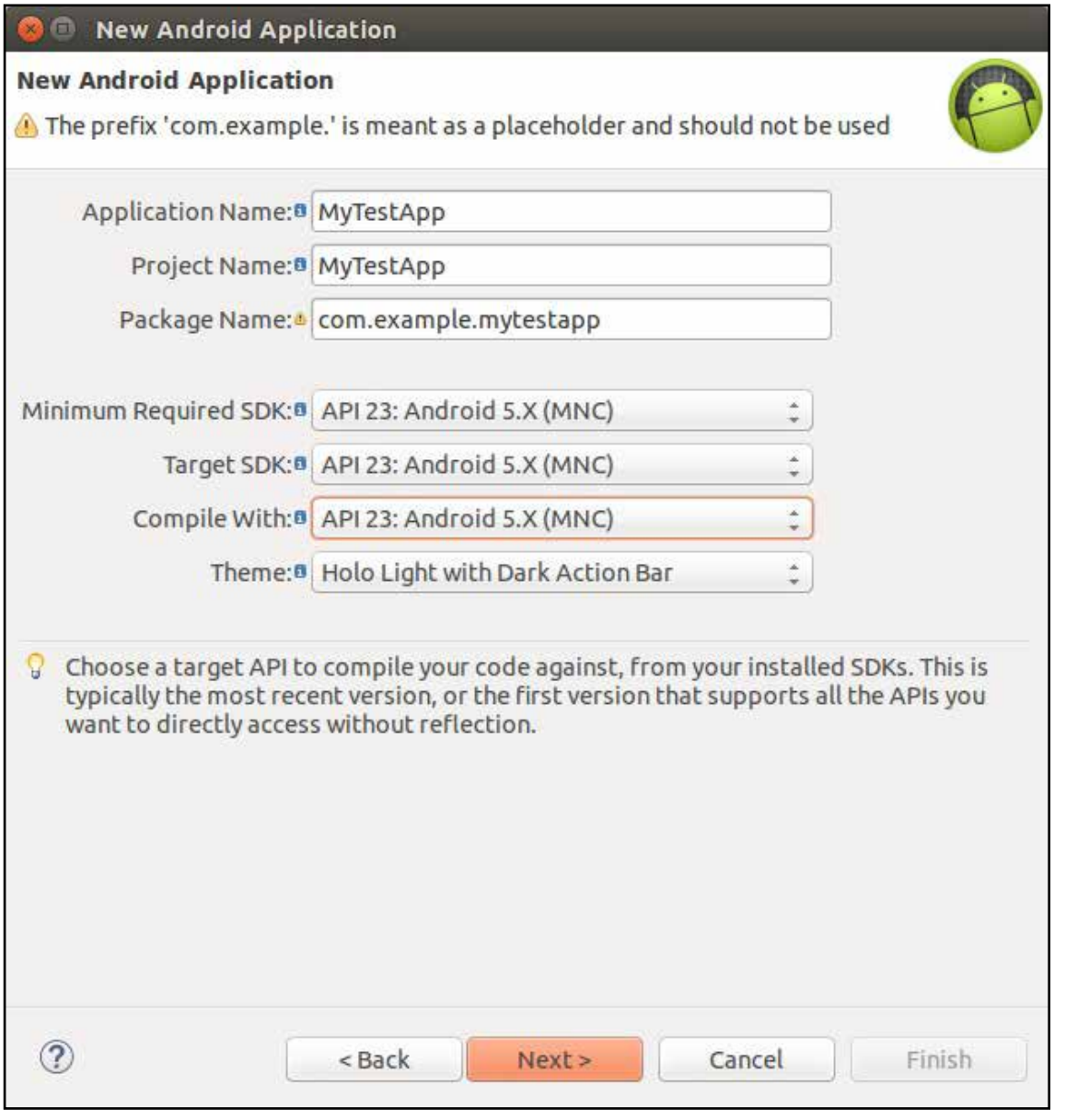


Figure 5.1: Entering Details in the New Android Application Dialog Box, API level-23

Table 5.1 explains the options shown in the dialog box in detail.

Parameters	Description
Application Name	Name seen by users for example, TestApp.
Project Name	Visible name in Eclipse project directory.

Parameters	Description
Package Name	Package represents the namespace of the application. To maintain the uniqueness of the name, the package name is usually given a reverse domain name. In this example, it is com.example.mytestapp. Such naming is followed as it is visible across all packages installed in the Android system. However, remember such namespace does not work on Google Play.
Minimum Required SDK	Lowest version of Android supporting the application. Expressed in terms of API level and is set by default. It is recommended to set this parameter to the lowest available version to enable the application to provide its core feature set. If any feature of the application is not part of the core feature set, then enable it to run only in versions that support it.
Target SDK	Highest version of Android that will test the application. It is best to test the new application in the latest versions.
Compile With	Platform version that compiles with the application. It is set by default to the latest version of Android available in the SDK. It is possible to set the parameter to older versions but at the loss of new features and lesser optimization of the application to user experience.
Theme	The Android UI style applied to the application. It is set to its default value.

Table 5.1: Parameters to Complete in New Android Application Dialog Box

5. Click Next.

6. Retain the default settings on the Configure Project screen as shown in figure 5.2.

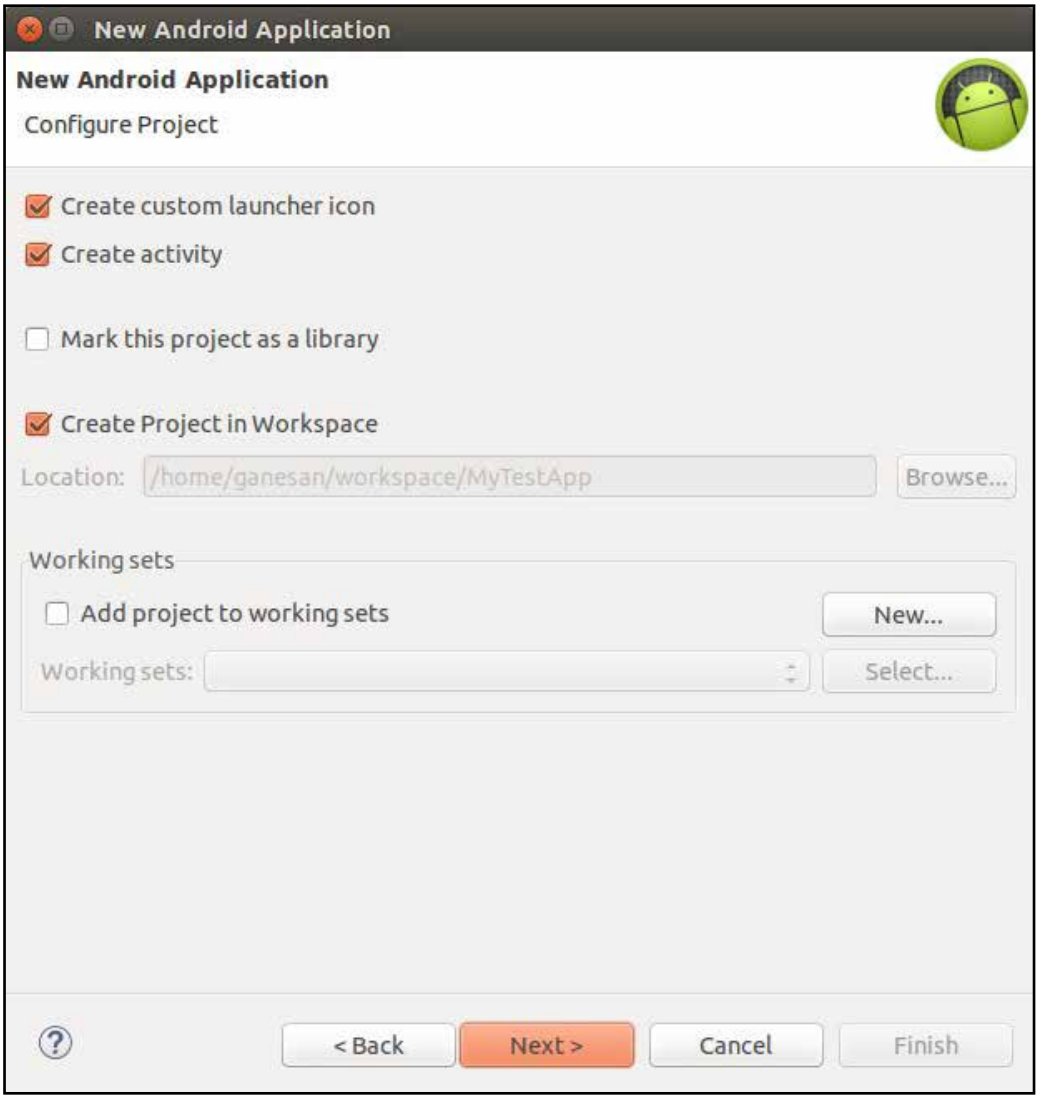


Figure 5.2: Default Settings in the Configure Project Window

7. Click Next.

8. Customize the application icon on the Configure Launcher Icon pane as shown in figure 5.3. Check whether it meets the specifications defined in the Iconography design guide. The icon is generated for all screen densities.

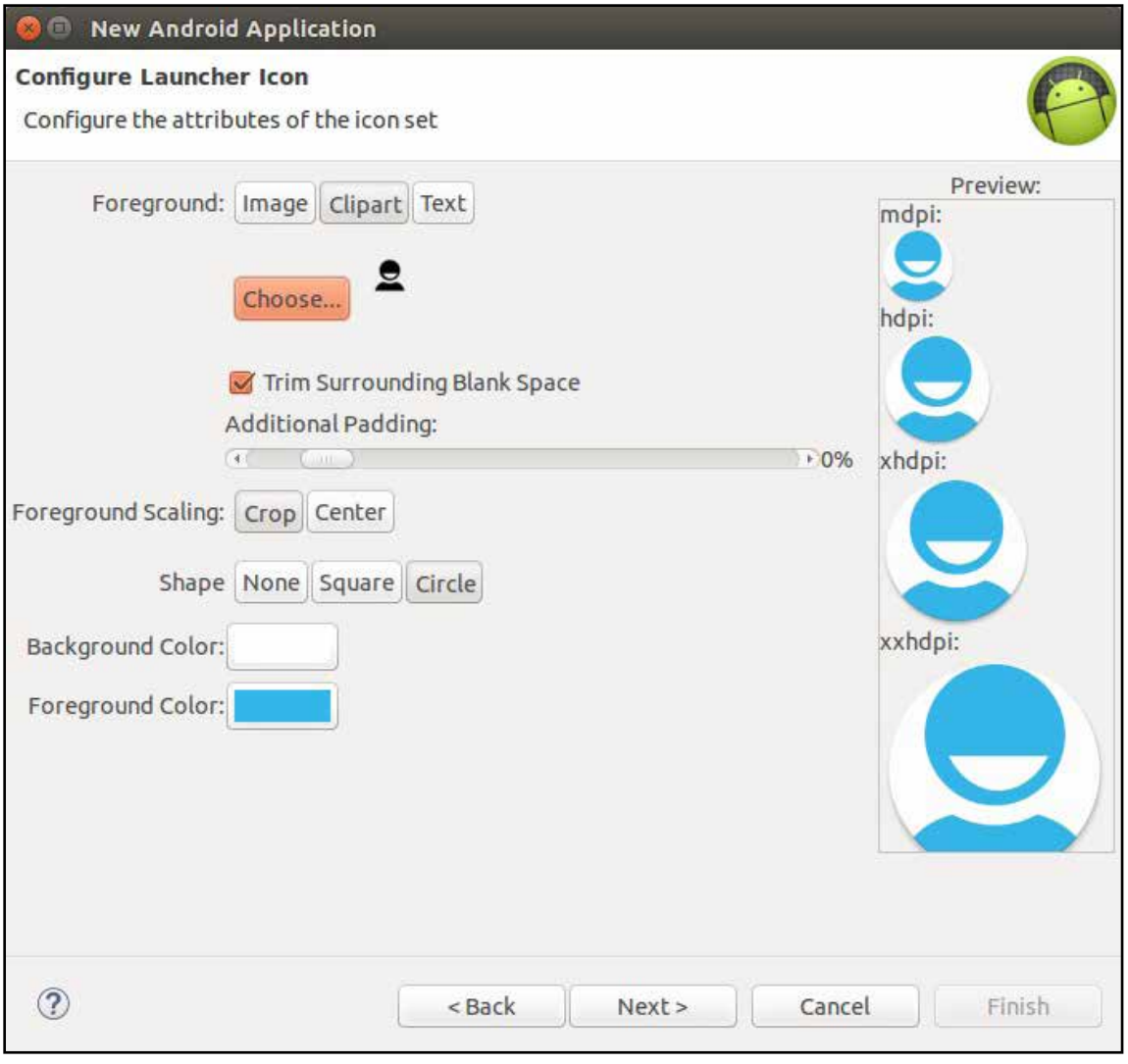


Figure 5.3: Customizing Launcher Icon

9. Click Next.

10. Select BlankActivity as the activity template on the Create Activity pane as shown in figure 5.4. The application will start with the selected template.

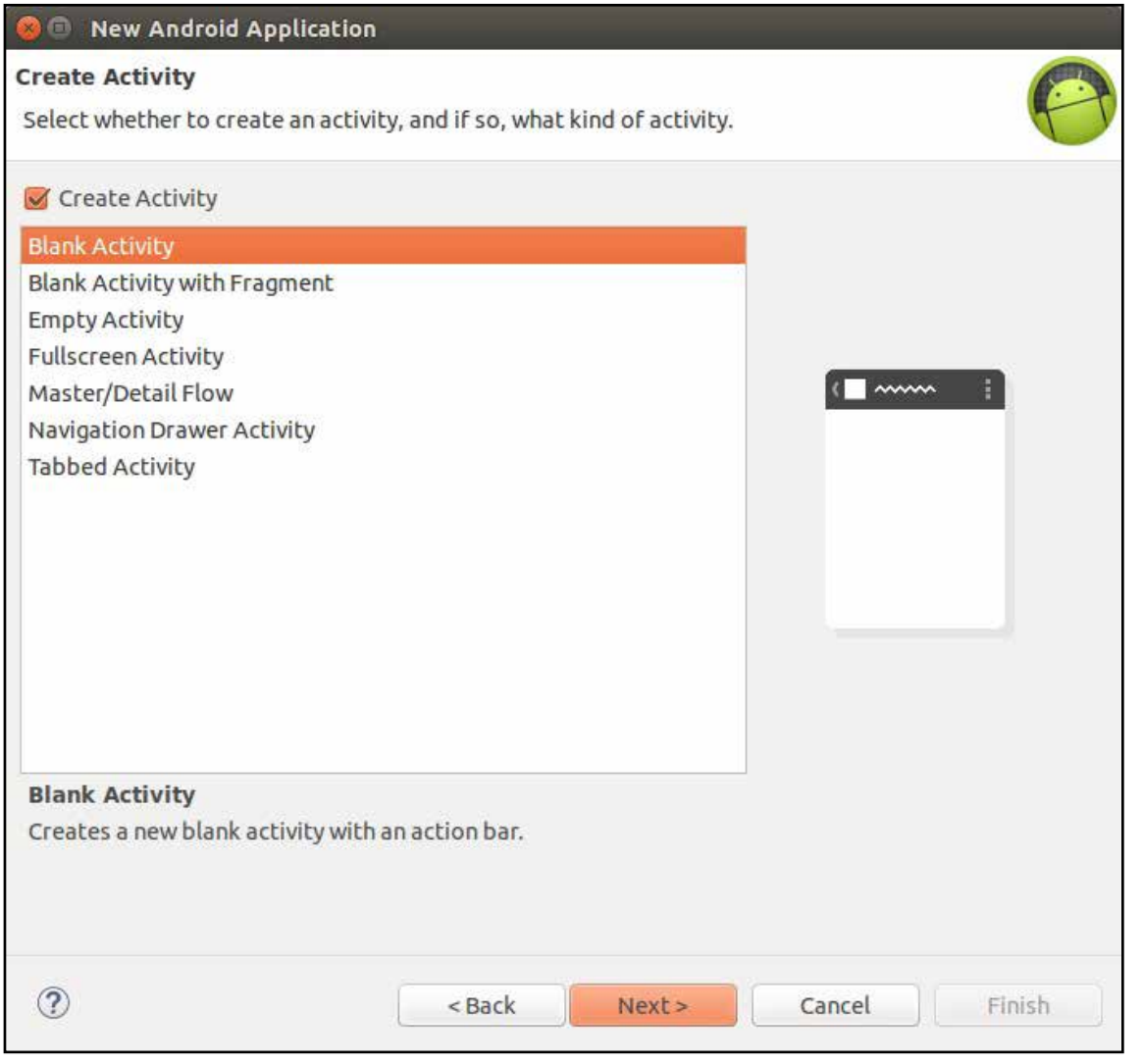


Figure 5.4: Create Activity Window

11. Click Next.

12. Retain the default setting for the New Blank Activity pane as shown in figure 5.5 and click Finish.

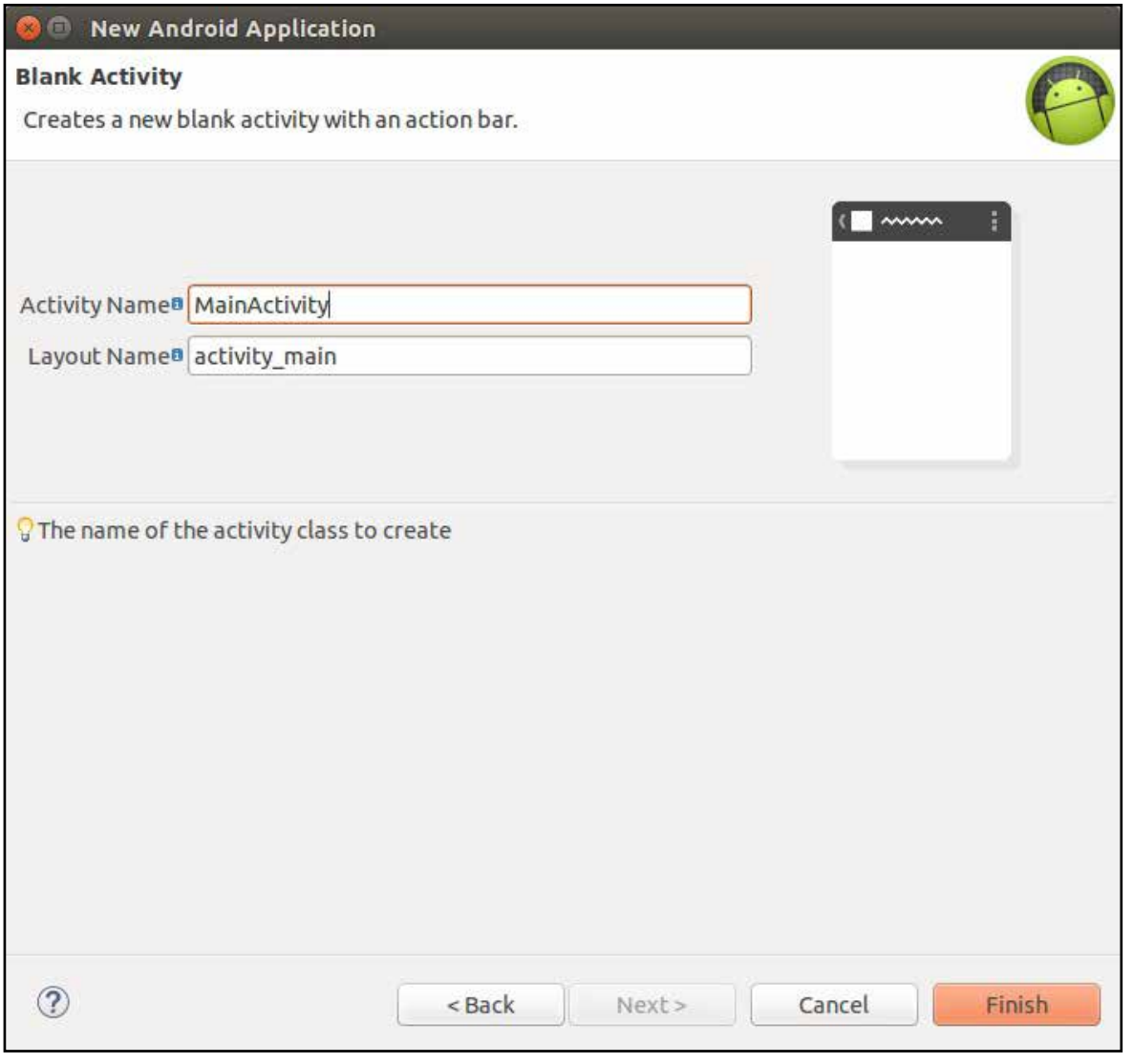


Figure 5.5: Blank Activity Screen Settings

The new Android application is created. Begin building the application.

13. Right-click the com.example.mytestapp folder and select New → Class.

14. Enter Modulo as class name and click Finish.

15. Modify the code of Modulo class as demonstrated in Code Snippet 1.

Code Snippet 1:

```
package com.example.mytestapp;

public class Modulo {

      public int getModulo(int a, int b){

            return a%b;

      }

}
```

The method, getModulo() returns the remainder of two numbers.

5.2.2 Using Command Line Tools

To create an Android project using command line tools, perform the following steps:

1. Open the command prompt in Windows or Ubuntu terminal.
2. (Optional) Set the path for tools and platform-tools directory as shown in figure 5.6.

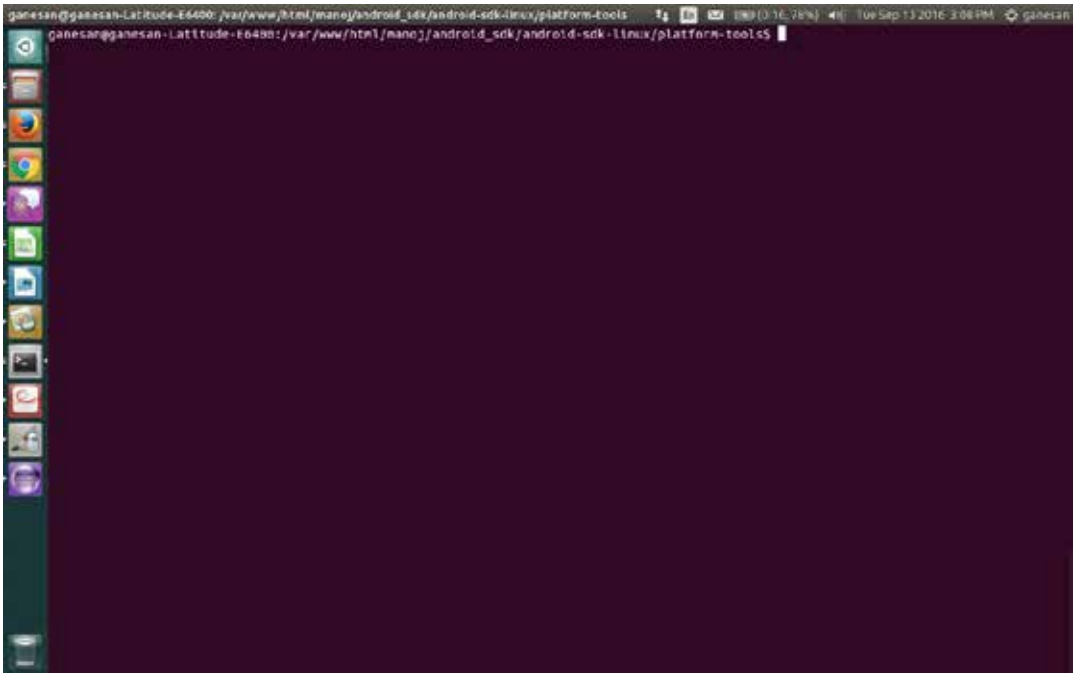


Figure 5.6: Android SDK Tools Path

Note - The Android SDK path may differ based on the location where it is installed on the system.

3. Check the SDK versions installed in the system by executing the command as demonstrated in Code Snippet 2.

Code Snippet 2:

```
android list targets
```

The command lists all the available Android platforms that are downloaded for the SDK as shown in figure 5.7.

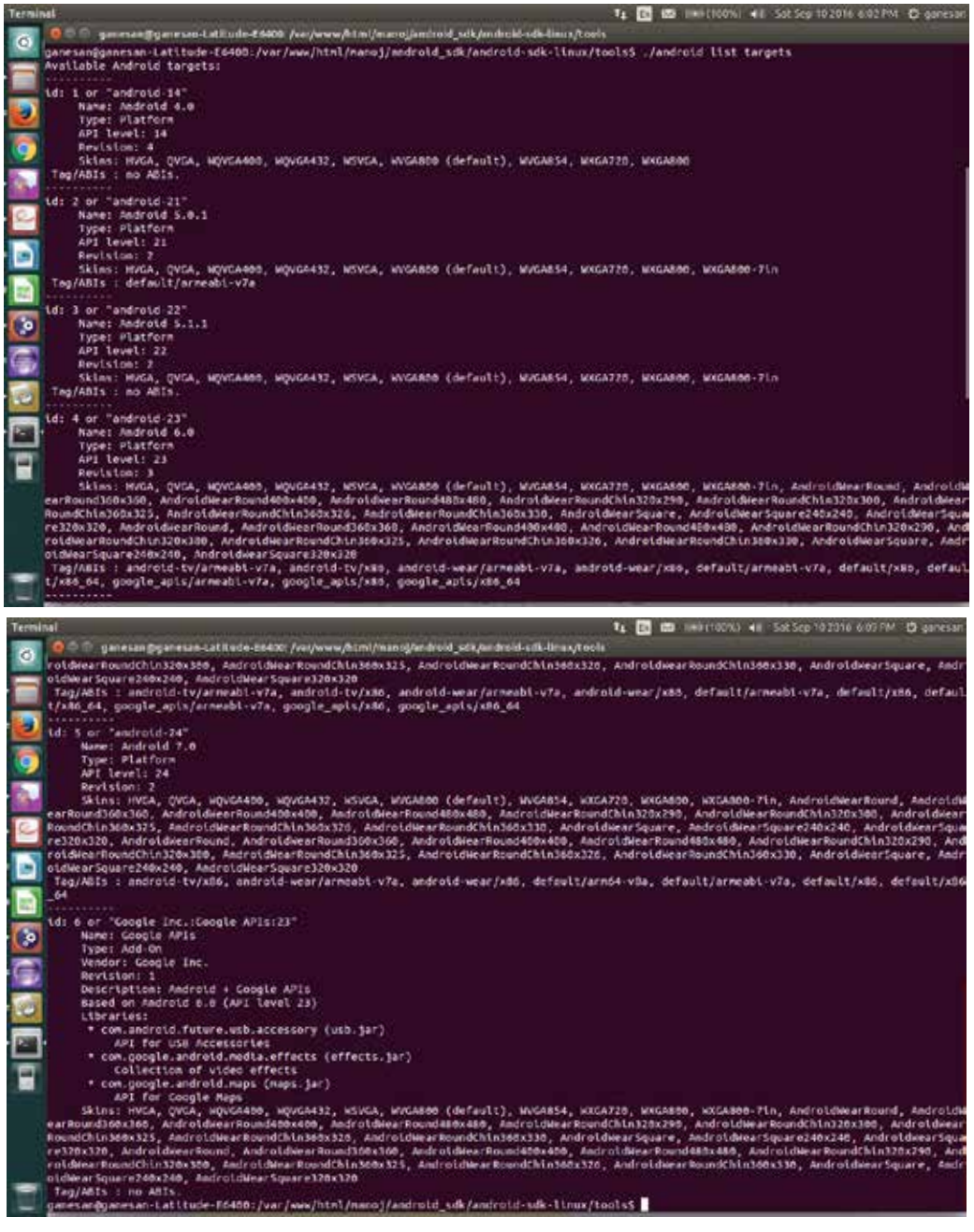


Figure 5.7: List of Available Android Platforms

Select the platform for compiling the application and note the target id. It is recommended to choose the highest version to optimize the application to latest devices.

It is however, possible to select older versions to support the application but with the disadvantage of lesser optimization to new features and latest devices. In case there are no targets listed, install a few using the Android SDK Manager tool.

4. With the build environment set, open the command prompt, and type the code as demonstrated in Code Snippet 3.

Code Snippet 3:

```
android create project --target <target-id> --name MyHelloApp
--path <path-to-workspace>/MyHelloAppProject --activity
MyHelloAppActivity --package com.example.myhelloapp
```

where,

- <target-id> will be the id selected from the list of available Android platforms.
- <path-to-workspace> will be the location where the application is saved in Android projects.

The new Android application is created as shown in figure 5.8.

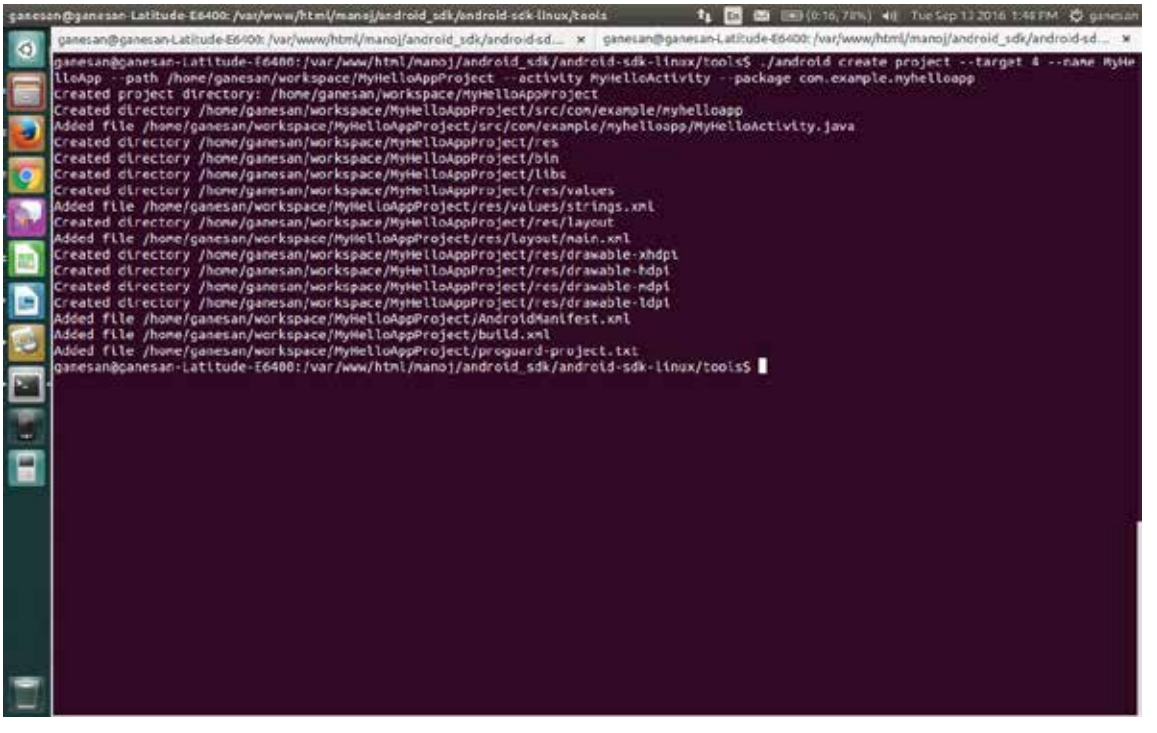


Figure 5.8: Command Line – Project Creation

Begin building the application as per requirement.

5.3 How to Create an Android Test Application?

Before assembling the test environment, the basic processes to create and run Android applications with ADT are to be set up. The preliminary step in creating a test environment is to have a separate test project that stores the test code.

A new test project stores the test code, resources, manifest file, and so on. The <instrumentation> element in the AndroidManifest.xml file connects the new test package to the application under test. The directory structure for the new project is same as any Android application.

Creating a test project in Eclipse with ADT has some advantages because ADT has several built-in features that help in setting up and managing the test environment successfully.

The features provided by ADT include the following:

- Quick creation of the test project that links to the application being tested.
- Automatic insertion of the specific <instrumentation> element to the test package manifest file.
- Quick import, inspection, and analysis of the application classes that is being tested.
- Creation of run configurations of the test package.
- Insertion of flags in the run configurations to be recognized by the Android testing framework.
- Provision to run test package without leaving Eclipse.
- Automatic building of the application being tested and test package.
- Installation of the application being tested and test package on the device or emulator.
- Run the test package.
- Provision of a separate window in Eclipse to display results.

To create a test project in Eclipse with ADT, perform the following steps:

1. Select File → New → Other in Eclipse as shown in figure 5.9.

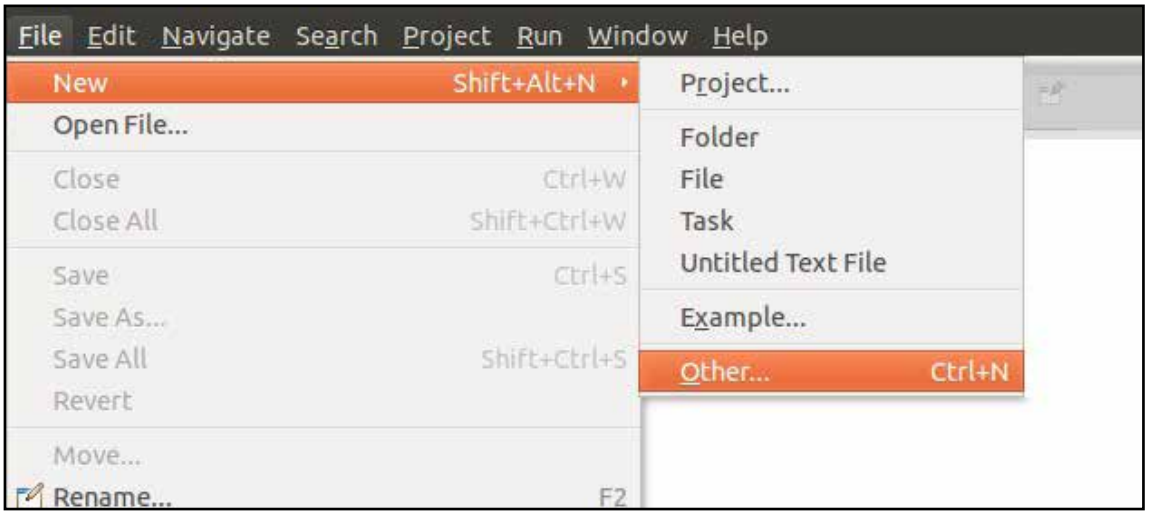


Figure 5.9: Navigating to Select a Wizard

2. In the 'Select a wizard' screen of the New dialog box, scroll through the Wizards list to the Android folder.
3. Expand the Android folder.
4. Select Android Test Project as shown in figure 5.10.

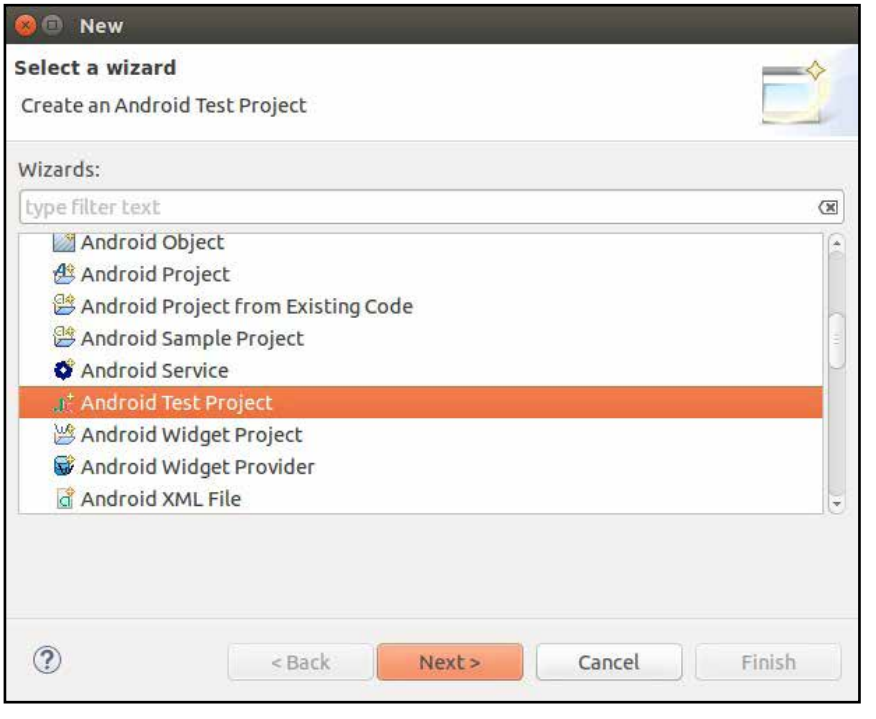


Figure 5.10: Selecting Android Test Project

5. Click Next to open the New Android Test Project wizard as shown in figure 5.11.

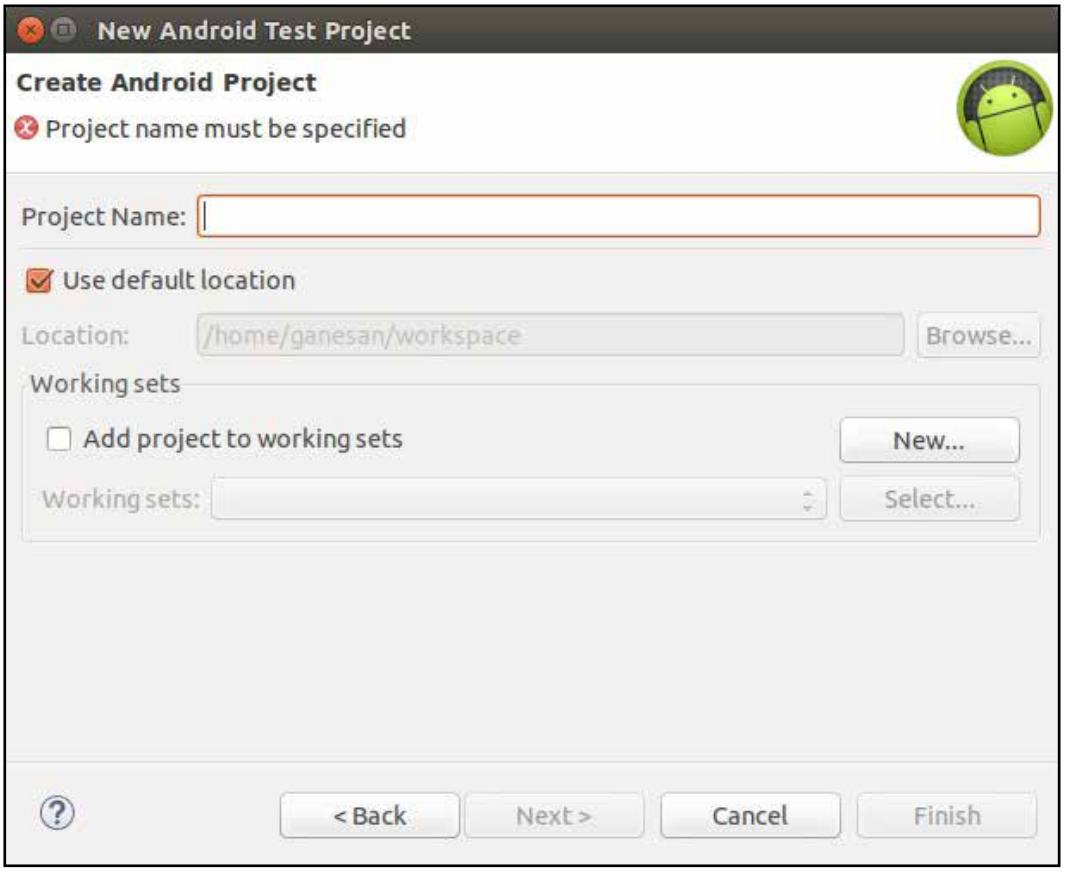


Figure 5.11: New Android Test Project Wizard

Use the New Android Test Project to generate a new test project. The new project can be created any time and will have the proper structure, including the <instrumentation> element in the manifest file. The new project dialog box appears as soon as a new Android main application project is created. It can also be used to create a test project for an application that has been created earlier.

6. Type MyAppTestProject in the Project Name box of the New Android Test Project dialog box as seen in figure 5.12.

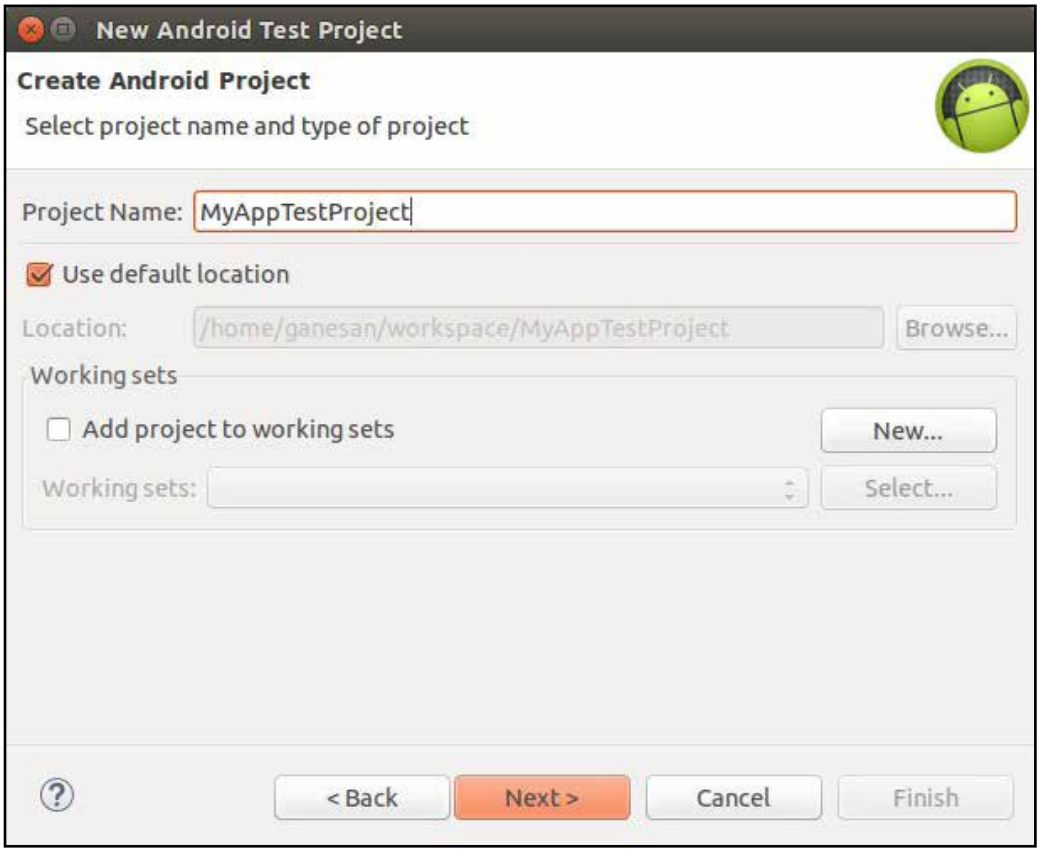


Figure 5.12: New Android Test Project

7. Click Next.

8. Select 'An existing Android project' from the Select Test Target pane.

9. Select the MyTestApp project as shown in figure 5.13.

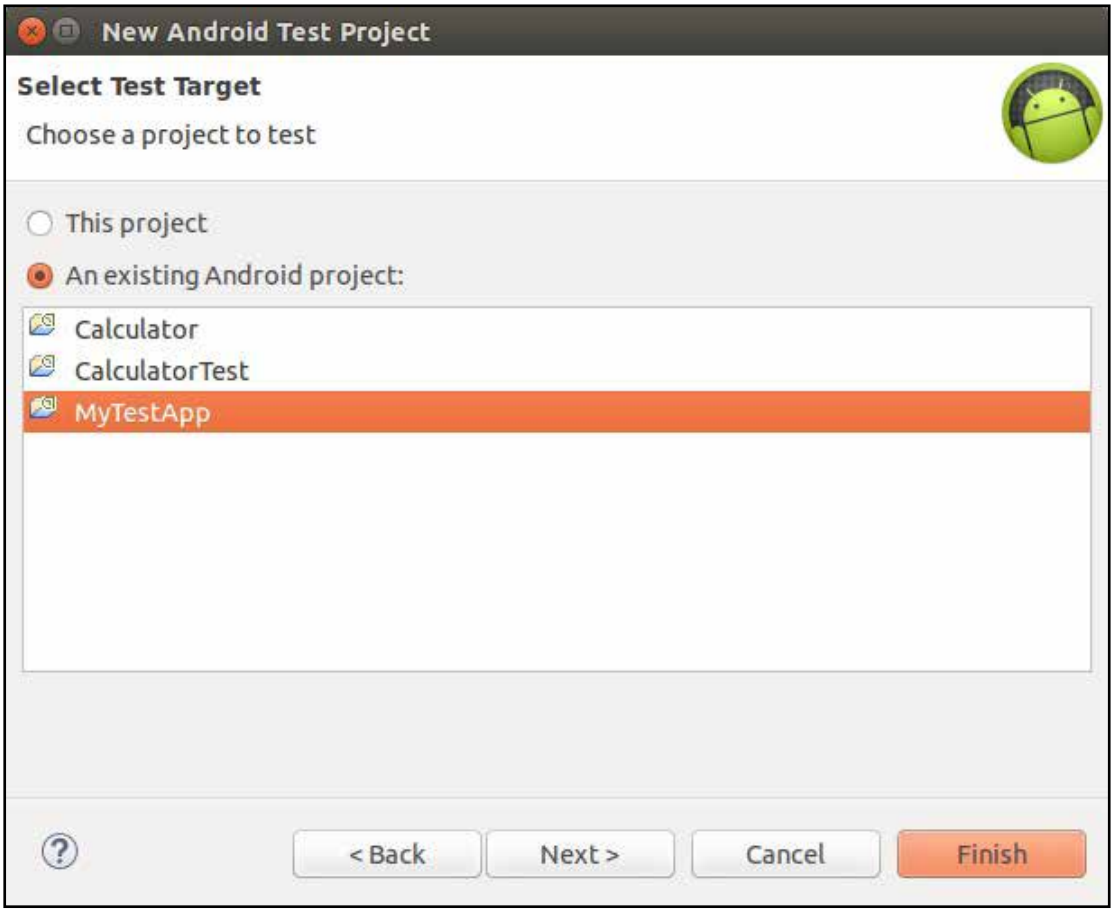


Figure 5.13: Select the Project to be Tested

10. Click Finish.

Thus, a test project is created for testing the existing MyTestApp application.

5.4 How to Create a Test Case?

The main test case extends from the Android test case classes or JUnit classes because it provides the best testing features. When test packages are executed in Eclipse with ADT, its results appear in the JUnit view. Test cases can combine Activity classes, test case classes, or ordinary classes. To create a test package, use one of Android's test case classes defined in android.test package.

A test package does not require an Activity, although one can be defined. The Android test classes for Activity objects also provide instrumentation for testing an Activity.

To create a test case, the first step is to choose the Java package identifier used for the test case classes and the Android package name.

To add a test case class to a test project, perform the following steps:

1. Open the test project in the Package Explorer pane.
2. Navigate to com.example.mytestapp.test folder from the Package Explorer tab as shown in figure 5.14.

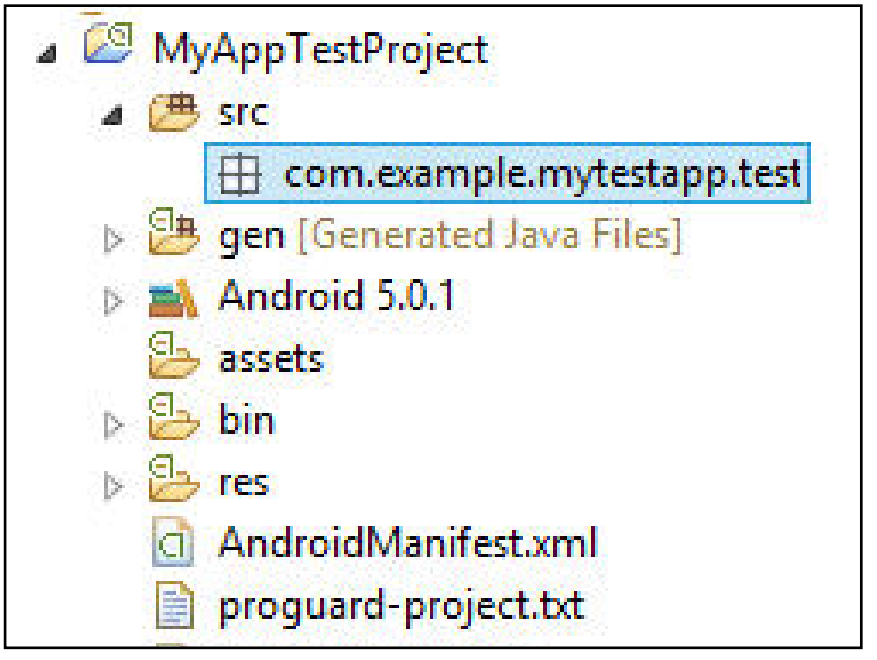


Figure 5.14: Project Folder Hierarchy

3. Right-click the package to display the context menu.
4. Select New → JUnit Test Case to display the New JUnit Test Case dialog box.

5. Type TestCase1 as shown in figure 5.15.

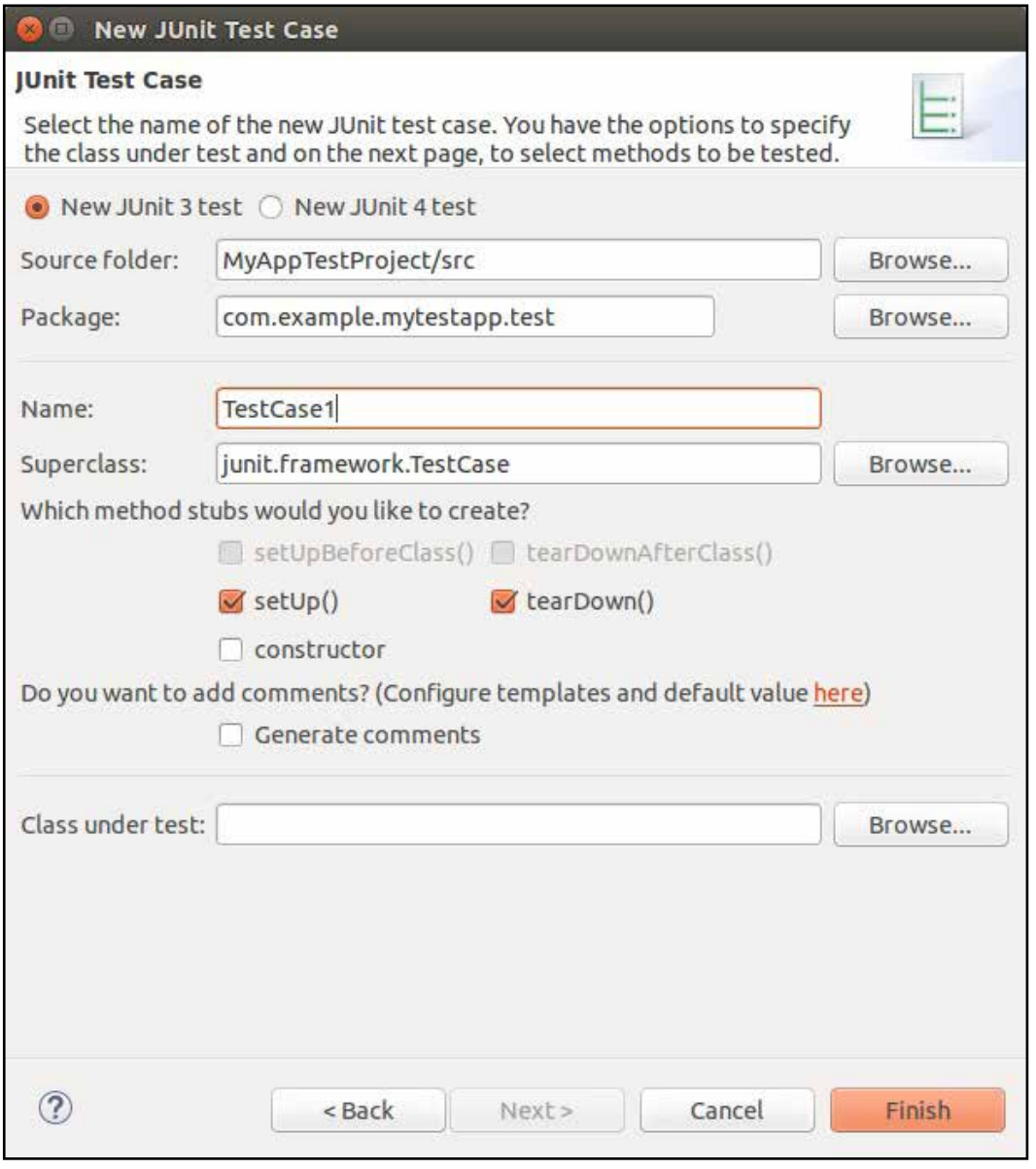


Figure 5.15: JUnit Test Case Screen

6. Click Finish.

7. Modify the Code Snippet of TestCase1 class as demonstrated in Code Snippet 4.

Code Snippet 4:

```
package com.example.mytestapp.test;
import junit.framework.TestCase;
import com.example.mytestapp.Modulo;
public class TestCase1 extends TestCase {
	private static final boolean DEBUG = false;
	public TestCase1() {
		super();
		//TODO Auto-generated constructor stub
	}
	public TestCase1(String name) {
		super(name);
		// TODO Auto-generated constructor stub
	}
	public void testModulo(){
		int expected=2;
		Modulo m1 = new Modulo();
	  assertEquals(expected,m1.getModulo(10, 5));
	}
	protected void setUp() throws Exception {
		super.setUp();
	}
	protected void tearDown() throws Exception {
		super.tearDown();
	}
}
```

Ensure that the constructor for the class is set correctly with no arguments. Such a constructor is required by JUnit. Each base test case class has its unique constructor signature. Add a call to the constructor as the first statement. A test method named testModulo() has been created. The assertEquals() method checks whether the two arguments are same. If yes, the test succeeds else it fails.

To control the test environment, override the following methods:

- **setUp()**: Is the first method to be invoked in the class. Use it to perform the following functions:
  - Set up the environment for the test fixture.
  - Instantiate a new Intent with the action ACTION_MAIN. Use this intent to start the Activity being tested.
- **tearDown()**: Is the last method to be invoked after all the test methods in the class. Use it to perform the following functions:
  - Waste collection.
  - Reset the test fixture.
- **testPreconditions()**: Is a method added to test class. Use it to perform the following function:
  - Correct initialization of the application being tested. The test failure of this method indicates that the initial conditions were not error-free. In this case, expect further test results, no matter whether the tests succeed or not.

5.5 How to Run the Test?

Test project can be executed from Eclipse, emulator, or the command line.

5.5.1 Using Eclipse

The result of executing a test package in Eclipse with ADT is seen in the Eclipse JUnit view. Eclipse can run the entire package or a single test at a time. Remember to attach the device or use an emulator to run the test package. When using an emulator, use an Android Virtual Device (AVD) that uses the same target as the test package. When running the test package, Eclipse uses the adb command so it is similar to running tests from the command line.

There are two ways to run a test in Eclipse. The first method is used for running all tests and the second method is used for running a single test from Eclipse.

These are explained as follows:

- Right-click the project and click Run As → Android JUnit Test from the test project context menu as shown in figure 5.16. The Android JUnit Test can also be selected from the main menu's Run option. Eclipse will run all the tests.

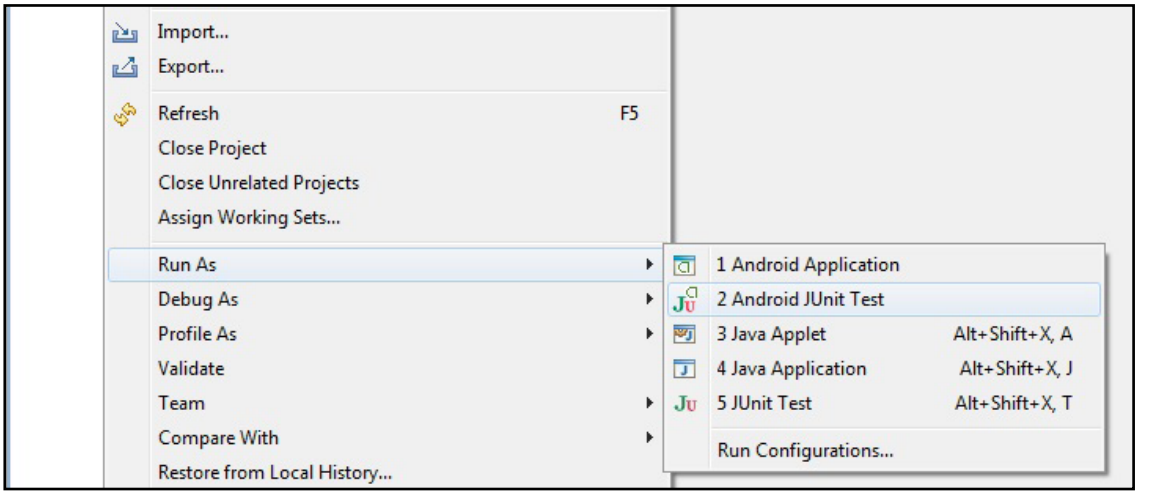


Figure 5.16: Running MyAppTestProject

- Run the test by creating an Eclipse run configuration for the test project. This method is used when multiple test suites are to be tested, where each test suite consists of several selected tests from the project. Eclipse will run a single test case.

To execute the test suite using run configuration, perform the following steps:

1. Select the test project in the Package Explorer pane.
2. Right-click the project to display the context menu and select Run As → Run Configurations as shown in figure 5.17.

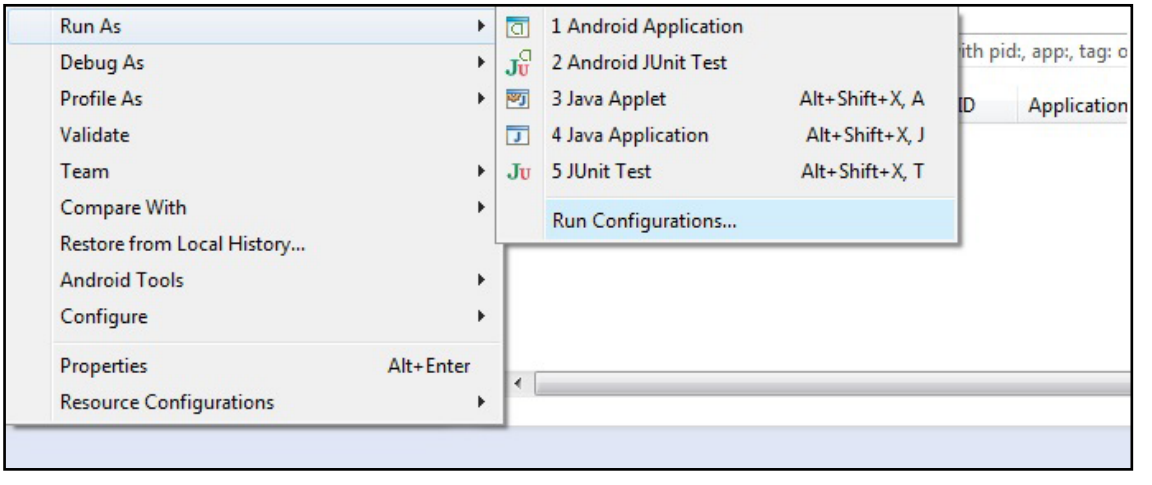


Figure 5.17: Running Test Package as Run Configurations

The Run Configurations dialog box is displayed.

3. Select Android JUnit Test entry in the left pane of the Run Configurations dialog box.

4. Right-click Android JUnit Test in the left pane and click New.

   The MyAppTestProject configuration is added to the Test tab as shown in figure 5.18.

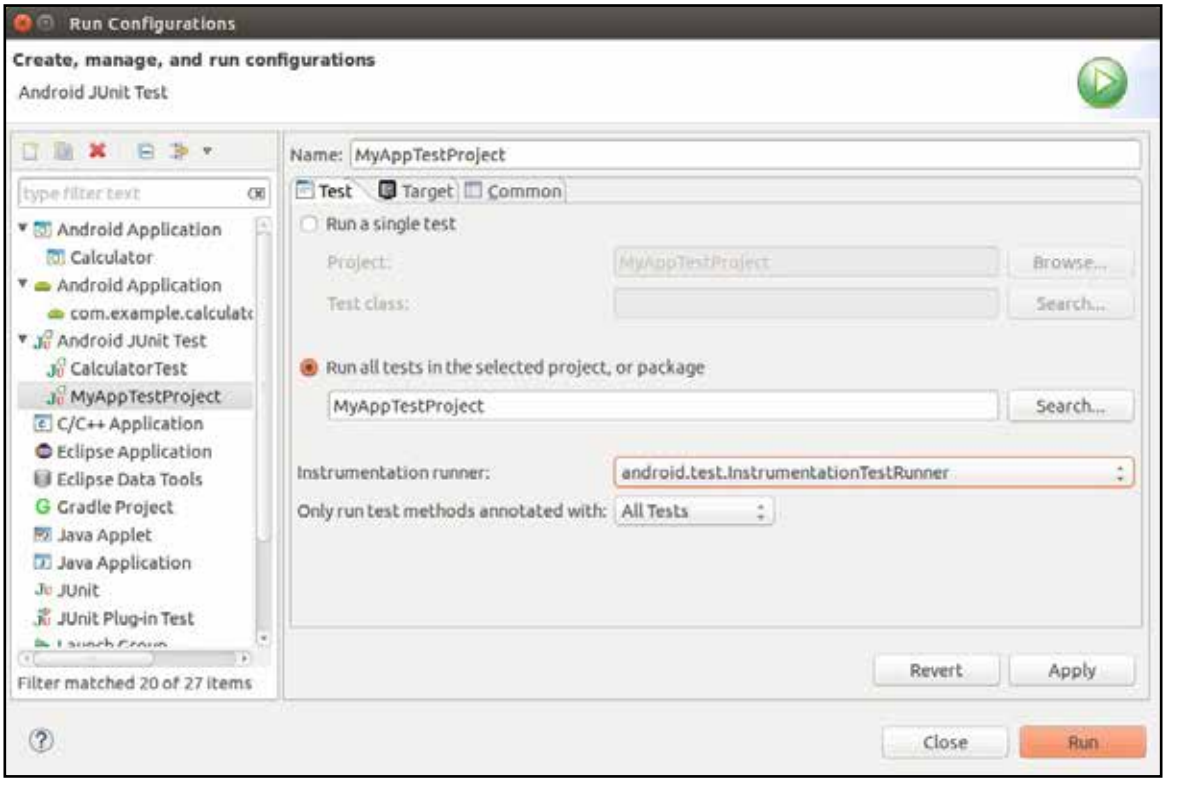


Figure 5.18: Run Configurations

5. To run all test classes:

   - Click Run all tests in the selected project or package.
   - Enter the name of the project or package name in the box.
   - Select android.test.InstrumentationTestRunner from the Instrumentation runner list.

6. Click the Target tab in the right pane.

   - Select ‘Automatically pick compatible devices’ for deployment target selection.

- Select an existing AVD from the AVD selection table as shown in figure 5.19.

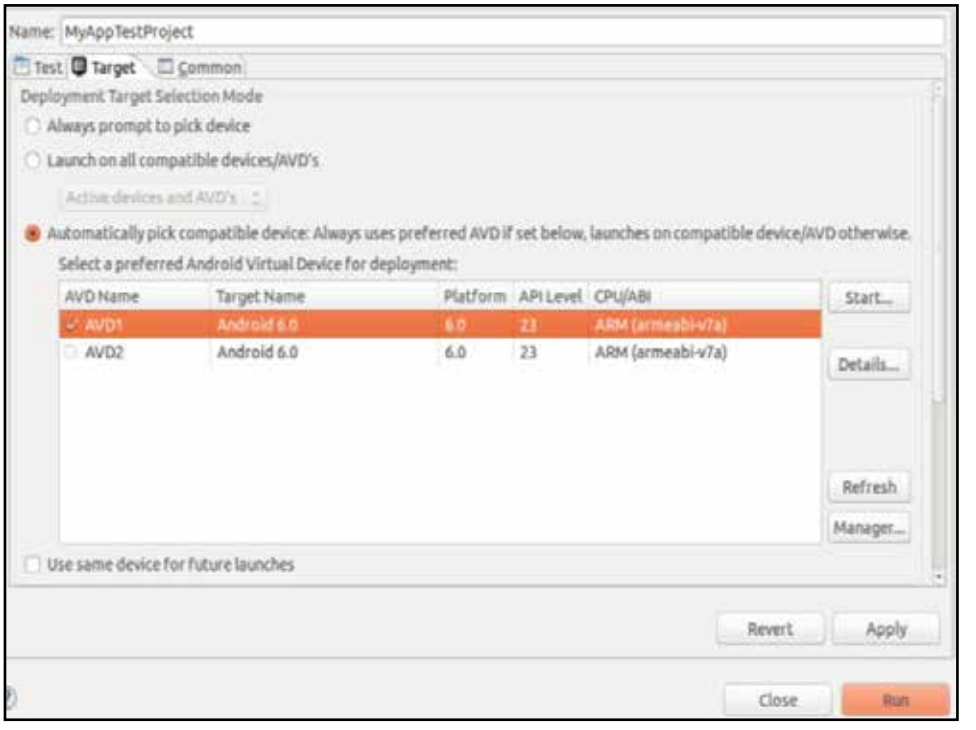


Figure 5.19: Deployment Target Selection in the Target Tab

- Set the values in the Android Emulator Launch Parameters section as shown in figure 5.20.

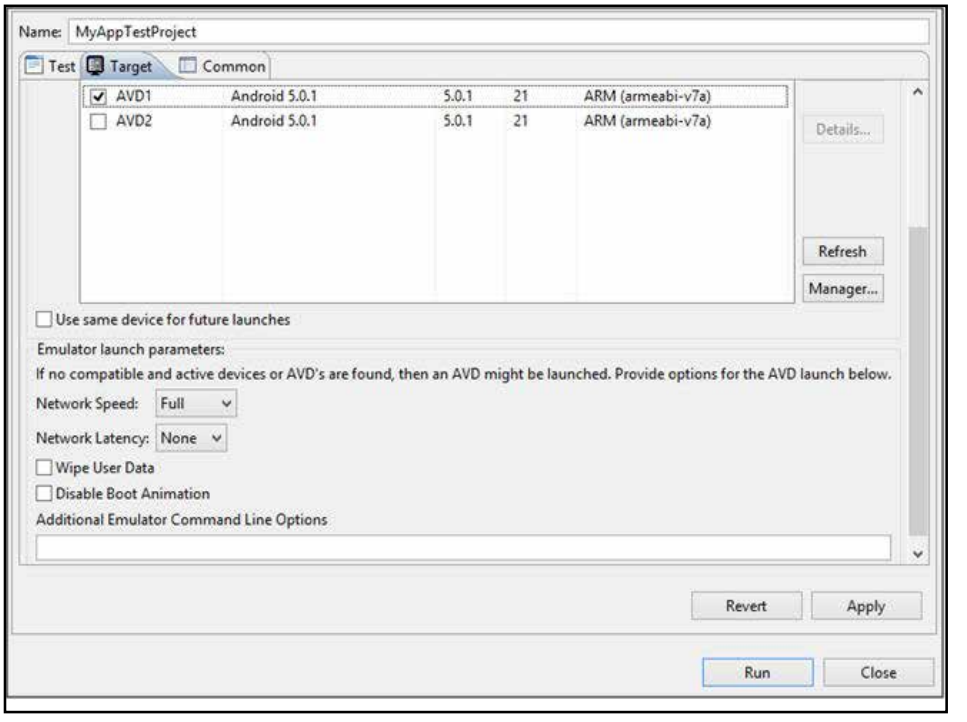


Figure 5.20: Emulator Launch Parameters in the Target Tab

7. Click the Common tab and do one of the following:
   - Click Local to save the configuration locally in the Save As pane.
   - Click Shared to save the configuration to another project.
8. Click Run to begin the test.

   The test output will be as shown in figure 5.21.

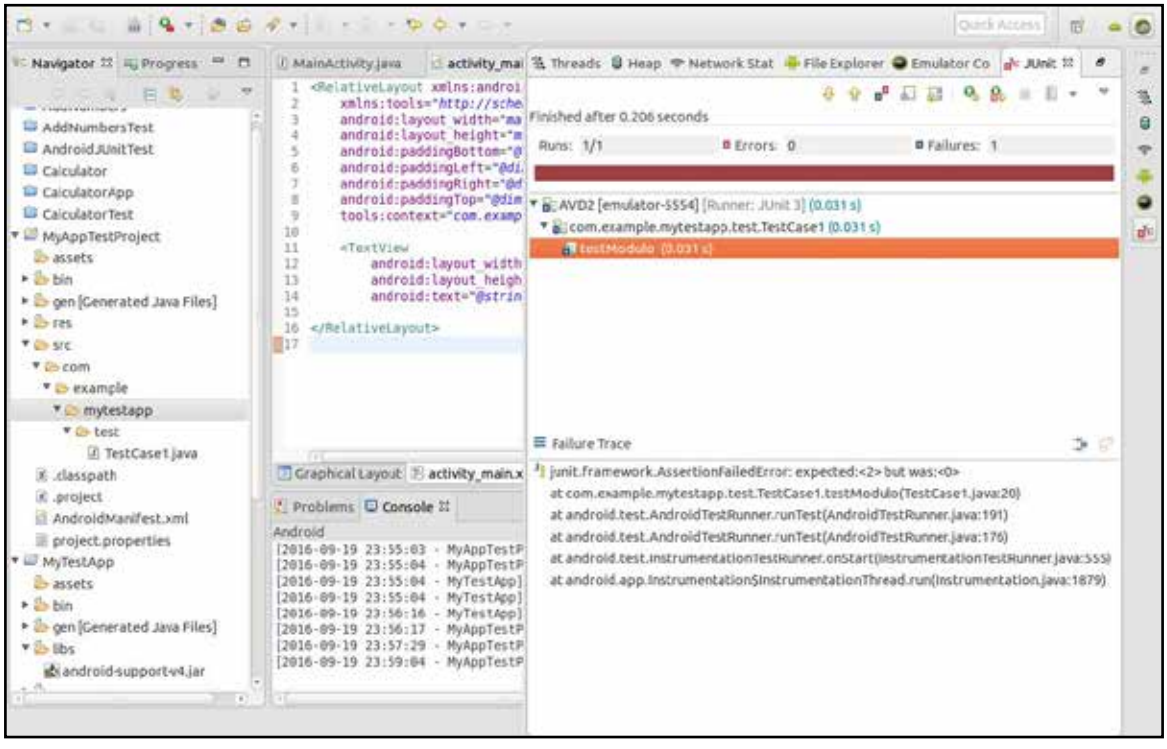


Figure 5.21: Test Result

The test fails due to the AssertionFailedError.

5.5.2 Using Emulator

Instrumentation is the only means to run tests from the emulator. The emulator has a default system image which has the Dev Tools installed. The Dev Tools application lists the handy tools and settings.

1. Click Dev Tools in the Emulator.

2. Select Instrumentation from the Dev Tools as shown in figure 5.22.

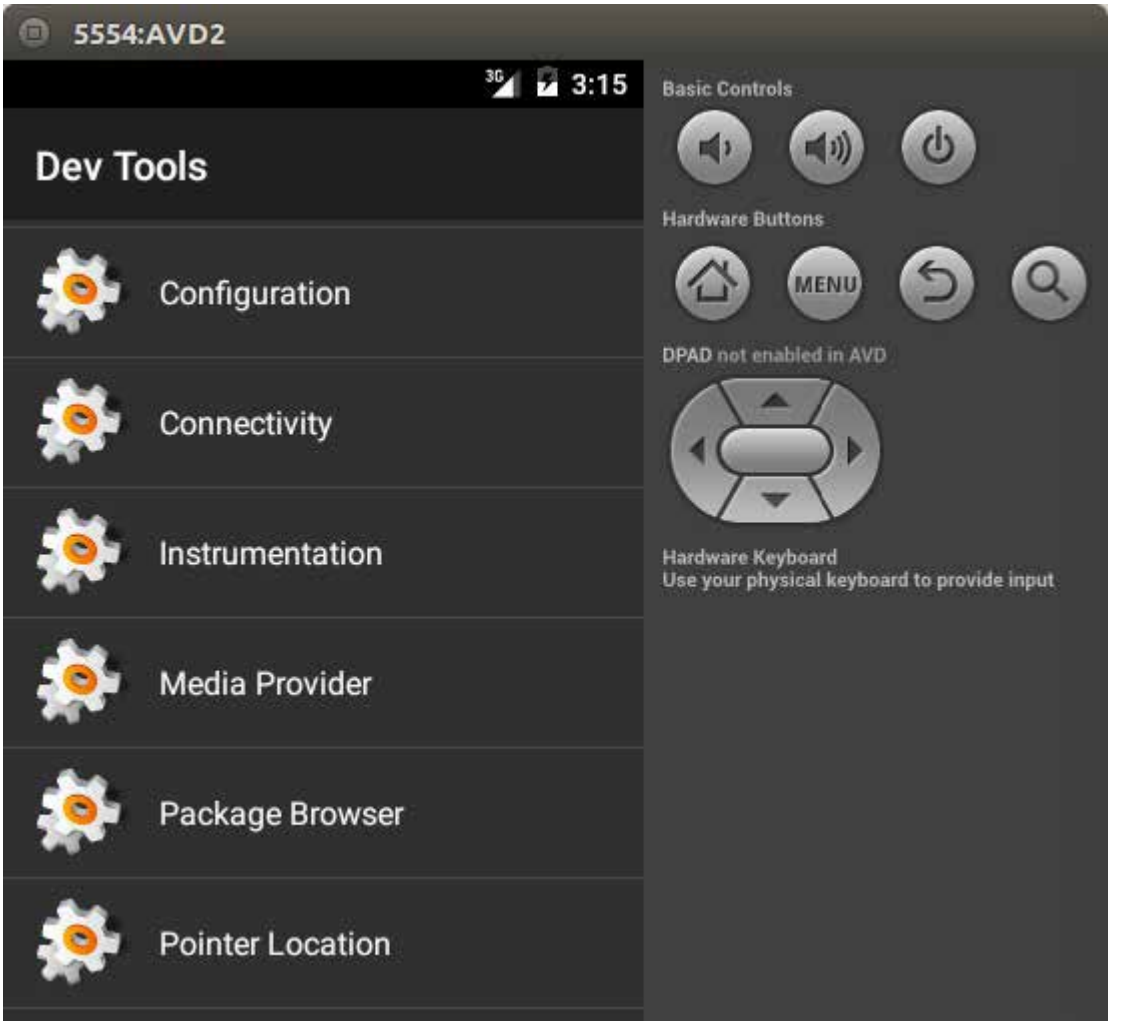


Figure 5.22: Dev Tools List

3. A list of installed packages having the instrumentation tag in their AndroidManifest.xml is listed as shown in figure 5.23.

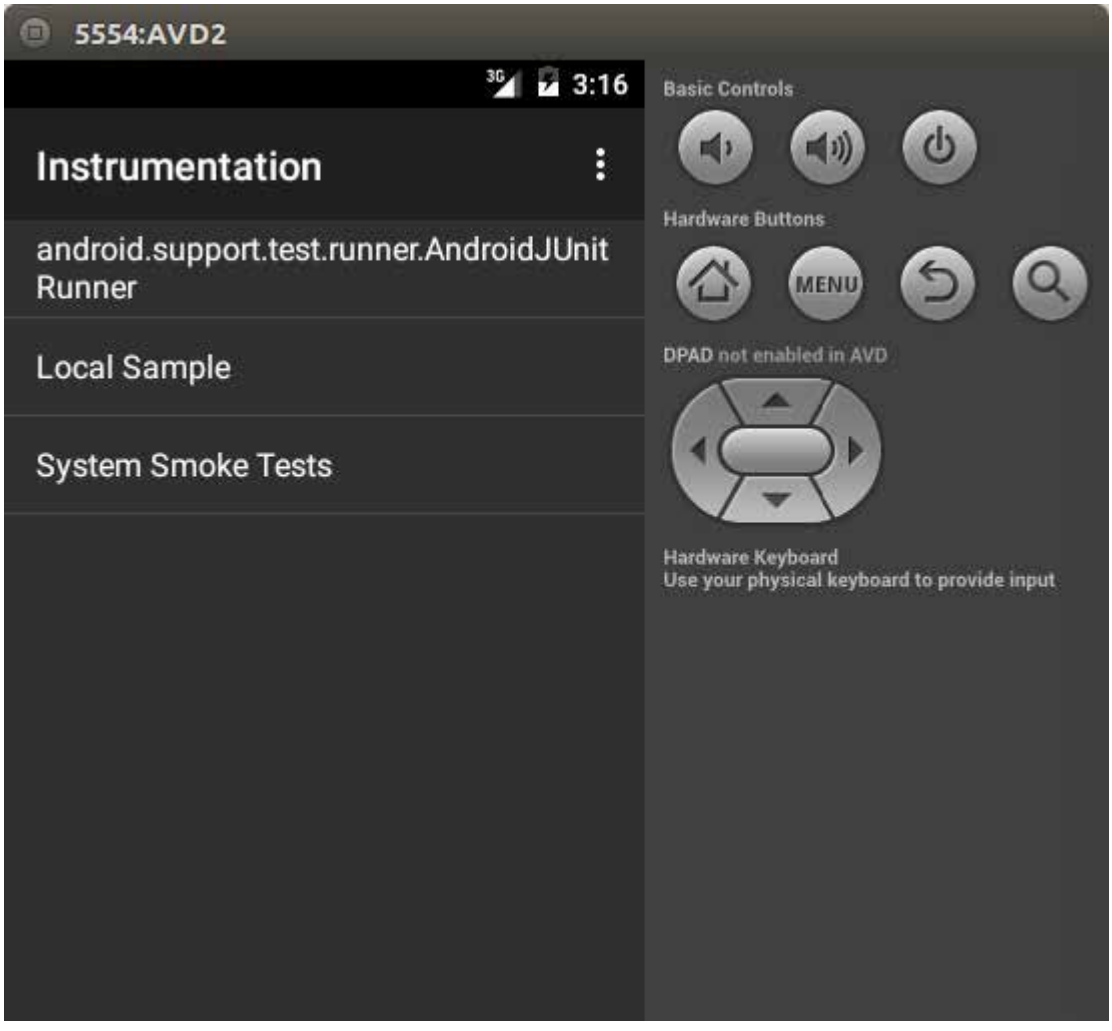


Figure 5.23: Instrumentation Tab

Remember that by default, packages are listed using android.test.InstrumentationTestRunner so recognizing the test package may be difficult when the list contains more than one test package.

4. Set an optional label in the manifest, under the Instrumentation tab, for easy identification of the test package as shown in figure 5.24.

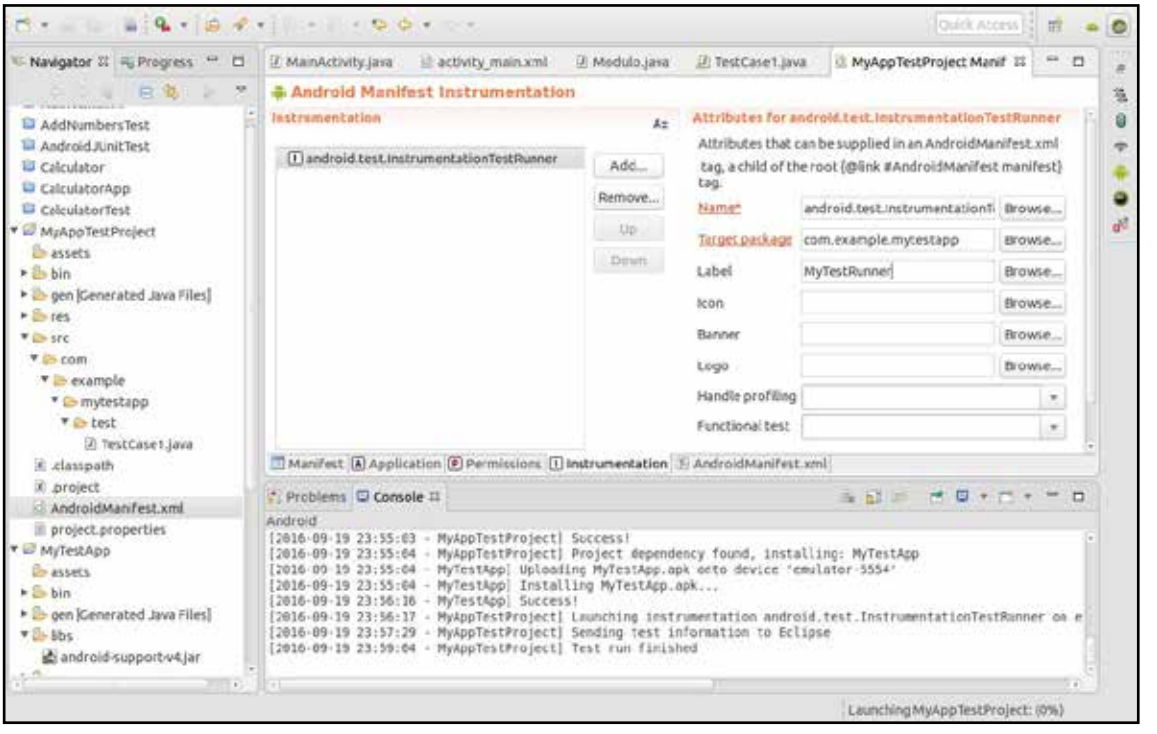


Figure 5.24: Setting Test Label in Manifest

5. Select the test with the new label after the Instrumentation list is re-displayed as shown in figure 5.25.



Figure 5.25: Label Displayed in Instrumentation List

6. Run the test, MyTestRunner. The test results are displayed through LogCat as shown in figure 5.26.

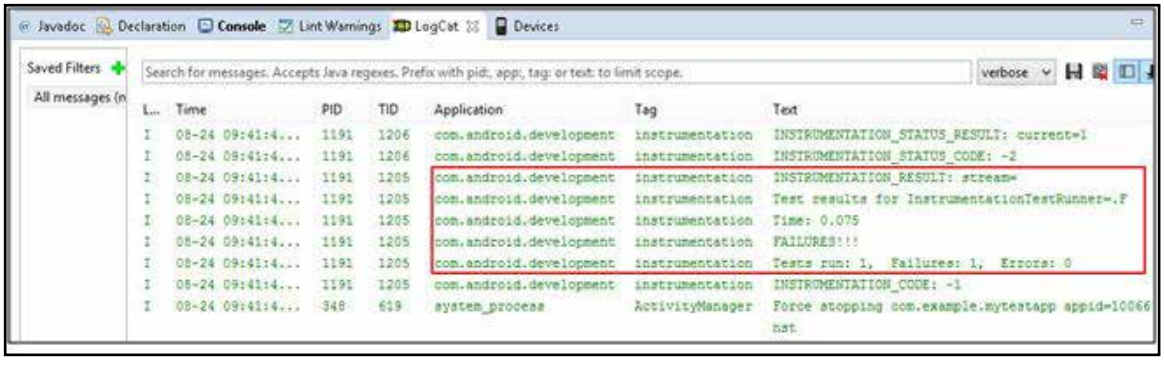


Figure 5.26: Test Result in LogCat View

5.5.3 Using Command Line

There are two ways to run tests from the command line.

- **Running with Ant**

  When a test project is created using the android tool, a run-tests target is created automatically. This target follows all the functions of re-building the main and test project, installing the test application to the current AVD, and then using ANT to run all test classes in the application. The test results are targeted to STDOUT.

  For an existing project to use this feature, use the update test-project option in the android tool.

- **Running with an Android Debug Bridge (ADB) shell**

  To initiate a test run, use the adb shell command on the device or emulator.

  Next, run the am instrument command in the shell or command prompt. Use command-line flags to control am and the tests. The adb shell after opening on the device or emulator and running the tests, produces the output. It then returns to the command line on the system.

  A faster way would be to use the command to start an adb shell, invoke am instrument, and specify command-line flags.

  There are certain advantages of running tests from the command line using adb shell command.

  These are as follows:

  - Provides more options for the tests to run than with any other method.
  - Provides selection of individual methods, helps to filter tests according to their annotation or specifies testing options.

- Customizes testing with shell scripts because it provides complete control from a command line.

Use 'am instrument' command to either run all test classes in the test package or all tests in a test case class. Before, beginning to run any of these tests, there are some preliminary steps that need to be followed. They are as follows:

1. Rebuild the main application and test package, if necessary.
2. Check for existing AVDs. Use the command 'adb devices' as shown in figure 5.27:



Figure 5.27: List of Existing AVDs

3. Launch an existing AVD.
4. Install the test package and the main application .apk files (Android package files) to the Android device or emulator that is to be used. The syntax for the 'am instrument' command for testing the project is as follows:

Syntax:

```
adb shell am instrument -w <test_package_name>/<runner_class>
```

Table 5.2 displays the description of each parameter.

Parameter	Description
<test_package_name>	Android package name of the test application. Refers to the value specified in the package attribute of the manifest element in the manifest file (AndroidManifest.xml) of the test package.
<runner_class>	Android test runner class name that is to be used. Most often used runner class is InstrumentationTestRunner.

Table 5.2: Parameters in Running Tests from Command Line

5. Enter the command as demonstrated in Code Snippet 5 to run all test classes in the test package.

Code Snippet 5:

```
adb shell am instrument -w com.example.mytestapp.test/
android.test.InstrumentationTestRunner
```

Figure 5.28 displays the command and the output of execution of the command at the command prompt.

```
sarathi@sarathi-desktop:~/Documents/sarathibackup/Android/Android/android-studio
/bin$ adb shell am instrument -w com.example.mytestapp.test/android.test.Instrum
entationTestRunner
adb server is out of date.  killing...
* daemon started successfully *

com.example.mytestapp.ApplicationTest:..
com.example.mytestapp.TestCase1:
Failure in testModulo:
junit.framework.AssertionFailedError: expected:<2> but was:<0>
        at com.example.mytestapp.TestCase1.testModulo(TestCase1.java:20)
        at android.test.AndroidTestRunner.runTest(AndroidTestRunner.java:191)
        at android.test.AndroidTestRunner.runTest(AndroidTestRunner.java:176)
        at android.test.InstrumentationTestRunner.onStart(InstrumentationTestRun
ner.java:555)
        at android.app.Instrumentation$InstrumentationThread.run(Instrumentation
.java:1879)

Test results for InstrumentationTestRunner=...F
Time: 0.008

FAILURES!!!
Tests run: 3,  Failures: 1,  Errors: 0


sarathi@sarathi-desktop:~/Documents/sarathibackup/Android/Android/android-studio
/bin$
```

Figure 5.28: Command Line Output

The test shows failure due to AssertionFailedError. The expected output was 2 but the method returned 0.

5.6 How to Debug the Test?

Debug techniques are implemented to remove bugs in tests. Debugging can be as simple as adding messages to LogCat or more complicated. Complex way of debugging is to attach the debugger to a test runner.

To debug a test, perform the following steps:

1. Select the test project.

2. Right-click the project folder and select Debug As → Android JUnit Test as shown in figure 5.29.

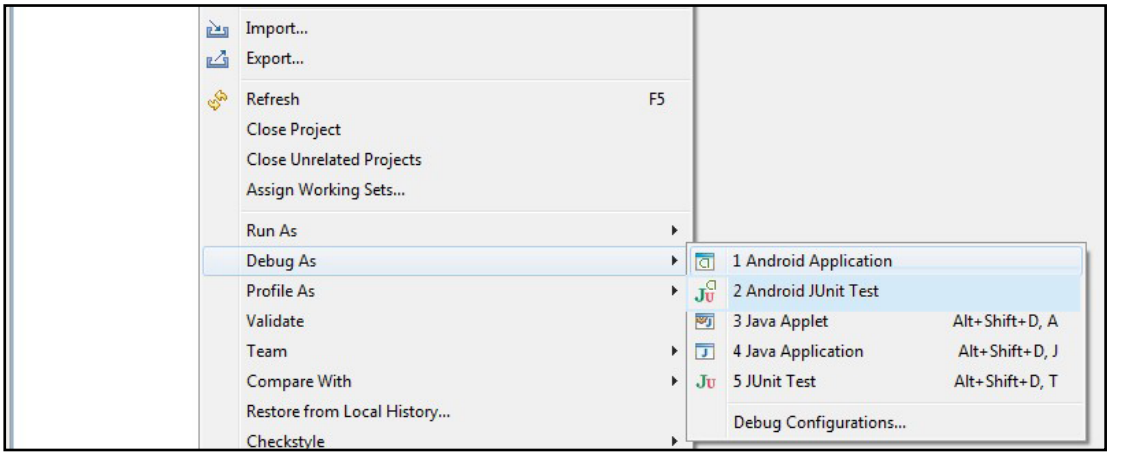


Figure 5.29: Debug a Test

3. Select the desired line in the editor where breakpoint is to be introduced. Here, the breakpoint has been set on line 29 consisting of the call to assertEquals() method.

4. (Optional) Use Run → Toggle Line Breakpoint to toggle a breakpoint as shown in figure 5.30.

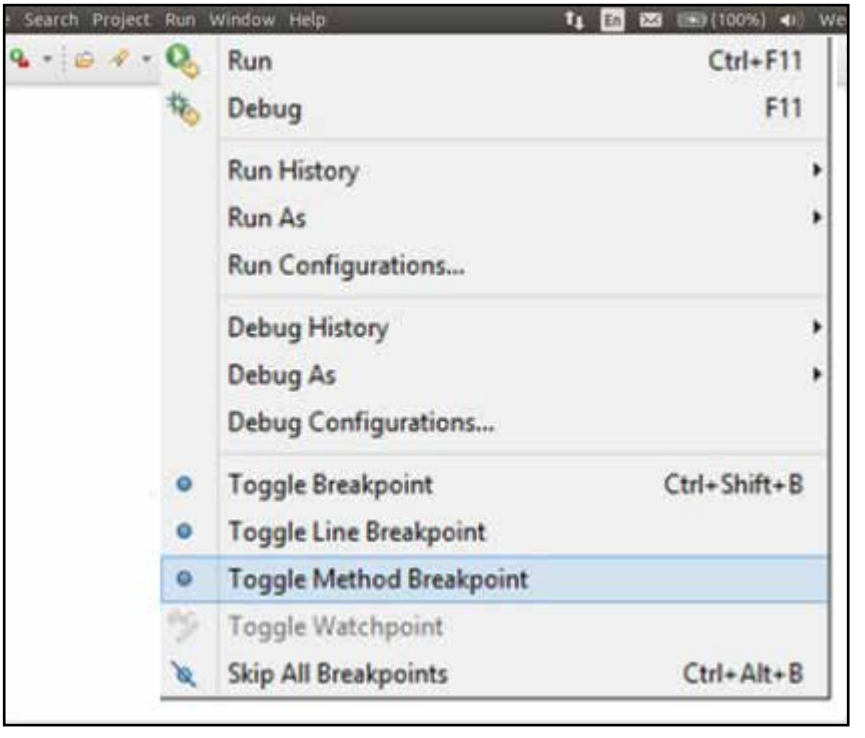


Figure 5.30: Toggle a Breakpoint

5. Modify the test codes as shown in Code Snippet 6 for the debugger connection. Add the code to the constructor at any location or add it to any other test method to debug.

Code Snippet 6:

```
package com.example.mytestapp.test;
import android.os.Debug;
import junit.framework.TestCase;
import com.example.mytestapp.Modulo;
public class TestCase1 extends TestCase {
     private static final boolean DEBUG = false;
     public TestCase1() {
          super();
         // TODO Auto-generated constructor stub
     }
     public TestCase1(String name) {
          super(name);
          // TODO Auto-generated constructor stub
     }
     public void testModulo(){
          if ( DEBUG ) {
                Debug.waitForDebugger();
          }
          int expected=2;
          Modulo m1 = new Modulo();
          assertEquals(expected,m1.getModulo(10, 5));
     }
     protected void setUp() throws Exception {
          super.setUp();
     }
     protected void tearDown() throws Exception {
         super.tearDown();
     }
}
```

6. Start the tests by executing the code as demonstrated in Code Snippet 7 at the command line prompt to debug the tests.

Code Snippet 7:

```
adb shell am instrument -w -e debug true com.example.mytestapp.test/
android.test.InstrumentationTestRunner
```

Figure 5.31 displays the output of the command.

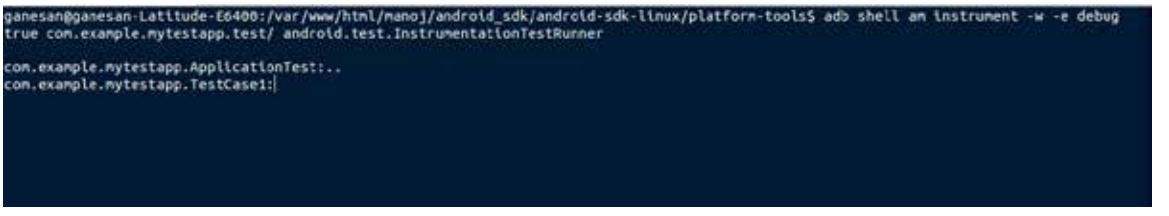


Figure 5.31: Debugging Test from Command Line

The test reaches the break point and halts. The debug window should be opened at this stage to view the debugging process as shown in figure 5.32.

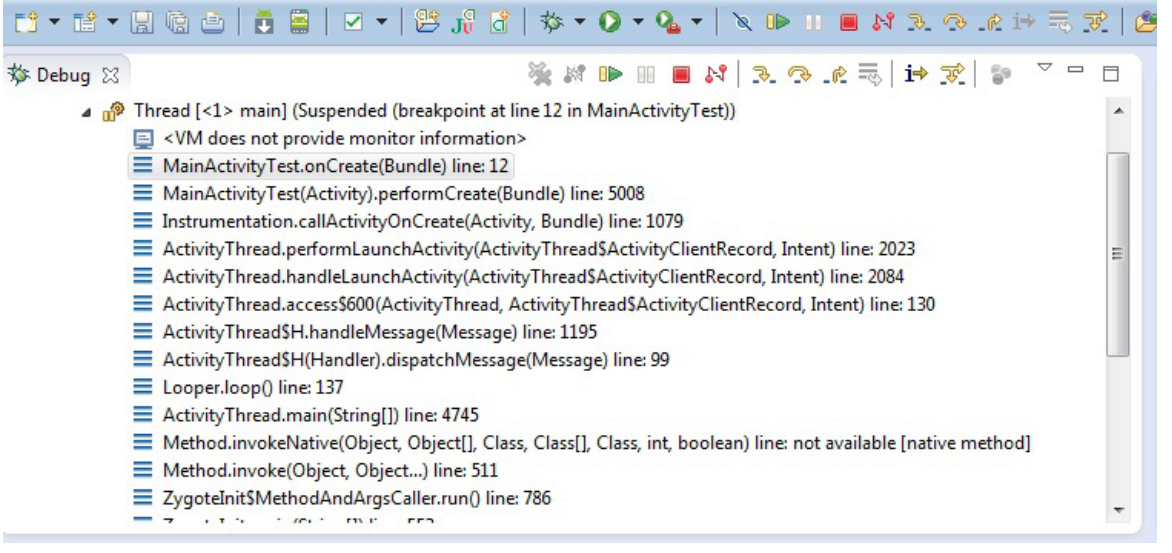


Figure 5.32: Debug Window

The Debug window shows that the test has halted at the breakpoint on line 29. Also, the Variables tab shows the value of expected variable. To begin debugging, click Run → Step Over. The test proceeds to the next line of execution. In case of an exception, the test opens the respective exception class documentation.

The Variables window shows the AssertionFailedError generated by the test as shown in figure 5.33.

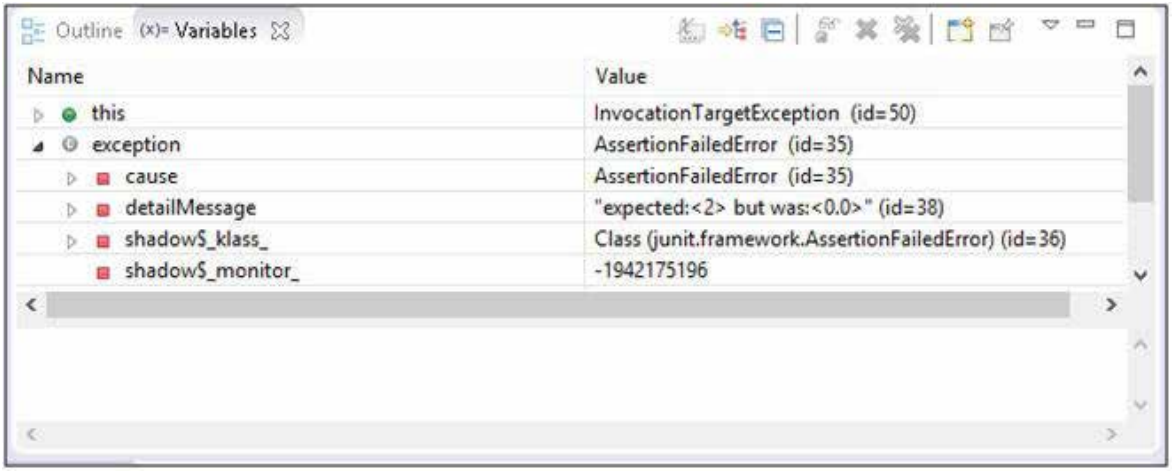


Figure 5.33: Variables Window

On clicking Resume in the Run menu, the test proceeds with execution of remaining lines of code. The output is finally seen in the command window as shown in figure 5.34.

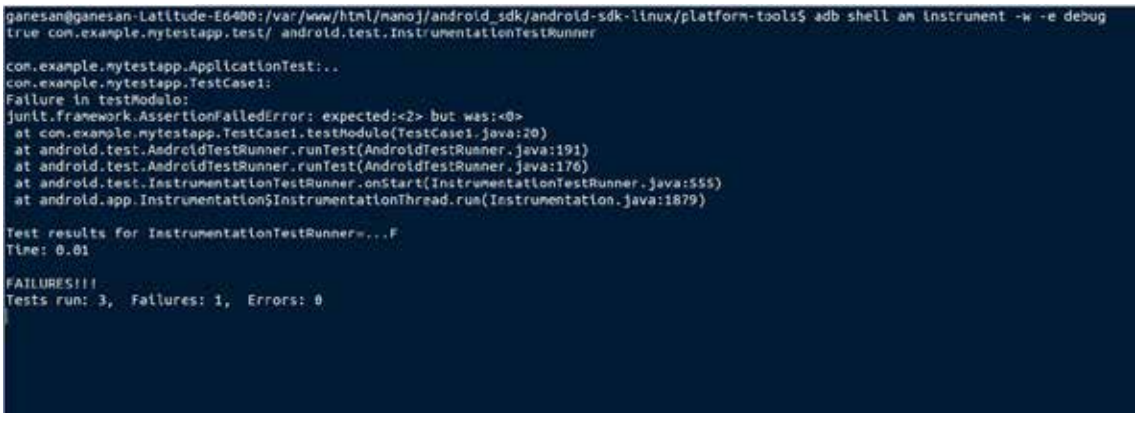


Figure 5.34: Output with Debugger

5.6.1 Example of Currency Converter

To create an Android project named CurrencyConverter, perform the following steps:

1. Create an Android Application Project named CurrencyConverter.

   Activity name: MainActivity

   Layout file: activity_main.xml

   Package: com.example.currencyconverter

2. Modify the code in activity_main.xml file as shown in Code Snippet 8.

Code Snippet 8:

```
<RelativeLayout  xmlns:android="http://schemas.android.com/apk/res/
android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingBottom="@dimen/activity_vertical_margin"
   android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    tools:context="com.example.currencyconverter.MainActivity" >
    <LinearLayout
        android:id="@+id/linearlayoutId"
        android:layout_width="200px"
        android:layout_height="280px"
        android:layout_alignParentTop="true"
        android:orientation="vertical" >
        <TextView
            android:id="@+id/TextView01"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Dollars"
            android:textStyle="bold" >
        </TextView>
       <EditText
           android:id="@+id/dollars"
```

```
            android:layout_width="100px"
           android:layout_height="50px"
            android:lines="1"
            android:numeric="integer" >
        </EditText>
        <TextView
            android:id="@+id/TextView02"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Euros"
            android:textStyle="bold" >
        </TextView>
       <EditText
            android:id="@+id/euros"
            android:layout_width="100px"
            android:layout_height="50px"
            android:lines="1"
            android:numeric="integer" >
        </EditText>
    </LinearLayout>
    <TextView
        android:id="@+id/textView1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignBottom="@+id/linearlayoutId"
        android:layout_alignLeft="@+id/convert"
        android:layout_marginBottom="69dp"
        android:layout_marginLeft="40dp"
        android:text="Result is"
```

```
        android:textSize="14dp"
        android:textStyle="bold" />
    <Button
        android:id="@+id/convert"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignBaseline="@+id/button1"
        android:layout_alignBottom="@+id/button1"
        android:layout_toRightOf="@+id/linearlayoutId"
        android:text="Convert" />
     <Button
        android:id="@+id/button1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignLeft="@+id/linearlayoutId"
        android:layout_centerVertical="true"
        android:layout_marginLeft="24dp"
        android:text="Clear" />
</RelativeLayout>
```

3. Modify the code in MainActivity class as shown in Code Snippet 9.

Code Snippet 9:

```
public class MainActivity extends Activity implements OnClickListener
{
      EditText dollars;
      EditText euros;
      Button convert;
      Button clear;
      TextView result;
      @Override
      protected void onCreate(Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        dollars = (EditText) findViewById(R.id.dollars);
        euros = (EditText) findViewById(R.id.euros);
        clear = (Button) findViewById(R.id.button1);
        convert = (Button) this.findViewById(R.id.convert);
        result = (TextView) findViewById(R.id.textView1);
        convert.setOnClickListener(this);
        clear.setOnClickListener(new OnClickListener() {
              @Override
              public void onClick(View v) {
                    dollars.setText("");
                    euros.setText("");
                    result.setText("");
              }
        });
  }
  @Override
  public void onClick(View arg0) {
        // TODO Auto-generated method stub
        if (dollars.length() > 0) {
              convertDollarsToEuros();
        }
        if (euros.length() > 0) {
              convertEurosToDollars();
        }
        if (dollars.length() > 0 && euros.length() > 0) {
              Toast.makeText(getApplicationContext(), "Please
```

```
fill only one field", Toast.LENGTH_SHORT).show();
            }
      }
      protected void convertEurosToDollars() {
            if (euros.length() > 0) {
                  // TODO Auto-generated method stub
                  double val = Double.parseDouble(euros.getText().
toString());
                  result.setText(Double.toString(val / 0.67));
            } else {
                  Toast.makeText(getApplicationContext(), "Please
enter the value", Toast.LENGTH_SHORT).show();
            }
      }
      protected void convertDollarsToEuros() {
            // TODO Auto-generated method stub
            if (dollars.length() > 0) {
                  double val = Double.parseDouble(dollars.getText().
toString());
                  result.setText(Double.toString(val * 0.67));
            } else {
                  Toast.makeText(getApplicationContext(), "Please
enter the value", Toast.LENGTH_SHORT).show();
            }
      }
      @Override
      public boolean onCreateOptionsMenu(Menu menu) {
            // Inflate the menu; this adds items to the action bar if
it is present.
            getMenuInflater().inflate(R.menu.main, menu);
            return true;
```

```
	}
	@Override
	public boolean onOptionsItemSelected(MenuItem item) {
		// Handle action bar item clicks here. The action bar
will
		// automatically handle clicks on the Home/Up button, so
long
		// as you specify a parent activity in AndroidManifest.
xml.
		int id = item.getItemId();
		if (id == R.id.action_settings) {
			return true;
		}
		return super.onOptionsItemSelected(item);
	}
}
```

4. Right-click the CurrencyConverter project to display the context menu and select Run As → Android Application.

Figure 5.35 displays the output of the application after entering a value in Dollars and clicking Convert.

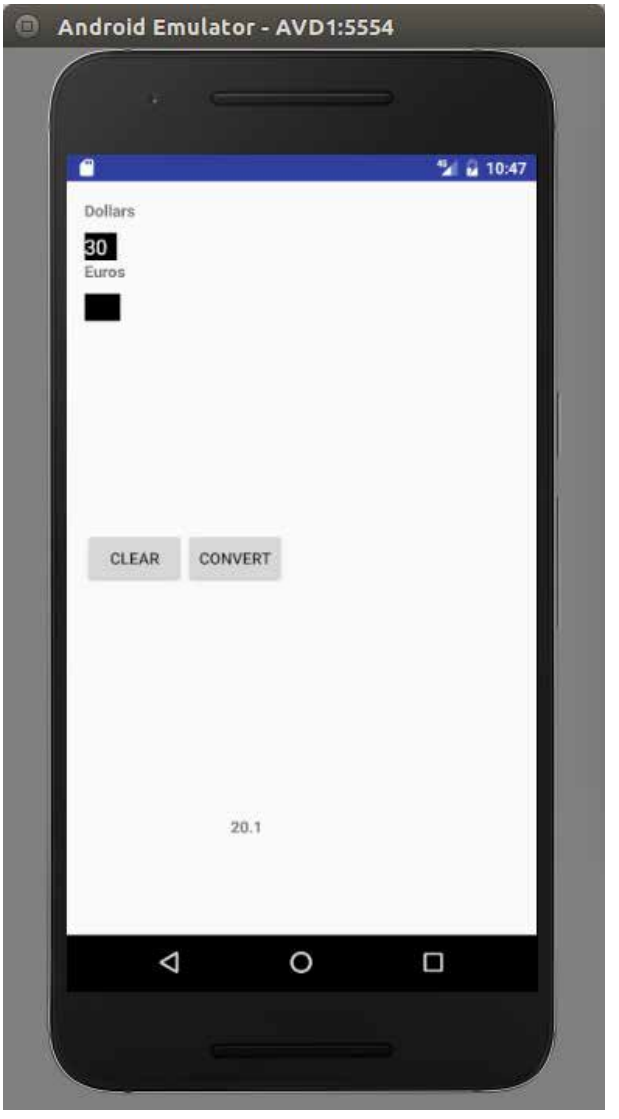


Figure 5.35: CurrencyConverter – Output

To create the Android Test Project for the CurrencyConverter application, perform the following steps:

1. Create an Android Test Project named CurrencyConverterTest.

   Package: com.example.currencyconverter.test

2. Create a test case class named CurrencyConverterActivityTest within the package.

3. Modify the code in the CurrencyConverterActivityTest class as shown in Code Snippet 10.

Code Snippet 10:

```
package com.example.currencyconverter.test;

import com.example.currencyconverter.MainActivity;

import android.test.ActivityInstrumentationTestCase2;

import android.test.TouchUtils;

import android.test.suitebuilder.annotation.SmallTest;

import android.view.KeyEvent;

import android.widget.Button;

import android.widget.EditText;

import android.widget.TextView;

public class CurrencyConverterActivityTest extends
ActivityInstrumentationTestCase2<MainActivity> {

      private MainActivity ConverterInstance;

      EditText dollars;

      EditText euros;

      Button convert;

      TextView result;

      @SuppressWarnings("deprecation")

      public CurrencyConverterActivityTest() {

      ("com.example.currencyconverter", MainActivity.class);

      }

      @Override

      protected void setUp() throws Exception {
```

```
            // TODO Auto-generated method stub
            super.setUp();
            ConverterInstance = (MainActivity) getActivity();
            result = (TextView) ConverterInstance.findViewById(com.
example.currencyconverter.R.id.textView1);
            dollars = (EditText) ConverterInstance.findViewById(com.
example.currencyconverter.R.id.dollars);
            euros = (EditText) ConverterInstance.findViewById(com.
example.currencyconverter.R.id.euros);
            convert = (Button) ConverterInstance.findViewById(com.
example.currencyconverter.R.id.convert);
      }
      /** Testing Conversion Operation */
                  @SmallTest
                  public void testConversion() {
                        String strNum1 = "5";
                        String strNum2 = "5 0";
                        double expected = 3.35;
                  /** Inputting Operand1 */
                  TouchUtils.tapView(this, dollars);
                  sendKeys(strNum1);
                  /** Selects Add operator from the button Widget */
                  TouchUtils.tapView(this, convert);
                  this.sendRepeatedKeys(1,   KeyEvent.KEYCODE_DPAD_
CENTER,  1,  KeyEvent.KEYCODE_DPAD_DOWN,  1,  KeyEvent.KEYCODE_DPAD_
CENTER);
                  /** Performs calculation */
                  try {
                  runTestOnUiThread(new Runnable() {
                        @Override
                        public void run() {
```

```
                        convert.performClick();
                        }
                        });
                        } catch (Throwable e1) {
                        e1.printStackTrace();
                        }
                        /** Getting the result shown */
                        Double actual = Double.parseDouble(result.
getText().toString());
                        assertEquals(expected, actual, 0);
                        }
}
```

4. Select CurrencyConverterTest project and click Run → Run Configuration.
5. Right-click Android Junit Test in the left pane and select New.
   - **Name**: CurrencyConverterTest
   - Select 'Run a single test' option
   - **Project**: CurrencyConverterTest
   - **Test class**: CurrencyConverterActivityTest
   - **Instrumentation runner**: android.test.InstrumentationTestRunner

6. Click Run. Ensure that you unlock the screen of the emulator. Figure 5.36 displays the output of the application.

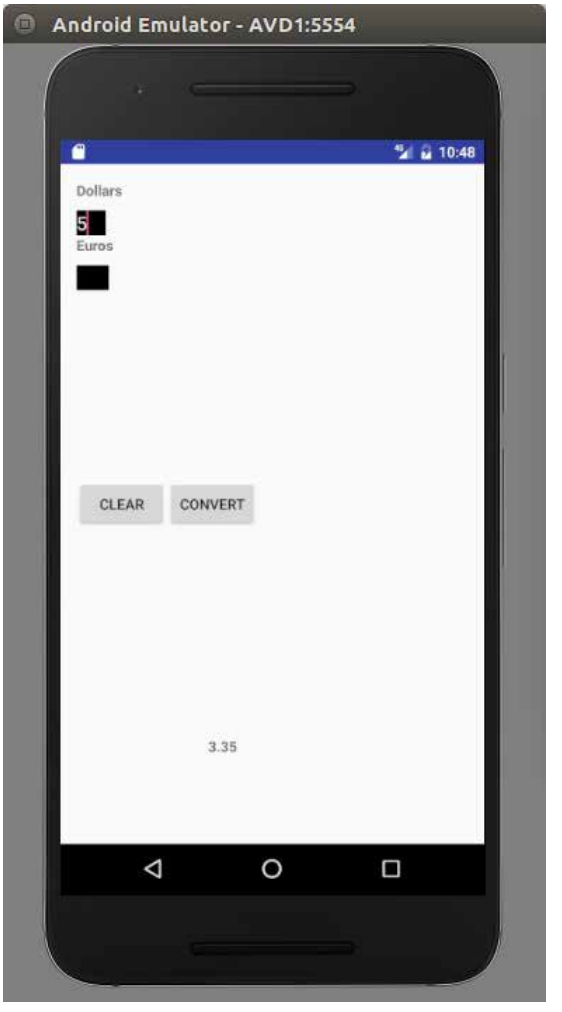


Figure 5.36: Test Run Output

The value 5 is automatically typed in the Dollars field using virtual key pad. The Convert button is clicked and the output is displayed. Since, the converted value is same as the expected value, the test shows success as shown in figure 5.37.

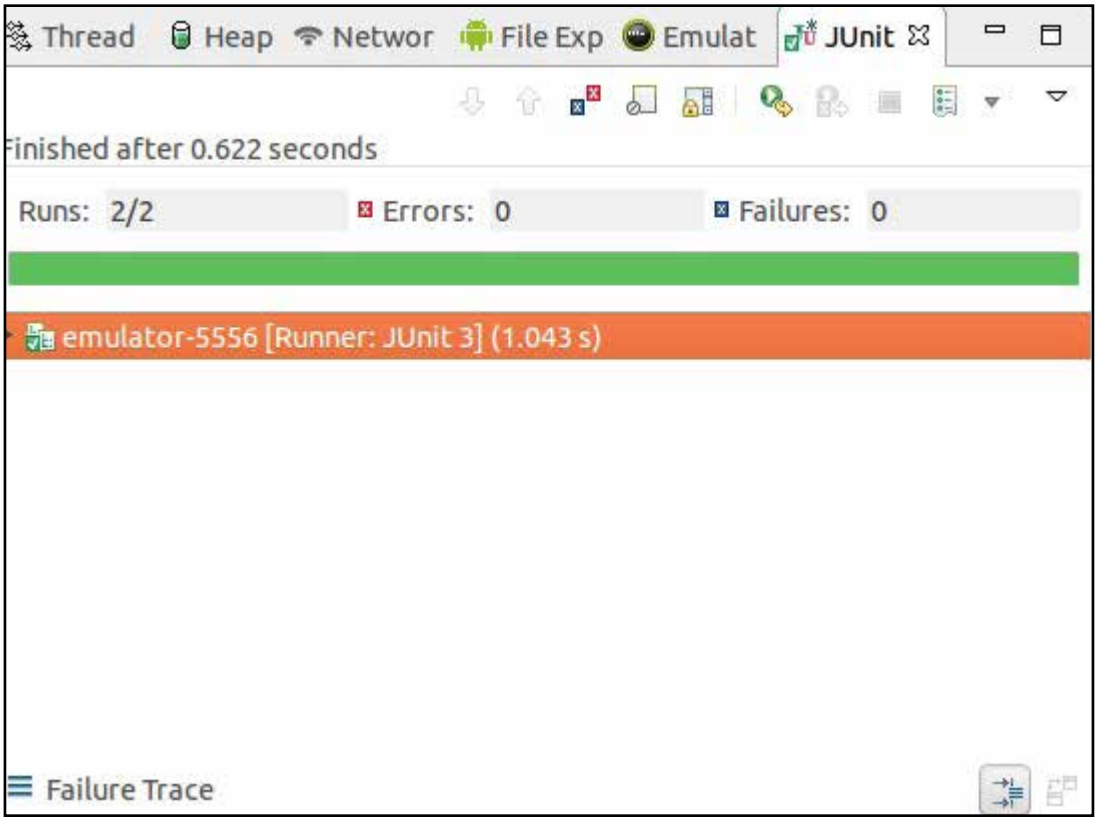


Figure 5.37: Test Result

Check Your Progress

1. Match the following with their parameters.

Keyword		Description	
a.	Application Name	1.	Visible name in Eclipse project directory.
b.	Project Name	2.	Name seen by users.
c.	Package Name	3.	Lowest version of Android supporting the application.
d.	Minimum Required SDK	4.	Highest version of Android tested with the application.
e.	Target SDK	4.	Reverse domain name.

(A)	a-2, b-1, c-5, d-3, e-4	(B)	a-1, b-2, c-3, d-5, e-4
(C)	a-3, b-4, c-5, d-1, e-2	(D)	a-4, b-5, c-3, d-2, e-1

2. Which of the following codes indicates the location where the application is created and saved?

(A)	`android list targets`	(B)	`android create project --target <target-id> --name myTestApp \ --path <path-toworkspace>/TestApp --activity MainActivity \ --package com.myTestApp.Android`
(C)	`$ adb shell am instrument -w \`	(D)	`$ adb shell am instrument -w com.android.demo.app.tests/android.test.InstrumentationTestRunner`

Check Your Progress

3. Where are the test results of running a test package in Eclipse with ADT displayed?

(A)	adb shell	(B)	Eclipse JUnit
(C)	LogCat	(D)	STDOUT

4. Which of the following command is used to check the installation of SDK versions?

(A)	list android targets	(B)	targets android list
(C)	android targets list	(D)	android list targets

5. The _____ target is created automatically using the 'android' tool.

(A)	run-tests	(B)	am instrument
(C)	android.test. InstrumentationTestRunner	(D)	LogCat

Answers

1.	A
2.	B
3.	B
4.	D
5.	A

Summary

- A new Android main project can be created using Eclipse with the ADT plugin or using the SDK tools from a command line.
- A test project quickly created in Eclipse with ADT helps to link the application being tested and automatically inserts the specific <instrumentation> element to the test package manifest file.
- Main test case extends from the Android test case classes or JUnit classes as it provides the best testing features.
- A test can be created, using one of Android's test case classes defined in Android test. To create a test case, the first step is to choose the Java package identifier used for the test case classes and the Android package name.
- Tests can be executed from Eclipse, emulator, or the command line. Eclipse can run the entire package or a single test at a time.
- Instrumentation is the only means to run tests from the emulator.
- Tests from the command line are executed using Ant or with an Android Debug Bridge (ADB) shell.
- Debugging can be as simple as adding messages to LogCat.

LEARN BETTER @ Onlinevarsity



06 Testing for Different Situations



Welcome to the Session, **Testing for Different Situations**.

This session briefly describes the process of performing testing under different situations. Whenever you work on an application, you may encounter various situations that need to be tested. Each situation needs to be tested in a specific way. Therefore, this session introduces few such situations and also provides necessary guidelines required to test those situations.

In this Session, you will learn to:

- Describe different levels of testing units
- Explain testing an Android application
- Describe testing of file system and database
- Explain testing for exceptions
- Identify automated testing for UI
- Explain testing for memory leaks
- List the checklist for testing accessibility

Session

06

Testing for Different Situations

6.1 Introduction

This session will provide a brief introduction to the different tests that might be required to be performed to ensure that the application works correctly. There will be few different areas under which a particular type of test may have to be performed.

For example, to ensure that the user is able to access all the different capabilities of an application, the developer must test the application under varying conditions. Testing the application in varying situations ensures that it functions properly in all possible situations.

6.2 Testing Units in Android

Unit testing refers to testing individual parts of an application. While working with an Android application, sometimes it becomes essential to individually test the components of the application without disturbing the underlying system. To implement this type of testing, an Android application can be tested at three different levels. They are as follows:

- Testing Activities
- Testing Content Providers
- Testing Services

6.2.1 Testing Activities

An application uses ActivityUnitTestCase<Activity> base class to perform isolated testing of a single Activity. This kind of testing is primarily done to test the general behavior of an Activity class and also to provide flexibility and control over the Activity under test.

Here, the activity to be tested is created with minimal number of connections to the underlying system. Care is also taken that the Activity will not interact with the other activities present in the system. In such cases, it may be required to add Mock Objects and wrapped versions of the dependents of the Activity class. The ActivityInstrumentationTestCase2 class defined in the package android.test is used to send mock Intents to the activity under test.

An application uses the ActivityInstrumentationTestCase2 to test the functions of a single activity. In contrast to isolated testing of a single Activity, here the activity to be tested is created invoking the InstrumentatonTestCase.lauchActivity() method of the system infrastructure. In this, it is possible to directly manipulate the activity.

However, in both circumstances, the setUp() and the tearDown() methods are called to automatically handle the work of an Activity. The methods must be overridden by extending the TestCase class in the testing class.

Following validation checks can be performed on the Activity which is to be tested:

- **Validation of Input**: An activity can be validated to check whether it responds correctly to input values in an EditText View. Perform the following steps to test the validity of the input values of an activity:

  1. Set up a keystroke sequence.
  2. Send it to the activity being tested.
  3. Use findViewById(int) method to examine the state of the View.

  To create the EditText by its id, the developer can perform the following steps:

  - Navigate to the res/layout folder in the android application.
  - Create the xml layout file by right-clicking the layout folder and selecting New → Android XML file or opening the existing layout xml file.
  - Open the layout xml file in graphical view.
  - Drag and drop the EditText view which is present in the Text Fields section of the Palette. The code generated will be as demonstrated in Code Snippet 1.

  Code Snippet 1:

```
<EditText
android:id="@+id/editText1"
android:layout_width="wrap_content"
android:layout_height="wrap_content" >
</EditText>
```

  Then, in the Activity file, the view object can be examined for its state by obtaining a reference to it using the method findViewById(R.id.editText1).

  4. To check whether a valid keystroke sequence has been performed, the developer can create an OK button which will be enabled or disabled depending on the action performed. An invalid keystroke will send an error message which is visible in the View.

- **Lifecycle Events**: Lifecycle events trigger the onCreate() or onClick() callback methods either from the system or from the user. In other words, these are actions generated by the system or the user. When testing the activities of an application, check whether the lifecycle events are handled correctly by the activity being tested.
- **Intents**: Each activity should correctly handle the intents which are listed in the intent filter. These intent filters are in turn specified in its AndroidManifest.xml file.

- **Runtime Configurations**: These indicate that each activity is tested to show the desired output for all the possible changes that are performed in the device's configuration while your application is running. The changes can include a change to the device's orientation, a change to the current language, and so forth. The main goal is that any change in the device's configuration while the application is running should not affect the working of an activity.
- **Screen Sizes and Resolutions**: To enable the application to be available on different screen sizes and densities, the application needs to be tested on multiple sizes and densities using AVDs or testing it directly on the target device such as real time android phones.

6.2.2 Testing Content Providers

Content providers are considered to be the main building blocks of the Android system since, they can be viewed as data APIs that provide tables of data. The internals of these tables of data are hidden from view. These are generally responsible for guarding the data in an application from view. A content provider may have many public constants, but no public variable. It may have a few public methods, if any. Thus, tests are written based only on provider's public members. The content providers are mainly used for sharing of data between one or more applications.

The ProviderTestCase2 class that is defined in the package android.test is the base class for testing content providers. It allows testing of the content provider in an isolated environment. Use the ProviderTestCase2 class to invoke the MockContentResolver for testing the content provider. The MockContentResolver initiates testing by making queries to the content provider. The setUp() and tearDown() methods are executed to start the initialization process before each test. Assertions are used to check retrieval of correct data from the content provider.

Following are a few specific guidelines for testing content providers:

- Use a resolver object with the appropriate URI to test the provider. This guarantees that the provider is tested against the same interaction that an application would use.
- Test it as a contract to ensure that the provider is public and available to other applications. Ensure that testing is done for all valid URIs offered by the provider as well as invalid URIs. Also, test with constants that the provider publicly displays.
- Verify that all access methods namely query, insert, delete, update, getType, and onCreate() work correctly.
- In general, a content provider most likely does not have business logic because activities that modify the data could be implementing them. However, if the provider handles business logic such as invalid values, arithmetic calculations, and so on then they are to be tested.

6.2.3 Testing Services

The service objects in an Android system have a separate testing framework. For testing application permissions and for controlling the application and service that is tested, Android uses the ServiceTestCase class. It initiates the testing process only with the assurance that Mock Objects are available for the tests. For this, it invokes the ServiceTestCase.startService() or ServiceTestCase.bindService() method. Service is a very important component in Android.

At the start of each test, the setUp() method for ServiceTestCase class is invoked to set up the test fixture. It makes a copy of the current system Context before being inspected by any of the test methods. The methods setApplication() and setContext(Context) are responsible for setting up a mock Context or mock Application (or both) for the service. Such setting isolates the test from the rest of the system.

By default, the test method named testAndroidTestCaseSetupProperly() is executed by ServiceTestCase class. This method checks whether the base test case class has successfully set up a Context before starting. Follow these specific guidelines when testing services:

- Ensure that the methods listed in table 6.1 are invoked in response to their triggers.

Methods Invoked	Trigger Methods
`onCreate()`	`Context.startService()or Context.bindService()`
`onDestroy()`	`Context.stopService(),Context.unbindService(), stopSelf(), or stopSelfResult()`

Table 6.1: Methods Invoked while Testing Services

- With reference to calls, check whether the service correctly handles calls from Context.startService() method as listed in table 6.2.

Number of Calls	Trigger Response
First call	`Service.onCreate()`
All calls	`Service.onStartCommand()`
Single call	`Context.stopService() or Service. stopSelf() stops service under test`

Table 6.2: Trigger Responses Invoked while Testing Services

- Test the business logic present in the service class which can be checked for invalid values, financial calculations, and so on.

6.3 Testing Application in Android

The base class of an Android application maintains the global state of the application whenever it deals with shared preferences. This action is undertaken with the assumption that the behavior of the real application remains unchanged though there may be some alteration in the preference values while testing.

A way to overcome this assumption is to mock a Context that in turn mocks access to the SharedPreferences. The RenamingDelegatingContext class though mocks database and files system access is of not much use here. So, to mock the SharedPreferences access, a specialized mock Context needs to be created.

Mock Context is used to inject other dependencies as well as mock or monitor other classes that are tested. All methods are non-functional and throw UnsupportedOperationException. Following example creates an empty mock Context that delegates to another Context, the RenamingMockContext. The new mock Context that is created uses a renaming prefix.

To develop the application, perform the following steps:

1. Start Eclipse.
2. Create a new project named MyApplication with the Activity name set to MockContextExampleActivity.
3. Navigate to src → com.example.myapplication folder.
4. Modify the code in MockContextExampleActivity file as demonstrated in Code Snippet 2.

Code Snippet 2:

```
package com.example.myapplication;
import android.app.Activity;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import java.io.BufferedReader;
import java.io.File;
import java.io.FileInputStream;
import java.io.InputStreamReader;
import android.annotation.SuppressLint;
import android.graphics.Color;
import android.widget.TextView;
public class MockContextExampleActivity extends Activity {
```

```
      @SuppressLint("SdCardPath")
      public final static String FILE_NAME = "/sdcard/myfile.
txt";
      private TextView tv;
            @Override
      protected void onCreate(Bundle savedInstanceState) {
            super.onCreate(savedInstanceState);
            setContentView(R.layout.activity_mock_context_
example);
            tv = (TextView) findViewById(R.id.textdisplay);
            final byte[] buffer = new byte[1024];
            try {
            File myFile = new File("/sdcard/data.txt");
            FileInputStream fIn = new
FileInputStream(myFile);
            BufferedReader myReader = new BufferedReader(new
            InputStreamReader(fIn));
            String aDataRow = "";
            String aBuffer = "";
            while ((aDataRow = myReader.readLine()) != null)
{
            aBuffer += aDataRow + "\n";
            }
            tv.setText(aBuffer);
            myReader.close();
            } catch (Exception e) {
            tv.setText(e.toString());
            tv.setTextColor(Color.RED);
            }
      }
```

```
	public String getText() {
		return tv.getText().toString();
		}
	@Override
	public boolean onCreateOptionsMenu(Menu menu) {
		// Inflate the menu; this adds items to the action
bar if it is present.
		getMenuInflater().inflate(R.menu.mock_context_
example, menu);
		return true;
	}
	@Override
	public boolean onOptionsItemSelected(MenuItem item) {
		// Handle action bar item clicks here. The action
bar will
		// automatically handle clicks on the Home/Up
button, so long
		// as you specify a parent activity in
AndroidManifest.xml.
		int id = item.getItemId();
		if (id == R.id.action_settings) {
			return true;
		}
		return super.onOptionsItemSelected(item);
	}
}
```

5. Right-click the com.example.myapplication folder to display the context menu.
6. Select New → Class to create a new class named MyMockContext.java.

7. Modify the code as demonstrated in Code Snippet 3.

Code Snippet 3:

```
package com.example.myapplication;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import android.content.Context;
import android.test.RenamingDelegatingContext;
import android.util.Log;
public class MyMockContext extends RenamingDelegatingContext
{
      private static final String TAG = "MyMockContext";
      private static final String MOCK_FILE_PREFIX = "test.";
      public MyMockContext(Context context) {
            super(context, MOCK_FILE_PREFIX);
            makeExistingFilesAndDbsAccessible();
    }
    /*
     * (non-Javadoc)
     *
     * @see android.test.
RenamingDelegatingContext#openFileInput(
     * java.lang.String)
     */
     @Override
     public FileInputStream openFileInput(String name)
throws FileNotFoundException {
           Log.d(TAG, "actual location of " + name + " is "
+ getFileStreamPath(name));
      return super.openFileInput(name);
```

```
        }
        @Override
        public FileOutputStream openFileOutput(String name, int
mode) throws FileNotFoundException {
            Log.d(TAG, "actual location of " + name + " is " +
getFileStreamPath(name));
            return super.openFileOutput(name, mode);
        }
}
```

8. Navigate and open AndroidManifest.xml file.

9. Modify the code as demonstrated in Code Snippet 4.

Code Snippet 4:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/
android"
    package="com.example.myapplication"
    android:versionCode="1"
    android:versionName="1.0" >
    <uses-sdk
        android:minSdkVersion="21"
         android:targetSdkVersion="21" />
   <uses-permission android:name="android.permission.READ_
EXTERNAL_STORAGE" />
   <uses-permission android:name="android.permission.WRITE_
EXTERNAL_STORAGE" />
   <application
         android:allowBackup="true"
        android:icon="@drawable/ic_launcher"
        android:label="@string/app_name"
         android:theme="@style/AppTheme" >
```

```
        <activity
            android:name=".MockContextExampleActivity"
            android:label="@string/app_name" >
            <intent-filter>
                <action android:name="android.intent.action.
MAIN" />

                <category android:name="android.intent.
category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

10. Navigate to res → layout folder.

11. Modify the code in activity_mock_context_example.xml file as demonstrated in Code Snippet 5.

Code Snippet 5:

```
<RelativeLayout xmlns:android="http://schemas.android.com/
apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingBottom="@dimen/activity_vertical_margin"
   android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
tools:context="com.example.myapplication.
MockContextExampleActivity" >
    <TextView
       android:id="@+id/textdisplay"
```

```
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/hello_world" />
</RelativeLayout>
```

12. Select Window → Android Virtual Device Manager to display the Android Virtual Device Manager dialog box.
13. Select an existing AVD or create a new one.
14. Click Start to display the Launch dialog box and click Launch. The emulator starts after some time.
15. Create a text file named data.txt with the content as shown in figure 6.1.

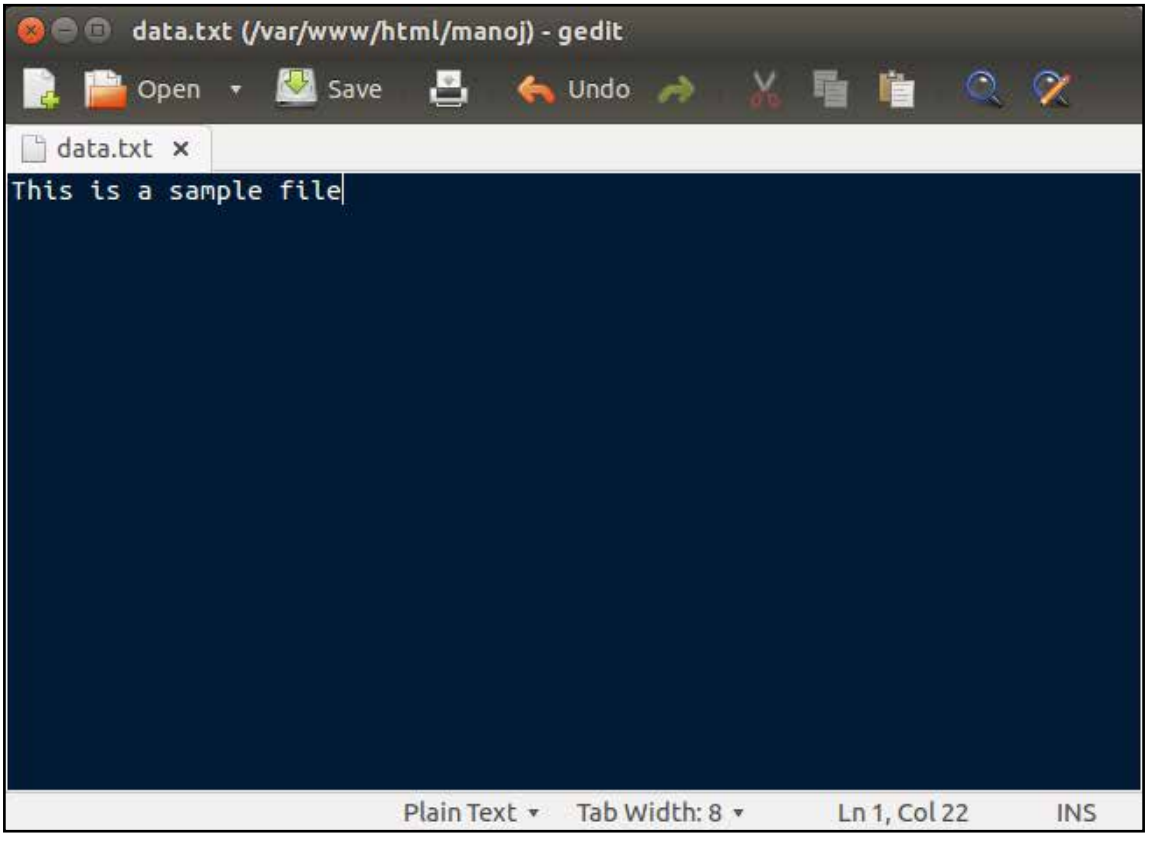


Figure 6.1: Sample Text File

16. Once the emulator has started, move to DDMS perspective.
17. Click File Explorer tab in DDMS perspective (Window → Perspective → Open Perspective → DDMS)

18. Select the emulator from the Device pane on the left.

19. Select sdcard from File Explorer tab on the right pane.

20. Click Push a file onto the device icon to display the Put File on Device dialog box as shown in figure 6.2.

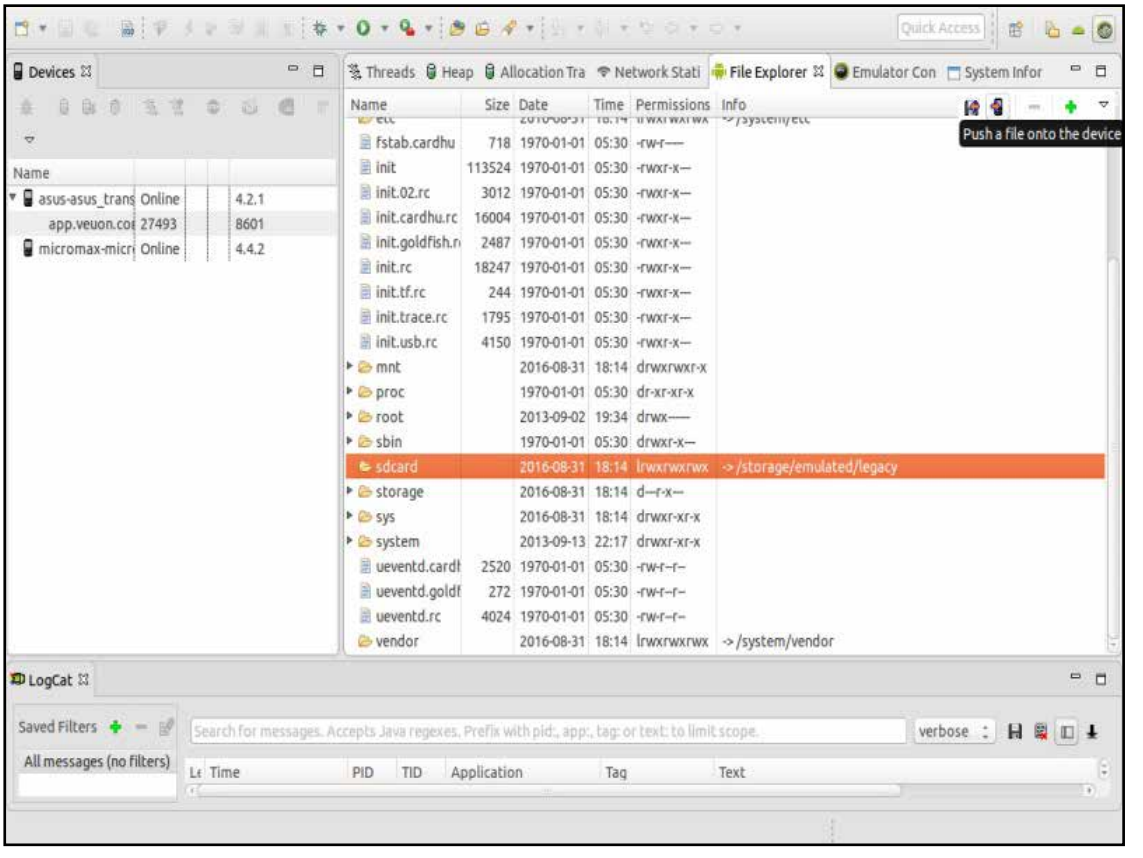


Figure 6.2: Push Icon

21. Navigate to the location of the data.txt and select it.

22. Click Open. The file will be placed in the sdcard.

Note - In case the error 'Failed to push selection: Read-only file system' is shown, perform any of the following steps to fix the issue:

- In the AVD Manager, select the device and click Edit and enter a value in the SD Card Size box (For example, 200 MB)
- Go to C:\Documents and Settings\<username>\.android\avd and select the folder of your AVD. Each AVD has its own folder. Right-click the appropriate AVD folder and uncheck Read-only checkbox. Click OK.
- Go to your ADV folder in .android folder and find your AVD config.ini file.

  Open it in a text editor such as Notepad.

  Find and change hw.sdCard=no to hw.sdCard=yes. If this attribute is not present, add it to the file and save it.
- On the command prompt, go to Android SDK/platform-tools and type the following commands:

```
adb shell
$su
mount -o remount, rw /sdcard
chmod 777 /mnt/sdcard
exit
```

23. Switch to Java perspective and right-click the application.

24. Click Run As → Android Application to display the output as shown in figure 6.3.



Figure 6.3: File Contents Displayed in the Emulator

To create the Android Test Project for the application created named MyApplication, perform the following steps:

1. Create an Android Test Project named MyApplicationTest.

   Package: com.example.myapplicationtest.test

2. Create a test case class named MockContextExampleTests within the package.

3. Modify the code in the test case class as demonstrated in Code Snippet 6.

Code Snippet 6:

```
package com.example.myapplication.test;

import com.example.myapplication.MockContextExampleActivity;

import com.example.myapplication.MyMockContext;

import android.content.Context;

import android.content.Intent;

import android.test.ActivityUnitTestCase;

public class MockContextExampleTests extends
ActivityUnitTestCase<MockContextExampleActivity> {

     public MockContextExampleTests() {

           super(MockContextExampleActivity.class);

 }

      protected void setUp() throws Exception {

      }

      protected void tearDown() throws Exception {

      }

      public void testSampleTextDisplayed() {

           // mock context

          Context mockContext = new

MyMockContext(getInstrumentation().getTargetContext());

          setActivityContext(mockContext);

          startActivity(new Intent(), null, null);

          final MockContextExampleActivity activity =

getActivity();
```

```
        assertNotNull(activity);
        assertFalse("This is an Example".equals(activity.
getText()));
    }
    public void testRealTextDisplayed() {
        // real context
        setActivityContext(getInstrumentation().
getTargetContext());
        startActivity(new Intent(), null, null);
        final MockContextExampleActivity activity =
getActivity();
        assertNotNull(activity);
        assertFalse("This is an Example".equals(activity.
getText()));
    }
}
```

The testSampleTextDisplayed() uses the mock context whereas the testRealTextDisplayed() method uses the actual context of the activity to be tested.

Once the code is executed using JUnit Framework, the output will be as shown in figure 6.4. Ensure that the screen of the emulator is unlocked for the test to run.

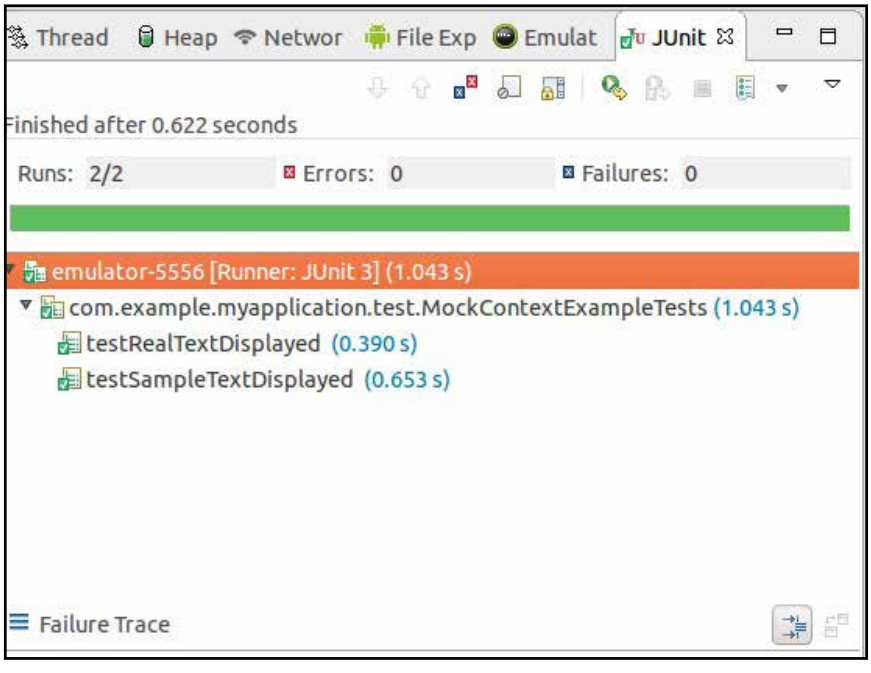


Figure 6.4: Test Result

The test result shows success for both tests; one using mock context and other using the real context. Note that the Assert method used was assertFalse().

This method asserts that the condition is false. The statement mentioned in the method was 'This is an Example'. Since the text file contains the different text, the assertFalse() method returned true since the statements did not match. Therefore, the assertion that the condition is false is proved and so the test shows success.

6.4 Testing Filesystem and Database in Android

Database and filesystem operations are tested so as to check whether they function correctly. Sometimes, when test cases exercise databases or content provider operations, it may be wise to test these operations either at a lower level in isolation or at a higher level through ContentProviders or from the application itself. For this, Android mocks such operations to keep the Activity class under test unaltered.

Consider the following examples:

- Testing an application on a real device such as changed values might be shared by more than one application. Here, testing needs to be isolated (separated) so that it does not affect other normal operations performed on the device.
- Testing an activity where some files and databases need to be controlled to introduce a specialized content. The specialized content is required to perform and drive the tests, but cannot be used in real files or database.

  Both these cases can be solved using the RenamingDelegatingContext. This is a mock class that is a part of the android.test package.

  Its function is to mock files and database operations. This is done by adding a prefix to the target file and database operations. The prefix is supplied in the constructor. Also, all other operations that are delegated to the delegating Context need to be specified in the constructor. It basically allows the developer to replace the context used when invoking the file system methods by re-directing the call to a mocked data file instead of the real file.

6.5 Testing for Exceptions in Android

During the execution of a test, some conditions or methods need to be tested for exceptions and wrong values. In Android, the assert method compares the expected and actual test results.

It throws up exceptions when there is a difference between results. JUnit provides a JUnit class named ExpectedExceptions that can be used to test an exception that is expected.

6.6 Testing UIs in Android

It is important to test the User Interface (UI) of an application to ensure that the UI functions correctly. It needs to respond and provide the correct output in response to the user's actions. It can be a sequence of actions on the device which the user desires such as selecting menus, responding to dialogs, and so on.

Android has a built-in automated UI testing tool. The tool consists of the following two main components:

- **uiautomatorviewer GUI tool**: which scans and analyzes the GUI components of the application.
- **uiautomator Java library**: which provides the necessary APIs for automated, customized UI tests, and an execution engine to automate and run them.

These tools are available only if the following versions of the Android development tools are installed:

- Android SDK tools, Revision 23 or higher.
- Android SDK platform, API 18 or higher.

To check the platform availability in your android sdk, go to the toolbar in Eclipse and click the Android SDK Manager icon. Then, one can check in the Android SDK Manager windows whether the correct platform has been installed in the status column. If not installed then select it and click install. It would take some time for it to be installed but it would be installed.

For automated testing of the UI, perform the following:

1. Ensure that the preliminary tasks are performed before the test can start. Such as:
   a. Installation of the application on a test device.
   b. Analysis of the UI components of the application.
   c. Checking the accessibility of the application by the test automation framework.
2. Create automated tests. These tests should simulate the specific user interaction of the application.
3. Compile test cases into a JAR file.
4. Install the file along with the application on the test device.
5. Run the tests.
6. View the test results.
7. Perform the bug fixing if any bugs are encountered.

6.7 Testing for Memory Leaks in Android

Checking the memory consumption by an application is important to test the working of an application which is often referred to as the test target. Before checking for this, it is necessary to detect the memory used by different classes.

To check for memory usage in Android, perform the following steps:

1. Navigate to Window → Perspective → Open Perspective → DDMS as shown in figure 6.5.

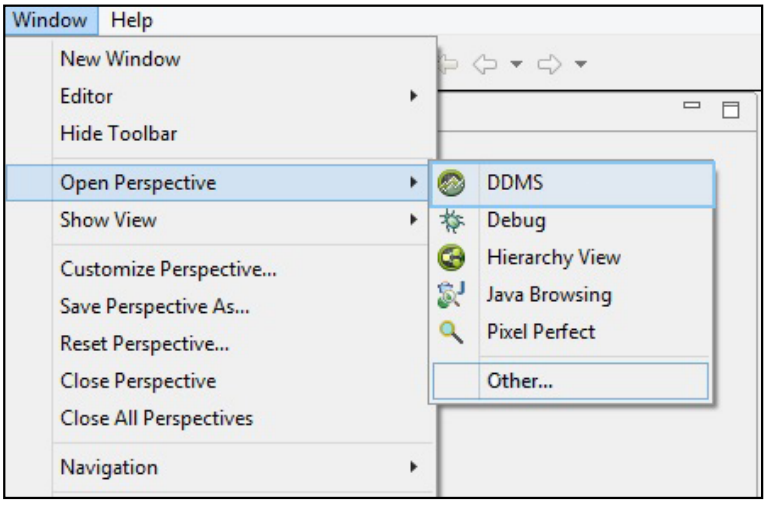


Figure 6.5: Navigating to Other Perspective

In case DDMS is not visible, click Other and then select DDMS from Open Perspective dialog box as shown in figure 6.6.

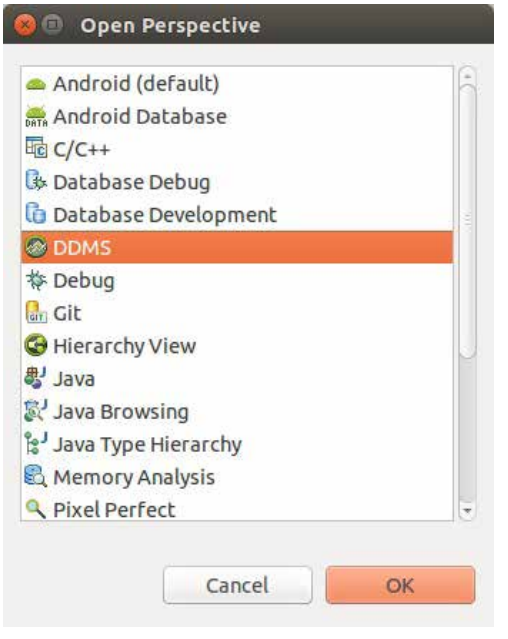


Figure 6.6: Other Perspective Dialog

Another way to navigate to DDMS perspective is by selecting the icon next to Java in the top right corner of the toolbar in Eclipse. Once the icon is clicked, the Open Perspective dialog box is displayed and the user can click DDMS. After selecting DDMS, the user clicks OK to display the DDMS perspective.

2. Click OK to display the DDMS perspective as shown in figure 6.7.

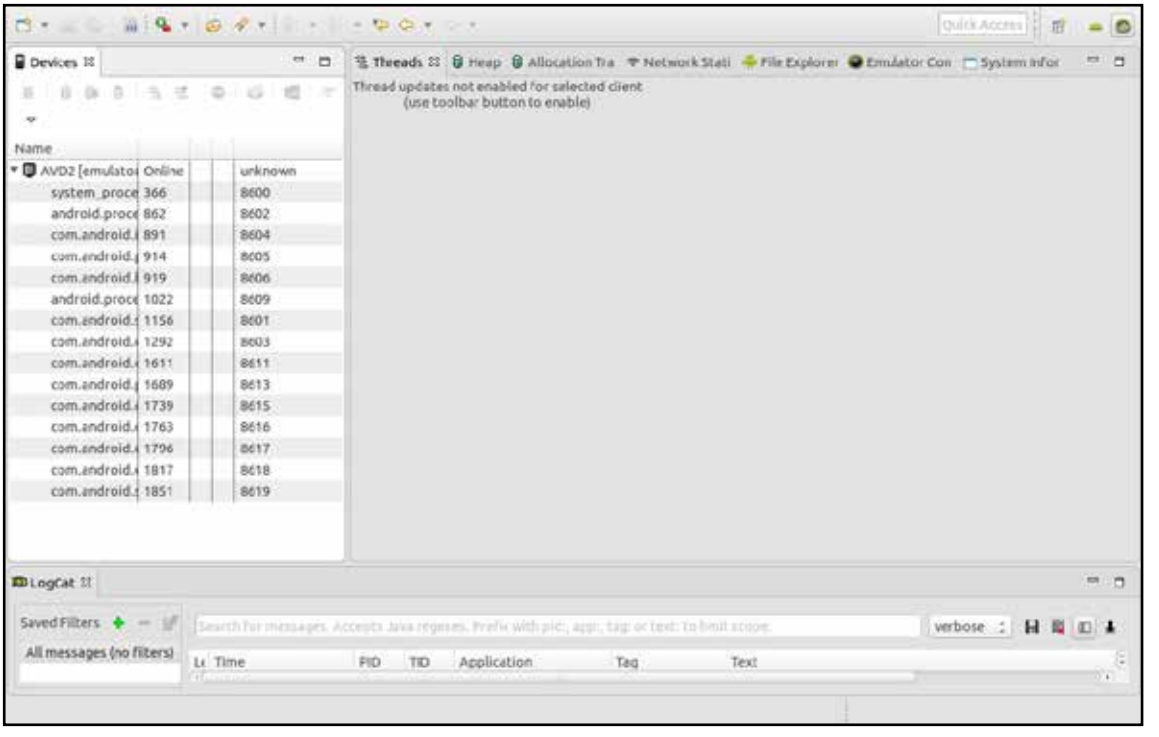


Figure 6.7: DDMS Perspective

3. Start an application in an AVD.

4. Select the required process name in the left panel.

5. Click Update heap icon from left pane and select Heap tab from the right pane as shown in figure 6.8.

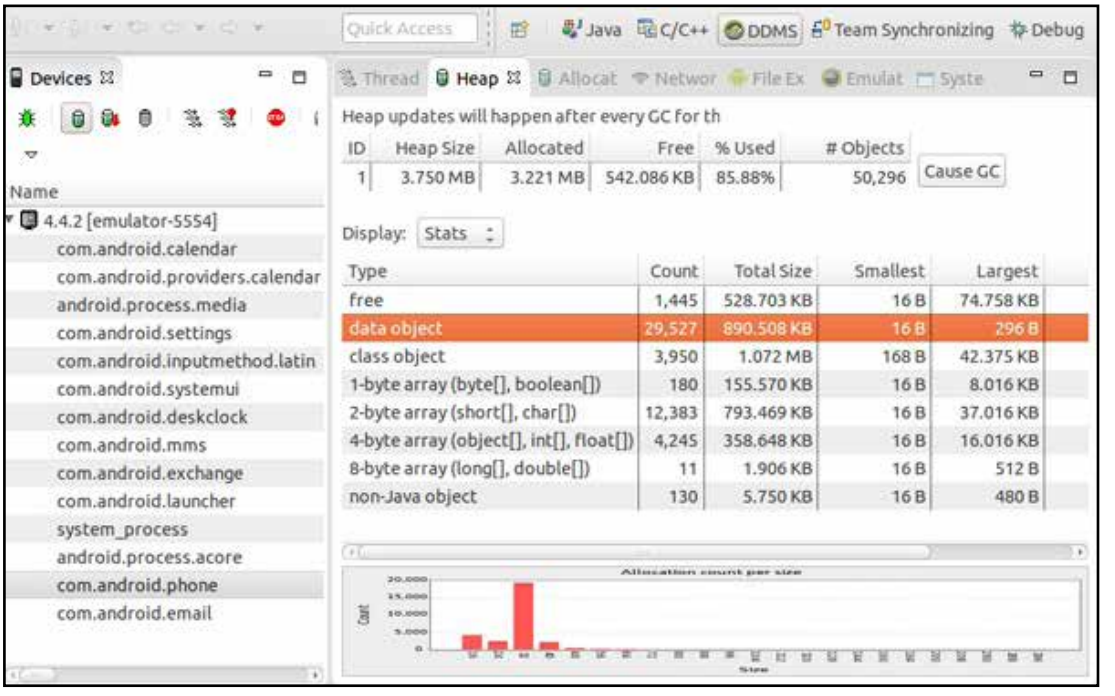


Figure 6.8: Selecting Heap

If the heap update is not visible, click the Cause GC icon to force garbage collection and then click Update Heap icon again.

6. Click Dump HPROF file as shown in figure 6.9.

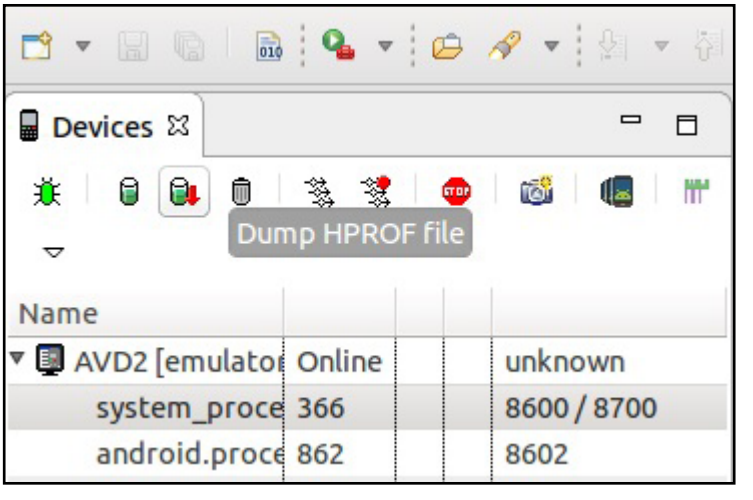


Figure 6.9: Dumping HPROF File

7. Save in desktop or any other drive.

8. Open the command prompt or Ubuntu terminal.

9. Set the path to SDK tools folder.

10. Type the following command:

```
cd C:\android-sdk\tools (or whichever directory that contains your Android sdk)
```

```
or
```

Set the path of Android sdk tools directory in the command prompt using the path command.

11. Assuming that the Dalvik file is present on the Desktop, type in the following command:

```
hprof-conv "C:\Users\Desktop\dumpfile.hprof" "C:\Users\Desktop\4mat.hprof"
```

Note -

- The hprof-conv tool converts the HPROF file that is generated by the Android SDK tools to a standard format so that one can view the file in a profiling tool of your choice.
- The hprof-conv.exe may be moved to Android SDK/platform-tools folder also. So, set the path accordingly.
- If it still does not work, copy the hprof-conv.exe file to the folder where your dump hprof file is saved (For example, D:\ drive).

Figure 6.10 shows the command entered in the command window.

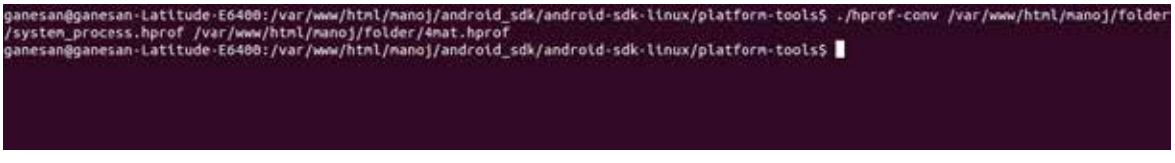


Figure 6.10: Convert Standard Format HRROF File

This will generate the standard format of HPROF file that is named as 4mat.hprof as shown in figure 6.11.

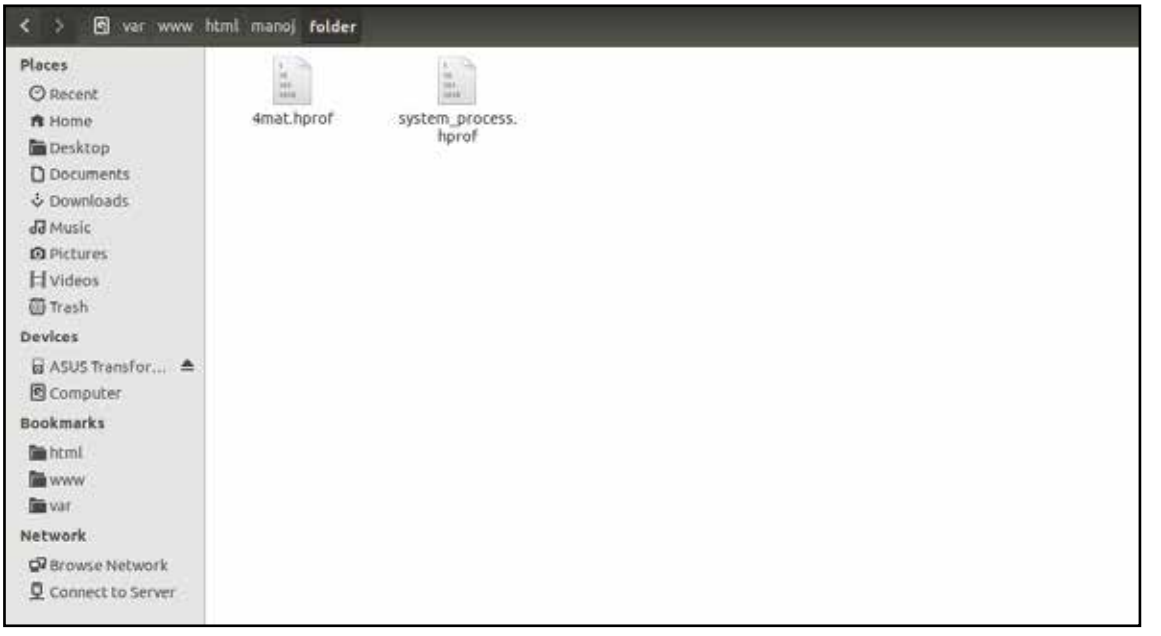


Figure 6.11: Converted File

12. Install Eclipse MAT plugin from the site http://www.eclipse.org/mat/downloads.php.

    Update Site: http://download.eclipse.org/mat/1.5/update-site/

13. Click Help → Install new Software.

14. Enter the link and select all items as shown in figure 6.12.

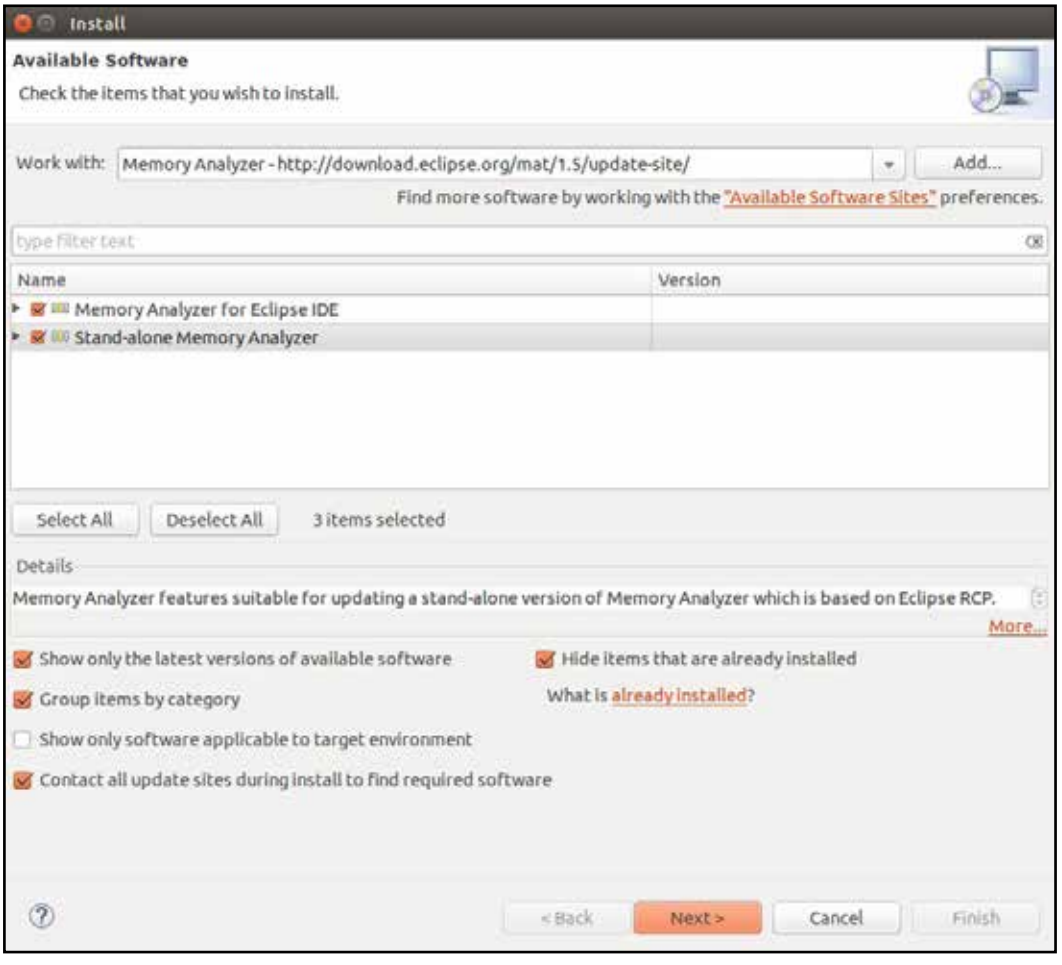


Figure 6.12: Installing Memory Analyzer Plugin

15. Open the saved HPROF file using Memory Analyzer (Click File → Open Heap Dump and select the converted 4mat.hprof file).

The output is as shown in figure 6.13.

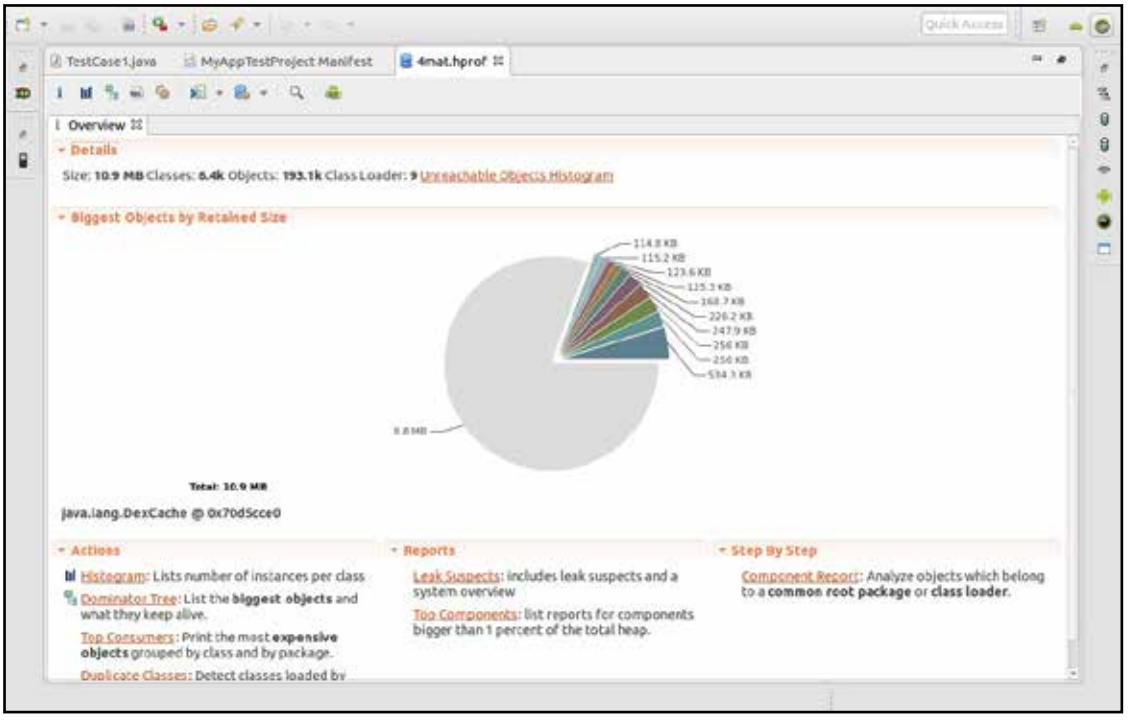


Figure 6.13: Saved HPROF File Opened in Memory Analysis Perspective

This will help the developer to view and detect the class using the memory most.

Following application displays leakage in memory. To develop the application, perform the following steps:

1. Start Eclipse.
2. Create a new project named AvoidingMemoryLeaks.

   Activity name: LeakedActivity
3. Navigate to src → com.example.avoidingmemoryleak folder.

4. Modify the code in LeakedActivity file as demonstrated in Code Snippet 7.

Code Snippet 7:

```
package com.example.avoidingmemoryleaks;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.app.AlertDialog;
mport android.content.DialogInterface;
public class LeakedActivity extends Activity {
        @Override
        protected void onCreate(Bundle savedInstanceState) {
            super.onCreate(savedInstanceState);
            setContentView(R.layout.activity_leaked);
            AlertDialog.Builder builder = new AlertDialog.
            Builder(this);
            builder.setIcon(android.R.drawable.ic_dialog_
            alert);
            builder.setMessage("This dialog leaks!").
            setTitle("Leaky Dialog")
            .setCancelable(false)
            .setPositiveButton("Ok", new
            DialogInterface.OnClickListener() {
            public void onClick(DialogInterface
            dialog, int which) {
            }
            });
            AlertDialog alert = builder.create();
            alert.show();
      }
```

```
        @Override
        public boolean onCreateOptionsMenu(Menu menu) {
            // Inflate the menu; this adds items to the action
bar if it is present.
            getMenuInflater().inflate(R.menu.leaked, menu);
            return true;
            }
}
```

Figure 6.14 displays the output when the code is executed.

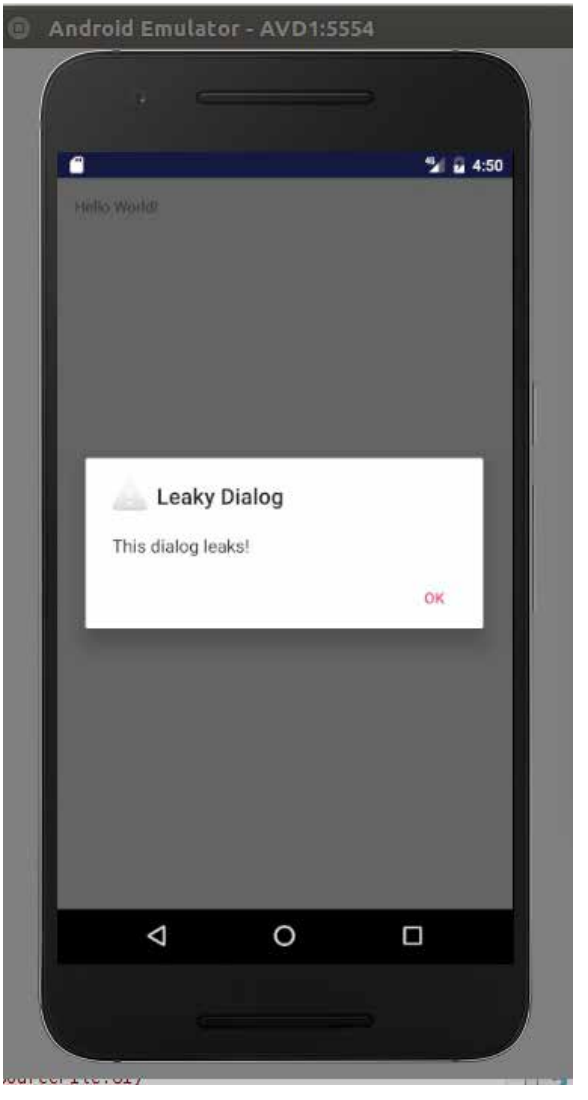


Figure 6.14: Leaky Dialog

While this application is running, take a look at LogCat in Eclipse by clicking Window → Show View → LogCat from the menu. It will display messages in green color.

While the dialog is displayed, rotate the device or the emulator by pressing Ctrl+F11. You will see red error text in LogCat as shown in figure 6.15.



Figure 6.15: LogCat Leaked Window Error

What actually happened here?

When the device rotates, the onPause and onStop Activity events are invoked. Then, a new Activity is created having a landscape orientation, and its onCreate() method is invoked.

Since, the dialog is displayed and has a reference in the Activity class, it cannot be garbage collected, and is subsequently leaked. In other words, an object is created that cannot be garbage collected.

To fix this issue, let us make the dialog a member variable of Activity class and override the onPause() method to dismiss it as demonstrated in Code Snippet 8.

Code Snippet 8:

```
AlertDialog _alert;

   @Override

   public void onPause() {

super.onPause();

if(_alert != null)

_alert.dismiss();

}
```

This will dismiss the dialog each time the Activity is destroyed and recreated and the error message will disappear from LogCat.

The LeakedActivity class will contain the code as demonstrated in Code Snippet 9.

Code Snippet 9:

```
package com.example.avoidingmemoryleaks;
import android.app.Activity;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.app.AlertDialog;
import android.content.DialogInterface;
public class LeakedActivity extends Activity {
      AlertDialog _alert;
      @Override
      protected void onCreate(Bundle savedInstanceState) {
            super.onCreate(savedInstanceState);
            setContentView(R.layout.activity_leaked);
            AlertDialog.Builder builder = new AlertDialog.
Builder(this);
            builder.setIcon(android.R.drawable.ic_dialog_
alert);
            builder.setMessage("This dialog leaks!").
setTitle("Leaky Dialog")
            .setCancelable(false)
            .setPositiveButton("Ok", new DialogInterface.
OnClickListener() {
            public void onClick(DialogInterface dialog,
            int which) {
            }
            });
            _alert = builder.create();
            _alert.show();
      }
```

```
	@Override
	public void onPause() {
	super.onPause();
	if(_alert != null)
	_alert.dismiss();
	}
	@Override
	public boolean onCreateOptionsMenu(Menu menu) {
		// Inflate the menu; this adds items to the action
bar if it		//is present.
		getMenuInflater().inflate(R.menu.leaked, menu);
		return true;
	}
	@Override
	public boolean onOptionsItemSelected(MenuItem item) {
		// Handle action bar item clicks here. The action
bar will
	// automatically handle clicks on the Home/Up button,
so long
		// as you specify a parent activity in
AndroidManifest.xml.
		int id = item.getItemId();
		if (id == R.id.action_settings) {
			return true;
		}
		return super.onOptionsItemSelected(item);
	}
}
```

6.8 Testing for Accessibility in Android

Testing the accessibility of an application for users with varying abilities is very important. Though an application is built following design and development guidelines, accessibility testing can showcase user interaction problems that may not be visible otherwise.

Accessibility testing, in general, should accomplish the following criteria in an application:

- Access to users with visual, physical, or age-related limitations.
- Use of only directional controls for navigation across all task workflows.
- Allow presentation of clear and specific feedback.
- Ensure that the following tests listed in table 6.3 are performed for accessibility testing of Android application.

Tests	Action
Directional controls	Accomplish primary tasks using directional controls only.
Talkback audio prompts	Receive clear and accurate audio descriptions when TalkBack is enabled. Talkback, is a pre-installed screen reader which uses spoken feedback to describe the results of an action.
Explore by Touch prompts	Have appropriate audio description when Explore by Touch prompts is enabled. Explore by Touch is a feature that accompanies TalkBack that reads what is touched on the device screen via spoken feedback. This feature supports users with low vision.
Size of Touch controls	Minimum of 48 dp (approx. 9 mm) in length and width.
Accessibility settings	Change device's display and sound options.
Functioning of Gestures	Whether app-specific gesture functions are used correctly or an alternative interface is required. App-specific functions include zooming images, scrolling lists, swiping between pages, navigating carousel controls, and so on.
No audio-only feedback	Secondary feedback mechanism, for example: sound alert accompanied by a visual alert to support users who are hard of hearing.
Repetitive audio prompts	Closely related controls not to repeat the same audio prompt.
Audio prompts overloading or underloading	Closely related controls provide appropriate level of audio information on a screen element for the user to understand and act.

Tests	Action
Special cases and consideration	Check special cases and considerations that apply to the application and take proper action for them. Special cases and consideration include test field hints, custom control with high visual context, custom controls and click handling, controls that change function, prompts for related controls, video playback and captioning, supplemental accessibility audio feedback, custom controls with complex visual interactions, sets of small controls, decorative images and graphics, and so on.
Prompts for controls that change function	Controls that have multiple functions provide audio prompts appropriate to their current function.
Video playback and captioning	Indicate availability of captioning for a video play back controls and specify a clear way to enable them.

Table 6.3: Accessibility Testing Checklist

Check Your Progress

1. Android Testing Framework is based on?

(A)	Selinium	(B)	JUint4
(C)	Sahi	(D)	JUnit3

2. Which element in the AndroidManifest.xml file is used to specify the application to be tested?

(A)	activity	(B)	intent-filer
(C)	application	(D)	instrumentation

3. Which of the following options is used to test the Android TestCase class for correctness of <application> element in AndroidManifest.xml file?

(A)	InstrumentationTestCase	(B)	Build/manifests files and directory structure
(C)	Automatically assign InstrumentationTestRunner	(D)	Test each part of an application

4. Which of the following is a functional testing TestCase class available in Android Testing Framework?

(A)	InstrumentationTestRunner	(B)	ActivityTestCase
(C)	ActivityUnitTestCase	(D)	ActivityInstrumentationTestCase2

5. Which of the following class can be used to assert views of an Android Application?

(A)	android.text.TextUtils	(B)	android.test.MoreAsserts
(C)	android.test.Asserts	(D)	ViewAsserts

Answers

1.	D
2.	D
3.	A
4.	D
5.	D

Summary

- An application uses ActivityUnitTestCase<Activity> base class for an isolated test of a single Activity. An application uses the ActivityInstrumentationTestCase2 to test the function of a single activity.
- The ProviderTestCase2 tests content providers in an isolated environment.
- ServiceTestCase class is used for controlling the application and Services that are to be tested.
- The RenamingDelegatingContext is a mock class that mocks files and database operations.
- Android has a built-in automated UI testing tool that consists of the uiautomatorviewer GUI tool and uiautomator Java library.
- uiautomatorviewer GUI tool scans and analyzes the GUI components of the application and the uiautomator Java library provides the necessary APIs for automated, customized UI tests and an execution engine to automate and run them.
- To detect memory consumption, one should use Memory Analysis view for the saved HPROF file.

ENHANCED LEARNING EXPERIENCE

Onlinevarsity

